

딥러닝과 LLM을 활용한 현대적 자연어 처리

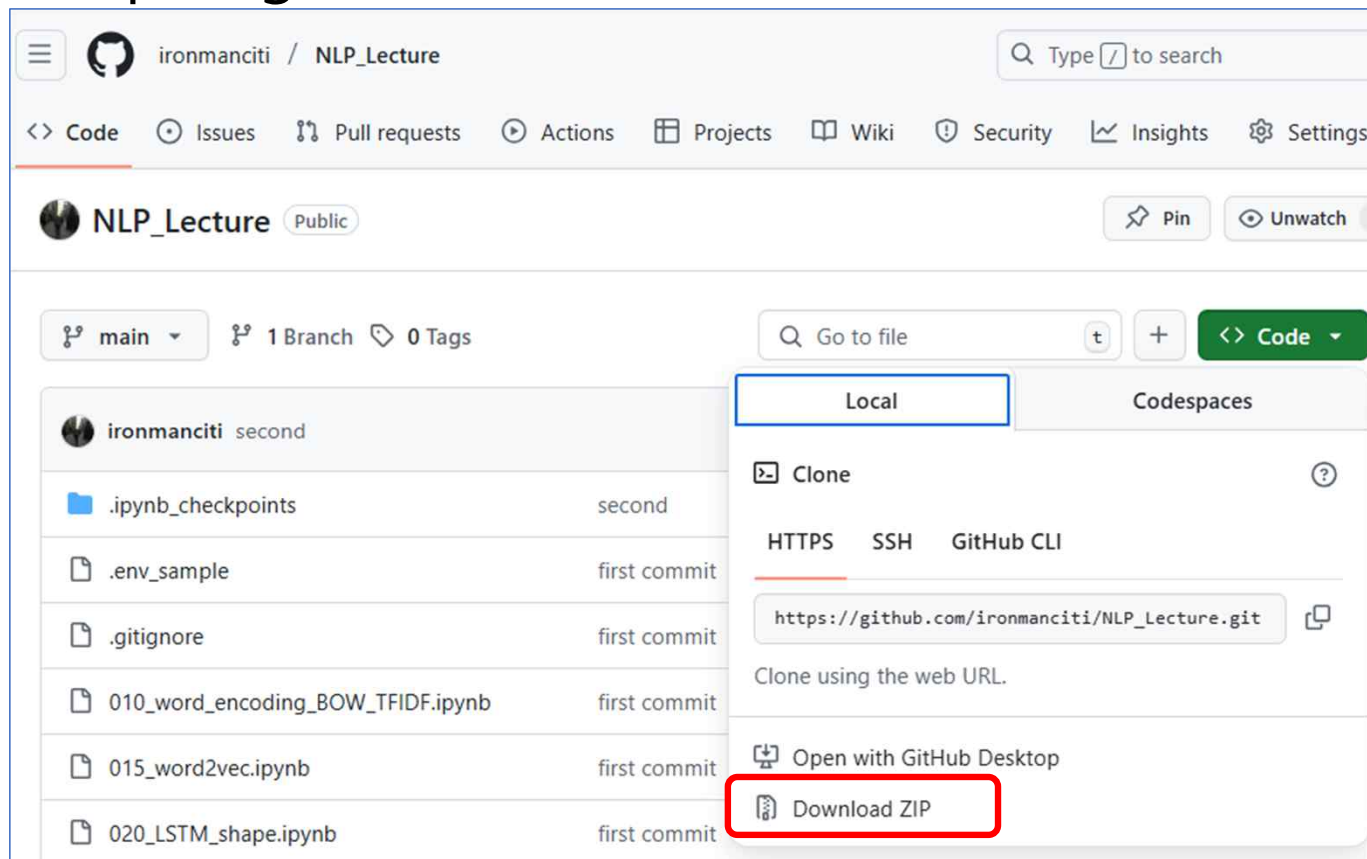
강사: 오영제

교육 환경 준비

- 인터넷 연결
- Chrome Browser
- Gmail ID for Colab

Github Repository 에서 실습 code download

https://github.com/ironmanciti/NLP_Lecture

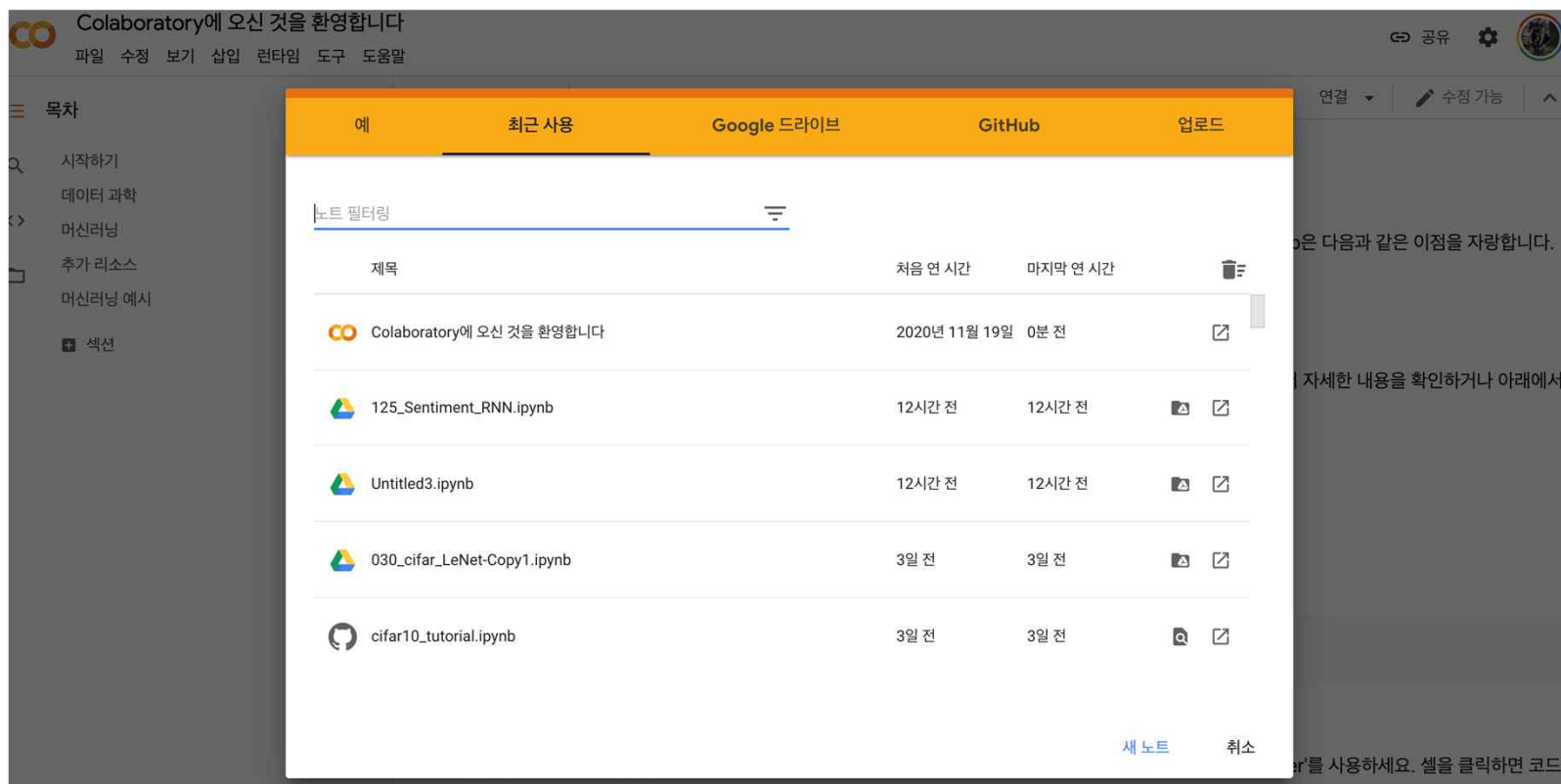


Google Colaboratory 소개

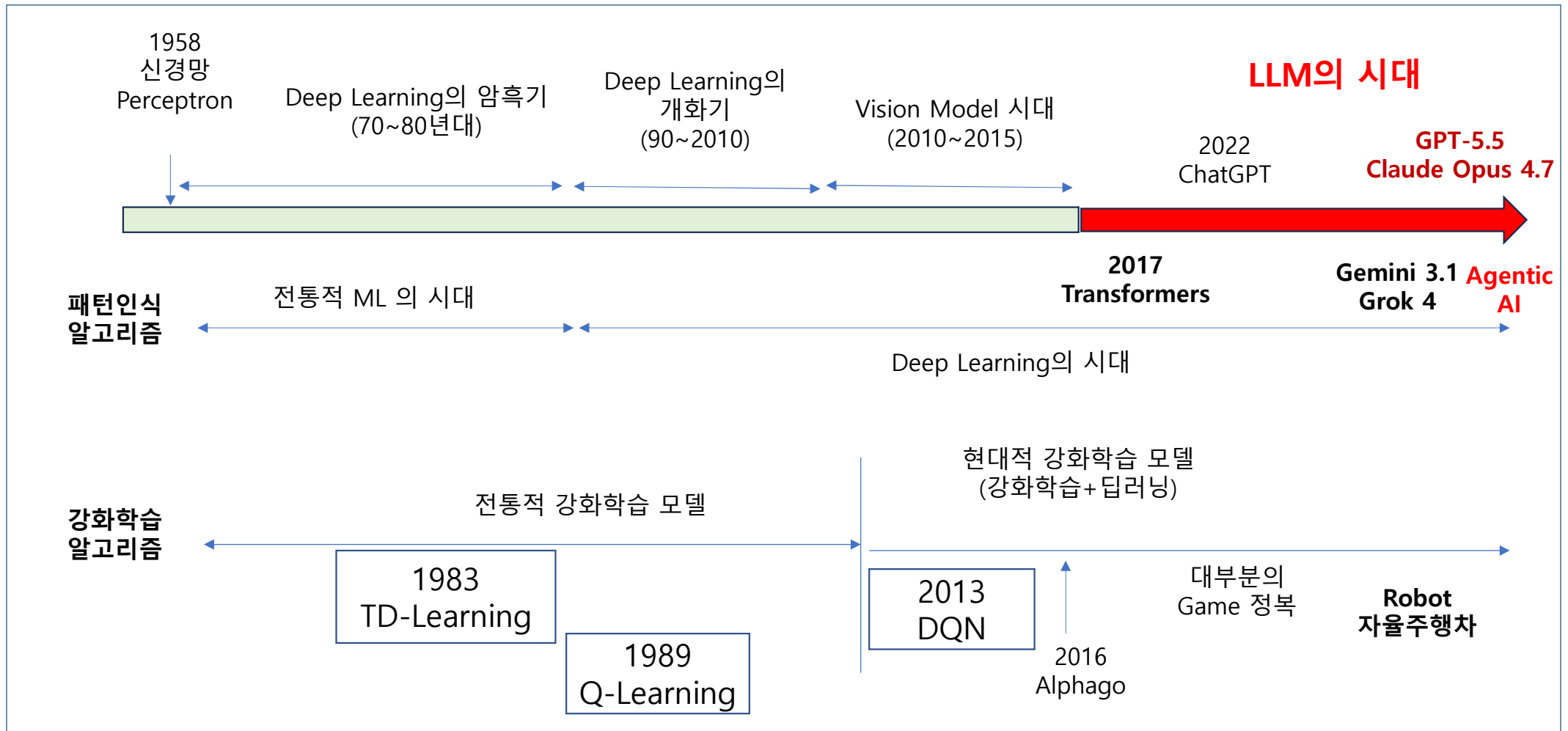
- Free GPU 제공
- Google Drive 와 연동
- Jupyter Notebook 환경
- Deep Learning beginner 를 위한 최적의 환경
- 각종 snippet 제공

Google Colaboratory 사용하기

<https://colab.research.google.com/>



AI 발전 History



Natural Language Processing

자연어 처리(Natural Language Processing)

- 자연언어 : 일반 사회에서 자연히 발생한 언어
 - 한국어, 영어, 일본어 등
- 인공언어 : 프로그래밍 언어, 에스페란토어
- 자연어 처리
 - 자연어를 컴퓨터가 해독하고 그 의미를 이해하는 기술

NLP 응용분야

Information
Extraction



Sentimental
Analysis



Image
Captioning



Video
Captioning

Advertisement
Matching



Speech
Recognition



Text Generation



Text
Summarization



Spell
Checking

Chatbot



Keyword
Search

Machine
Translation



Topic Modeling



Q&A

NLP 에 대한 접근 방법 (Language Modeling)

예전 방식 (Data-Driven Approach)

2010 년대 이전 전통적 방식

- 언어 전문가에 의한 고품질 sample data 확보가 중요
- **규칙기반 (Rule-Based)**
언어학을 기반으로 한 rule-based program
- **통계기반 (Statistical Machine Translation)**
 - 말뭉치 (Corpus) 를 기반으로 통계 모델 및
 - 전통적 machine learning 적용

2010 년대 딥러닝 방식

- **Deep Learning 기반**
 - RNN을 이용한 어순, 단어의 의미, 문맥 등을 스스로 학습

최근 방식 (Large Language Model)

LLM (Large Language Model) 이용

- **Transformer 기반의 대규모 사전학습**
- **언어 전문가 불필요 / 태스크별 학습 데이터 최소화**
 - Transformer Architecture
 - RLHF (Reinforcement Learning Human Feedback)
 - PEFT / LoRA / QLoRA (효율적 Fine-tuning)
 - RAG (Retrieval-Augmented Generation)
 - Function Calling / Tool Use / MCP
 - Multimodal LLM, AI Agent / Multi-Agent

최근의 NLP 발전 과정

- 2014 – RNN based sequence-to-sequence model 제안 (조경현 외)
- 2017 – **Transformer** (Google 연구진, "Attention is all you need") 논문 발표
- 2018. 6 – OpenAI GPT (Generative Pre-Training)
- 2018.11 – Google's BERT (Bidirectional Encoder Representations from Transformers)
- 2019. 2 – OpenAI GPT-2
- 2020. 5 – OpenAI GPT-3, 1750억 개의 parameter size
- 2022. 11 – ChatGPT (chatting version of GPT-3)
- 2023 – GPT-4, BARD
- 2025 – GPT-5, Gemini, Claude **BIG, BIGGER, MORE BIGGER**
- 2026 – **GPT-5.5, Claude Opus 4.7, Gemini 3.1** – **Agentic AI 시대 본격화**

NLP(자연어처리)의 기본 recipe

- Corpus (말뭉치, collection of texts)
 - 자연어 분석 작업을 위해 만든 샘플 문서 집합
 - 단순히 소설, 신문 등의 문서를 모아 놓은 것. 혹은 품사, 형태소 등의 보조적 의미를 추가하여 구조적인 형태로 정리해 놓은 것 포함
- 토큰 (token)
 - 자연어 문서를 분석하기 위해 긴 문자열을 작은 단위로 나눈 것
 - Tokenize
 - 문자열을 여러 개의 조각, 즉 여러 개의 Token(토큰, 단어)들로 쪼개는 것
 - 특수 token : <START>, <EOS>, <UNK>, <PAD>, etc

- Text / sentence (문장)
 - Sequence of words
- Words (단어, 한글의 경우 형태소)
 - Sequence of meaningful characters
 - Word 는 영어의 경우 space 나 구두점(, ;) 으로 구분
 - 한국어, 일본어의 경우는 간격 없이 사용해도 읽을 수 있는 특징
 - 한글 형태소 분석기 or Sub-word 기법 사용

Tokenization & Embedding

Tokenization 기본 개념

1. Tokenization 정의

- 입력 text를 의미를 가진 덩어리(Token: 단어, 구, 절, 문장 등)로 분할하는 작업

2. Vocabulary (어휘 사전)

- 모델이 인식할 수 있는 모든 토큰의 집합
- GPT-4: **cl100k_base** (약 10만 개 토큰)

3. Token ID (정수 인덱스)

- 각 토큰은 고유한 정수 ID로 매핑 (예: "Hello" → 9906, " world" → 1917)

4. Special Token & Chat Template

- <|begin_of_text|>, <|eot_id|>, <|system|> 등 → 대화 구조(system/user/assistant)를 모델에 전달

Tokenization 방법

1. 전통적 Tokenization의 한계

- **Word-level:** Vocabulary 폭발 + OOV(미등록어) → <UNK>
- **Character-level:** 시퀀스가 너무 길고 의미 단위 손실

2. Sub-word Tokenization (현대 LLM의 표준)

- 단어를 의미 있는 작은 조각으로 분해
- **BPE(GPT), WordPiece(BERT), SentencePiece(Llama·T5), Byte-level BPE 등**

3. 주요 라이브러리

- **NLTK:** sent_tokenize, word_tokenize (전통적 분리)
- **Hugging Face tokenizers:** BPE, WordPiece, SentencePiece 등 통합 제공 (현대 LLM 표준)

한국어 자연어 처리

1. 한국어 토큰화의 어려움

- 교착어 특성: 조사가 띄어쓰기 없이 바로 붙음 → 조사 분리 필요
- 띄어쓰기 불규칙: 실제 사용에서 띄어쓰기가 잘 지켜지지 않음
- 어순 자유: 어순이 의미 결정에 결정적이지 않음

2. 과거: 형태소 분석기 기반

- **KoNLPy** (사전식): Komoran, Mecab, Okt 등 내장
- 카카오 **Kharii** (CNN 기반 형태소 분석기)

3. 현재: Sub-word Tokenization 표준

- 형태소 분석기 없이 end-to-end 학습 가능
- 대표 알고리즘: **BPE, WPM, SentencePiece**

BPE 알고리즘 동작 원리

1. BPE(Byte Pair Encoding)

- 1994년 제안된 데이터 압축 알고리즘 → 토큰화에 응용
- 훈련 데이터의 단어를 글자(character) 또는 유니코드 단위로 분해해 초기 vocabulary 구성
- 가장 자주 등장하는 인접 쌍을 하나의 토큰으로 반복 병합 → 최종 sub-word 사전 완성

2. 토큰화 예시

- **입력:** Jet makers feud over seat width with big orders at stake
- **결과:** _J et _makers _fe ud _over _seat _width _with _big _orders _at _stake
- 희귀어 "Jet", "feud"는 sub-word(J+et, fe+ud)로 분해됨 → OOV 해결

Sub-word Tokenization 예 - WPM

1. WPM((Word Piece Model)이란?

- 하나의 단어를 내부단어(sub-word) 단위로 통계 기반 분할하는 토큰화 기법
- 의미 있는 작은 단위들의 조합으로 단어를 표현 → 희귀어·신조어도 sub-word 조합으로 처리

2. 도입 배경

- 2015년 제안 → Google 번역기에 적용(2016)
- 이후 **BERT** 등 주요 LLM의 표준 토큰라이저로 채택

3. 동작 방식

- 입력 문장의 띄어쓰기는 언더바(_)로 치환 → 분할 후에도 원문 복원 가능
- **BPE**(Byte Pair Encoding)와 유사한 방식 사용 — BPE는 빈도 기반, WPM은 likelihood 기반 병합

Sub-word Tokenization 예 – Google SentencePiece

1. 핵심 특징

- 사전 토큰화 없이 단어 분리 → 언어에 종속되지 않음
- 영어·한국어 등은 의미 있는 단위로 분리 가능

2. 동작 방식

- **Unigram** 알고리즘 기반 sub-word 분할
- 띄어쓰기를 언더바(_)로 치환 (우측 예시 참고)

em _bed _ding
➡ embedding

3. 활용

- **Open Source**로 개방, 실무 적용 가능 (**한국어 적용 가능**)

실습 : 010-Tokenizers 실습

실습에서 다룰 4종의 토크나이저

1 Keras tokenizer

rule-based

공백·구두점 기준 분리
영어에 적합, 한국어에 취약

2 KoNLPy Okt tokenizer

사전 기반 형태소 분석

한국어 형태소 단위 분리
미등록어(OOV)에 취약

3 Google SentencePiece

subword (데이터로 학습)

사전 없이 학습
OOV 발생하지 않음

4 cl100k_base (tiktoken)

BPE (byte 단위)

GPT-4 · GPT-3.5-turbo
text-embedding-ada-002 에서 사용

rule-based → 사전 기반 → subword → BPE : 토크나이저의 진화 흐름

자율 실습 : 011 Tokenizers

토큰라이저 4종 비교 — 같은 문장이 어떻게 다르게 쪼개질까?

학습 목표

동일한 영어·한국어 문장을 4가지 토큰라이저에 넣어 결과를 비교하고, 영어/한국어 처리 차이와 사전에 없는 단어(OOV) 처리 방식을 직접 관찰한다.

과제 1

Keras Tokenizer (영어)

공백·구두점 분리
word_index 확인

과제 2

Keras Tokenizer (한국어)

조사 포함 통째 분리
교착어 한계 관찰

과제 3

KoNLPy Okt 형태소

사전 기반 형태소 분리
미등록어 OOV 확인

과제 4

SentencePiece 학습

NSMC 15만 문장으로
vocab 3만 모델 학습

과제 5

SentencePiece 토큰화

subword 분리
신조어도 OOV 없음

과제 6

tiktoken (BPE)

GPT 계열 인코딩
영/한 토큰 수 비교

핵심 관찰 포인트 : 영어 vs 한국어 처리 차이 · 사전 기반 vs subword/BPE · OOV 처리 방식

단어 (word) 와 문장의 숫자 표현



단어/문장 (string) 을 컴퓨터가 이해할 수 있는 숫자 (vector) 로 변환



How ?

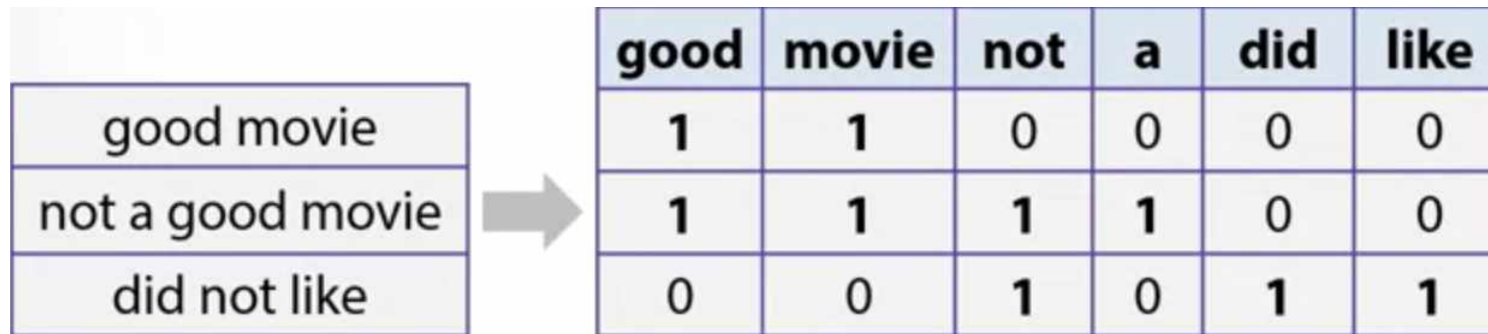
BOW

TF-IDF

Word Embedding

BOW (Bag of Words)

- 모든 문장을 토큰화 하고 각 문장에 토큰이 몇 번 등장하는지 count
- 각 token 을 feature 화 ➔ Text Vectorization



	good	movie	not	a	did	like
good movie	1	1	0	0	0	0
not a good movie	1	1	1	1	0	0
did not like	0	0	1	0	1	1

- Scikit-learn 의 nltk.CountVectorizer method 이용

BOW (Bag of Words) 의 문제점

- 단어들 간의 순서를 유지할 수 없음

→ n-grams 기법으로 일부 해결

bigrams 인접 단어의 쌍으로 구성 → ex) "is working" vs "not working"

trigrams 인접 세개 단어로 구성 → ex) "not an issue"

- Counter 가 normalize 되어있지 않음

→ TF-IDF 로 해결

- 단순히 단어가 나타나는 횟수만 count 하므로 "**not** an issue, phone is working" 과 "an issue, phone is **not** working" 을 같은 의미로 간주

n-grams : BOW 의 단어 순서 유지

- 1-gram : token 한 개
- 2-grams : token 두 개
- N-grams : token N 개

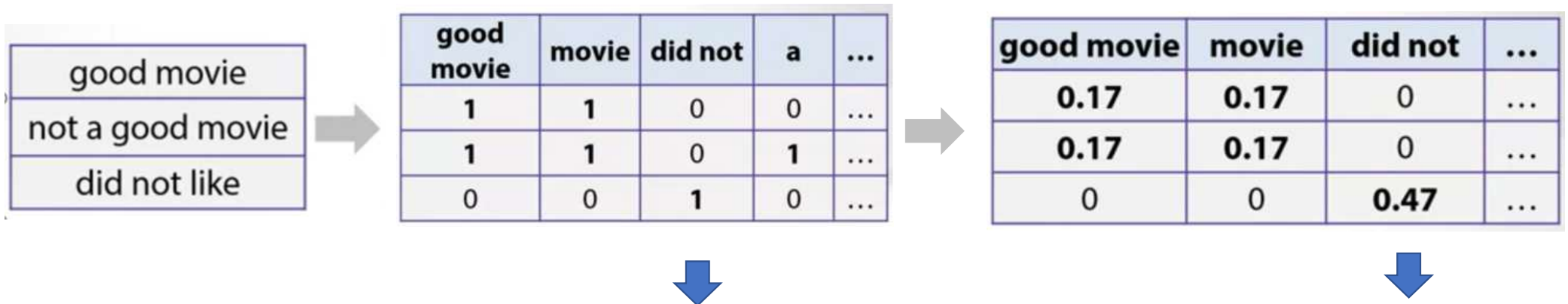
Corpus		good movie	movie	did not	a	...
Document 1	good movie	1	1	0	0	...
Document 2	not a good movie	1	1	0	1	...
Document 3	did not like	0	0	1	0	...

➡ 문제점 : feature 개수가 기하급수적으로 증가

TF-IDF (Term Frequency - Inverse Document Frequency)

- TF(단어 빈도, term frequency)
 - 특정한 단어가 문서 내에 얼마나 자주 등장하는지를 나타내는 값.
 - 이 값이 높을수록 특정 문서에서 중요하다고 간주
- DF(문서 빈도, document frequency)
 - 단어 자체가 전체 문서 집단 내에서 사용되는 빈도
 - DF 가 높으면 그 단어가 흔하게 등장한다는 것을 의미 (the, a, is, etc..)
- IDF(역문서 빈도, inverse document frequency) → DF 의 역수
- $TF\text{-}IDF = TF * IDF$
 - low TF-IDF - 전체 문서에서 공통으로 사용되는 단어임을 의미
 - high TF-IDF - 모든 문서가 아닌, 특정 문서에서 자주 사용되는 단어임을 의미

- TF-IDF 로 BOW 의 counter 를 대체
- row-wise normalize



“did not” 2-gram 은 document 3 에만 나타나므로 TF-IDF가 High

- Scikit-learn 의 TfidfVectorizer method 사용

One-Hot-Encoding 표현

Vocabulary 사전

a -1
aaron - 2
.
apple - 456
.
king - 4914
.
orange - 6257
.
queen - 7157
.
zebra - 9,999
<UNK> - 10,000

One-Hot encoding



해당되는 index 위치만 1

King Queen Apple Orange
(4914) (7157) (456) (6257)

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

10,000

One-Hot-Encoding

```
1 sentence = ["I really think this is amazing. honest."]
2 sequence = tokenizer.texts_to_sequences(sentence)
3 print(sequence)
4 print()
5 print(to_categorical(sequence))
```

```
[[5, 1, 9, 1, 10, 11, 1]]
```

```
[[[0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
  [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]]
```

One-hot-encoding 표현의 문제점

- 단어를 단순히 index 번호에 따라 one-hot-encoding 으로 vectorize 하므로 단어 간의 유사성을 파악 못함



해결책 : **Word Embedding**

- 단어/문장 간 관련도 계산
- 의미적/문법적 정보 함축
 - 전이 학습 가능

Word Embedding

- 숫자화된 단어의 나열로 부터 sentiment 추출
- 연관성 있는 단어들을 군집화하여 multi-dimension 공간에 vector 로 표시
→ 단어나 문장을 **vector space** 로 끼워 넣음 (**embedding**)
- 예를 들어, 호감(positive), 비호감(negative) 두가지 label 에 따라 관련 단어들을 두개의 category 로 군집화
 - ex) boring, bad, unfunny → negative
 - funny, good, interesting → positive

Word Embedding (Feature 화 표시)

dimension	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
성별	-1	1	-0.95	0.97	0.00	0.01
귀족	0.01	0.02	0.93	0.95	-0.01	0.00
나이	0.03	0.02	0.7	0.69	0.03	-0.02
음식	0.04	0.01	0.02	0.01	0.95	0.97



Man (5931) 의 4 dimension vector 표시



King (4914) 의 4 dimension vector 표시

Embedding matrix (example)

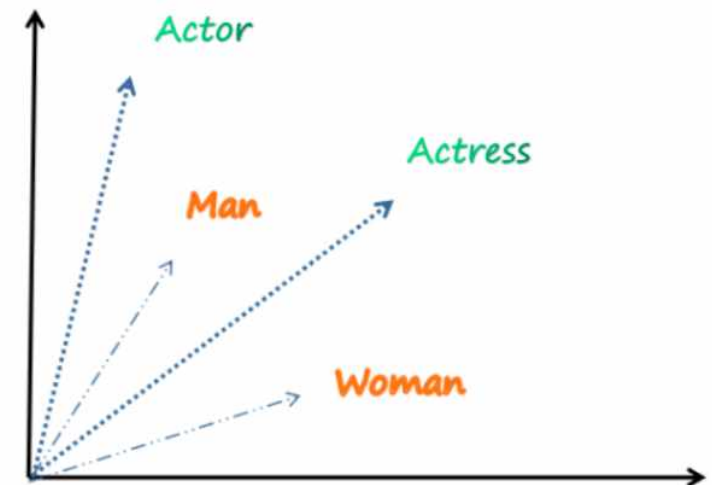
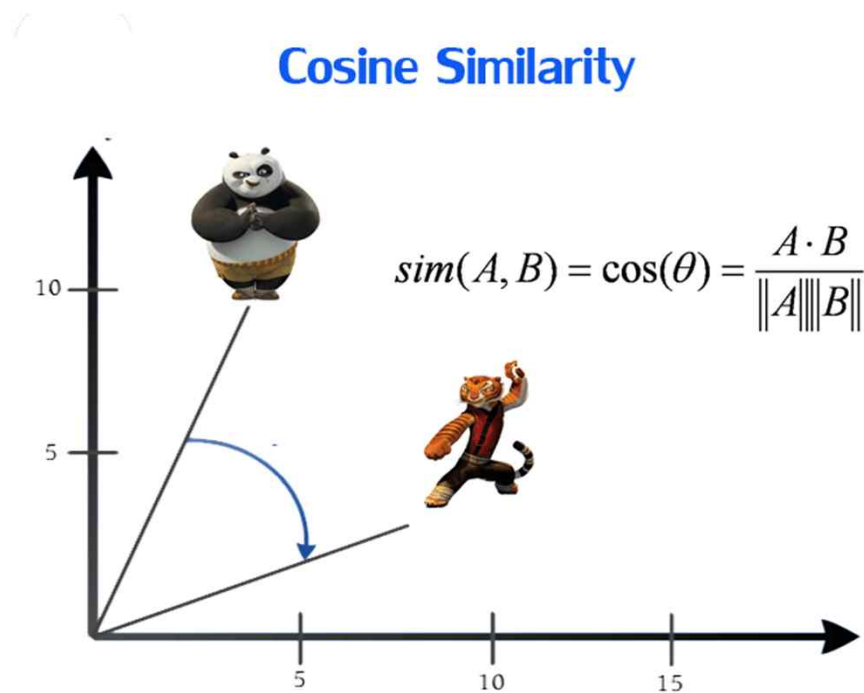
학습된 300 차원 features

words

	0	1	2	3	4	5	6	7	8	9	...	290	291	292	
fox	-0.348680	-0.077720	0.177750	-0.094953	-0.452890	0.237790	0.209440	0.037886	0.035064	0.899010	...	-0.283050	0.270240	-0.654800	0.105
ham	-0.773320	-0.282540	0.580760	0.841480	0.258540	0.585210	-0.021890	-0.463680	0.139070	0.658720	...	0.464470	0.481400	-0.829200	0.354
brown	-0.374120	-0.076264	0.109260	0.186620	0.029943	0.182700	-0.631980	0.133060	-0.128980	0.603430	...	-0.015404	0.392890	-0.034826	-0.720
beautiful	0.171200	0.534390	-0.348540	-0.097234	0.101800	-0.170860	0.295650	-0.041816	-0.516550	2.117200	...	-0.285540	0.104670	0.126310	0.120
jumps	-0.334840	0.215990	-0.350440	-0.260020	0.411070	0.154010	-0.386110	0.206380	0.386700	1.460500	...	-0.107030	-0.279480	-0.186200	-0.545
eggs	-0.417810	-0.035192	-0.126150	-0.215930	-0.669740	0.513250	-0.797090	-0.068611	0.634660	1.256300	...	-0.232860	-0.139740	-0.681080	-0.370
beans	-0.423290	-0.264500	0.200870	0.082187	0.066944	1.027600	-0.989140	-0.259950	0.145960	0.766450	...	0.048760	0.351680	-0.786260	-0.368
sky	0.312550	-0.303080	0.019587	-0.354940	0.100180	-0.141530	-0.514270	0.886110	-0.530540	1.556600	...	-0.667050	0.279110	0.500970	-0.275
bacon	-0.430730	-0.016025	0.484620	0.101390	-0.299200	0.761820	-0.353130	-0.325290	0.156730	0.873210	...	0.304240	0.413440	-0.540730	-0.038
breakfast	0.073378	0.227670	0.208420	-0.456790	-0.078219	0.601960	-0.024494	-0.467980	0.054627	2.283700	...	0.647710	0.373820	0.019931	-0.035
toast	0.130740	-0.193730	0.253270	0.090102	-0.272580	-0.030571	0.096945	-0.115060	0.484000	0.848380	...	0.142080	0.481910	0.045167	0.055
today	-0.156570	0.594890	-0.031445	-0.077586	0.278630	-0.509210	-0.066350	-0.081890	-0.047986	2.803600	...	-0.326580	-0.413380	0.367910	-0.265
blue	0.129450	0.036518	0.032298	-0.060034	0.399840	-0.103020	-0.507880	0.076630	-0.422920	0.815730	...	-0.501280	0.169010	0.548250	-0.318
green	-0.072368	0.233200	0.137260	-0.156630	0.248440	0.349870	-0.241700	-0.091426	-0.530150	1.341300	...	-0.405170	0.243570	0.437300	-0.461
kings	0.259230	-0.854690	0.360010	-0.642000	0.568530	-0.321420	0.173250	0.133030	-0.089720	1.528600	...	-0.470090	0.063743	-0.545210	-0.195
dog	-0.057120	0.052685	0.003026	-0.048517	0.007043	0.041856	-0.024704	-0.039783	0.009614	0.308416	...	0.003257	-0.036864	-0.043878	0.000
sausages	-0.174290	-0.064869	-0.046976	0.287420	-0.128150	0.647630	0.056315	-0.240440	-0.025094	0.502220	...	0.302240	0.195470	-0.653980	-0.295
lazy	-0.353320	-0.299710	-0.176230	-0.321940	-0.385640	0.586110	0.411160	-0.418680	0.073093	1.486500	...	0.402310	-0.038554	-0.288670	-0.245
love	0.139490	0.534530	-0.252470	-0.125650	0.048748	0.152440	0.199060	-0.065970	0.128830	2.055900	...	-0.124380	0.178440	-0.099469	0.008
quick	-0.445630	0.191510	-0.249210	0.465900	0.161950	0.212780	-0.046480	0.021170	0.417660	1.686900	...	-0.329460	0.421860	-0.039543	0.150

20 rows x 300 columns

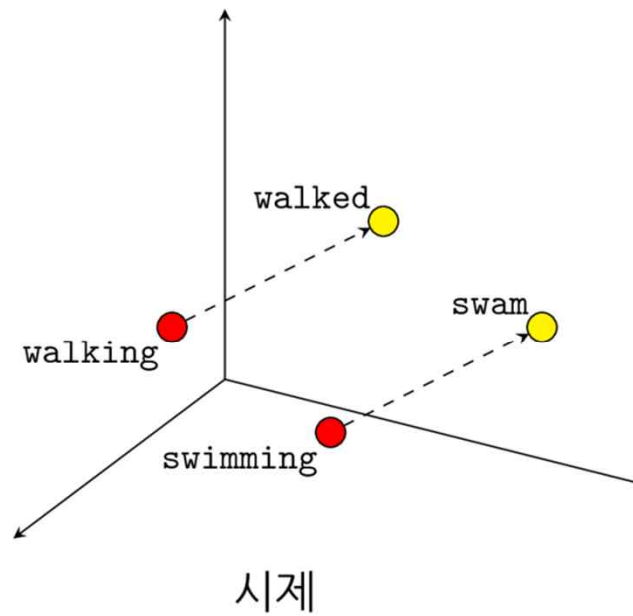
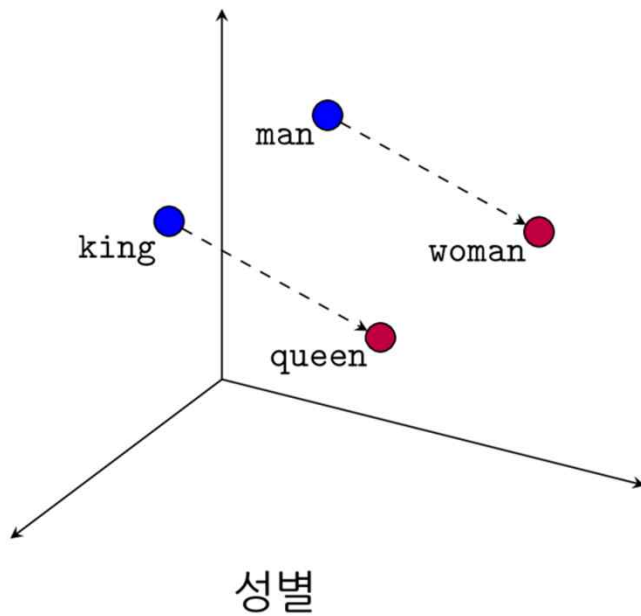
유사도 측정 (Cosine Similarity)



→ A, B 유사하면 ≈ 1
관련 없으면 ≈ 0

Word Embedding 의 결과

king - queen $\approx$ man - woman



국가-수도

<https://ronxin.github.io/wevi/>

실습 : 020-TFIDF_Embedding

문장을 숫자로 — 빈도 기반(TF-IDF) vs 의미 기반(Embedding) 비교

1 Vector 표현 방법 비교

텍스트 → 숫자

문장을 컴퓨터가 다룰 수 있는
vector 로 바꾸는 여러 방식 살펴보기

2 Sklearn BOW / TF-IDF

빈도 기반 표현

CountVectorizer 와 TfidfVectorizer 로
단어 빈도·중요도 가중치 비교

3 Embedding 모델로 변환

의미 기반 표현

사전학습 임베딩 모델(KURE-v1)로
문장 → embedding vector 변환

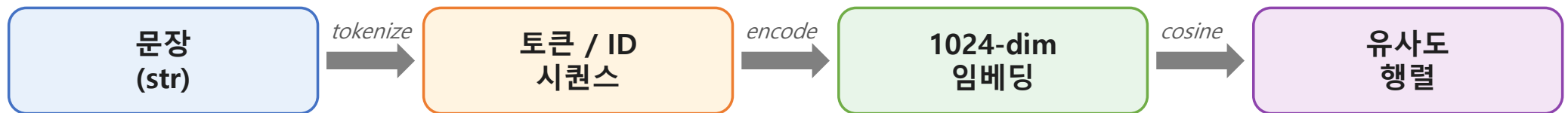
4 TF-IDF vs 임베딩 비교

유사도 테스트

의미는 같지만 표현이 다른 문장 쌍을
cosine 유사도로 비교 — 한계 관찰

핵심 : 빈도 기반은 단어 일치에만 강하고, 임베딩은 의미적으로 가까운 문장을 잡아낸다

자율 실습 : 021 문장에서 임베딩까지



▣ 과제 단계

①	모델 로드	<code>SentenceTransformer('nlpai-lab/KURE-v1')</code>
②	토큰 문자열	<code>tokenizer.tokenize(s)</code> → ['_삼성', '전자', ...]
③	토큰 ID	<code>tokenizer.encode(s)</code> → [0, ..., 2] (<s>, </s>)
④	원문 복원	<code>tokenizer.decode(ids)</code>
⑤	문장 임베딩	<code>model.encode(sentences)</code> → shape (4, 1024)
⑥	길이 비교	가변 토큰 수 → 고정 1024차원 (mean pooling)
⑦	유사도 계산	<code>cosine_similarity(embeddings)</code>

Transformers

NLP Mountain

ChatGPT/Gemini/Lama

2022

BERT/GPT/T5

"Attention is all you need"

Transformer

beginner → Can Start from here ?

2017

Attention

Transformer
이해에 필요한
사전 지식

Encoder-Decoder

Bi-LSTM

RNN

LSTM

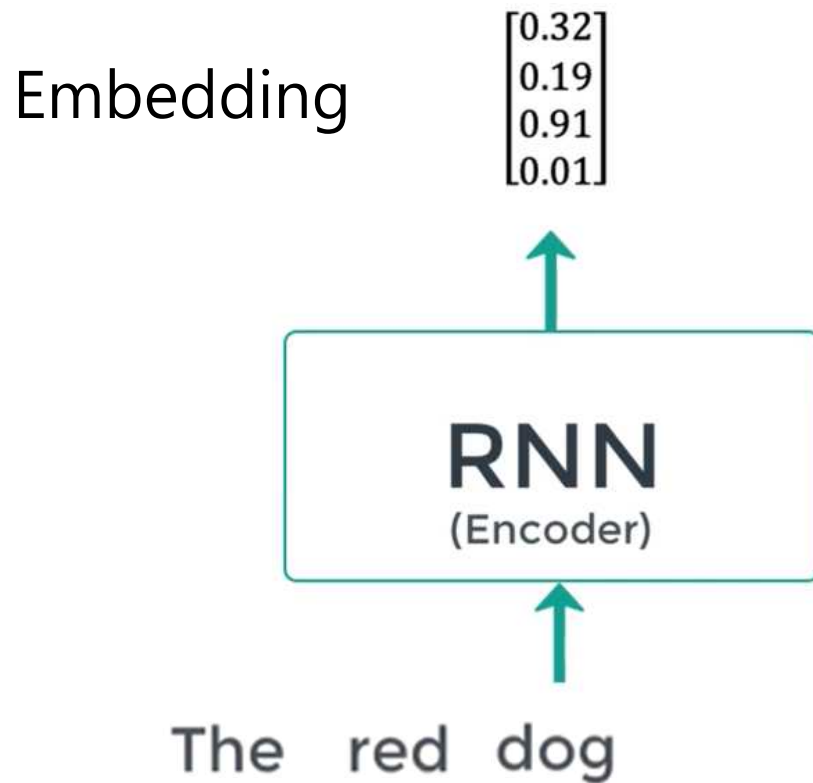
2014

NLTK, Konlpy, BOW, TF-IDF, HMM(Viterbi), CNN

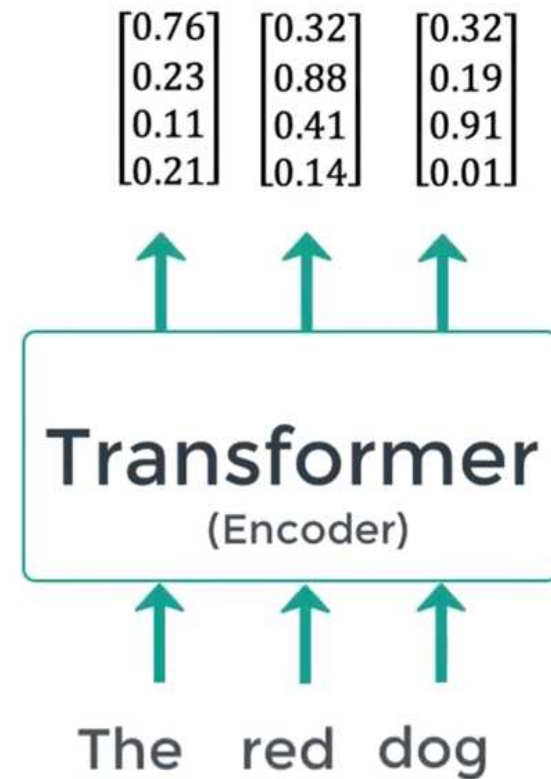
Transformer Model 의 특징

- RNN 혹은 CNN 을 이용하여 sequence-align 하지않은 최초의 **self-attention** model → "**Attention is all you need**"
- 기존의 **encoder-decoder 구조**를 유지
- **RNN 을 제거**함으로 **병렬처리** 가능 (GPU 효율 극대화)
- 1.6GB 의 WMT2014 English-German Translation task dataset 과 P100 GPU 8 개를 이용하여 3.5 일 train → BLEU score SOTA 달성

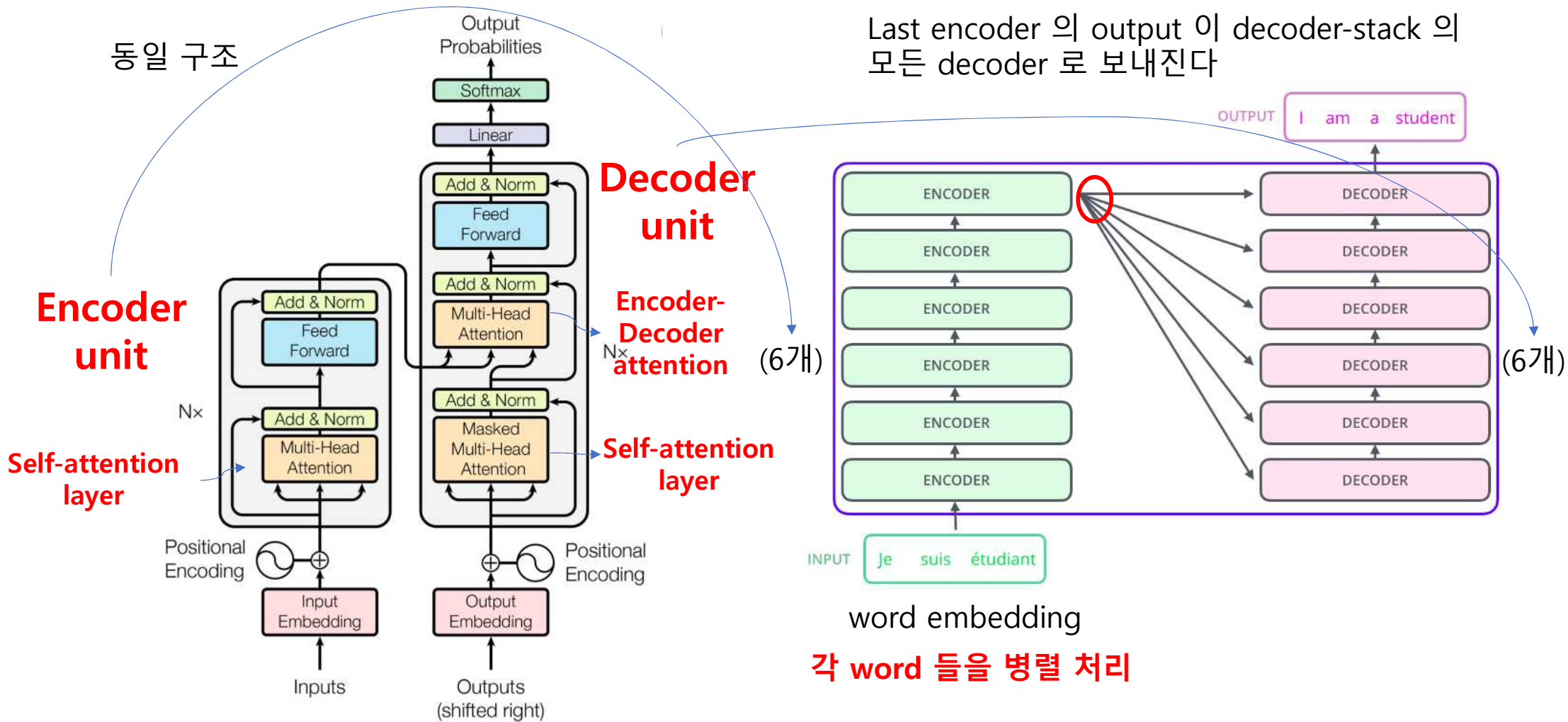
Word Embedding 은
time step 별로 순차적 생성



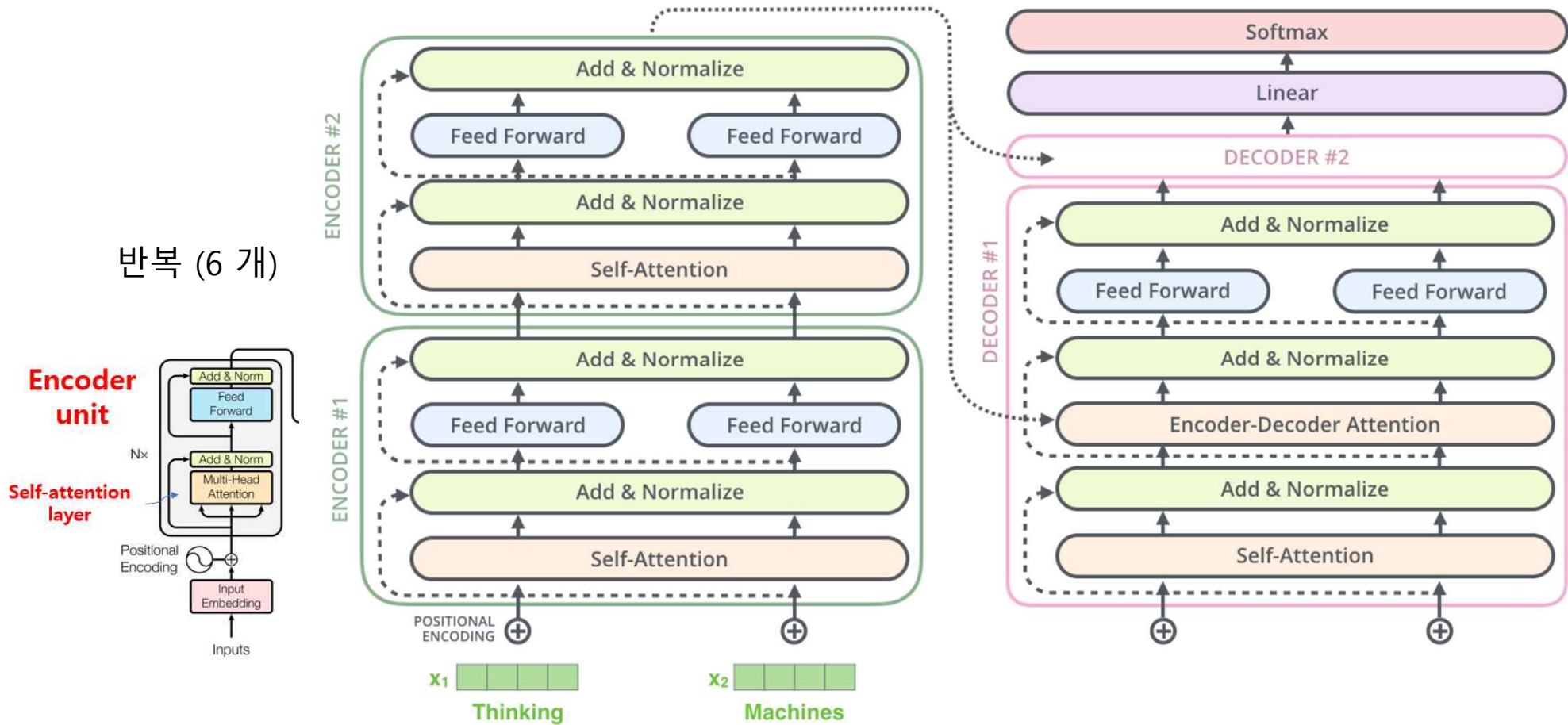
Word Embedding
동시 생성



Transformer Architecture



- 기존의 encoder-decoder 와 동일한 구조 유지



Self-Attention (Intra-Attention)

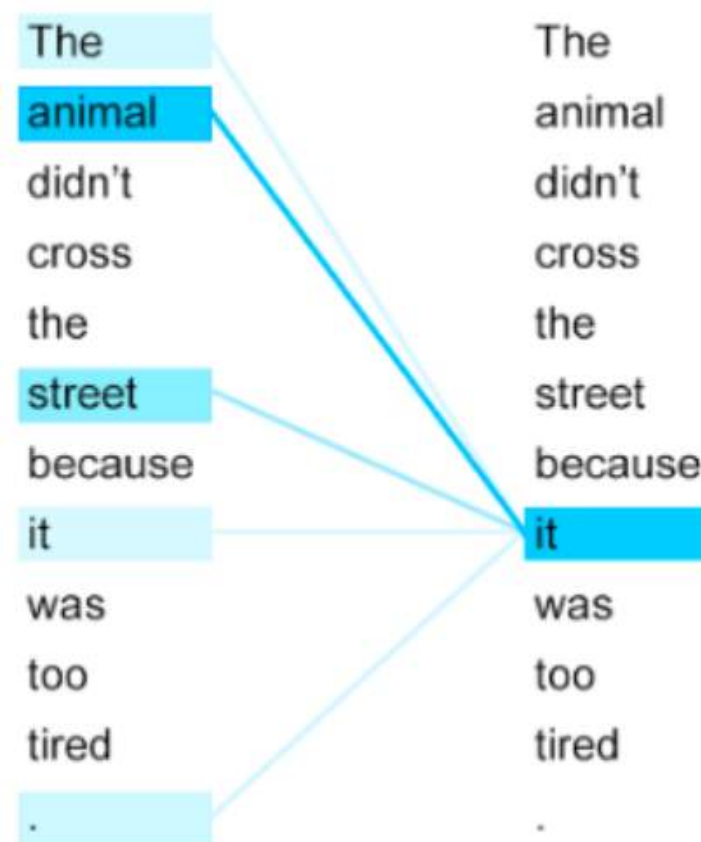
Attention 을 자기 자신에 대해 수행

→ 문장 내 단어들 간의 유사도를 구함

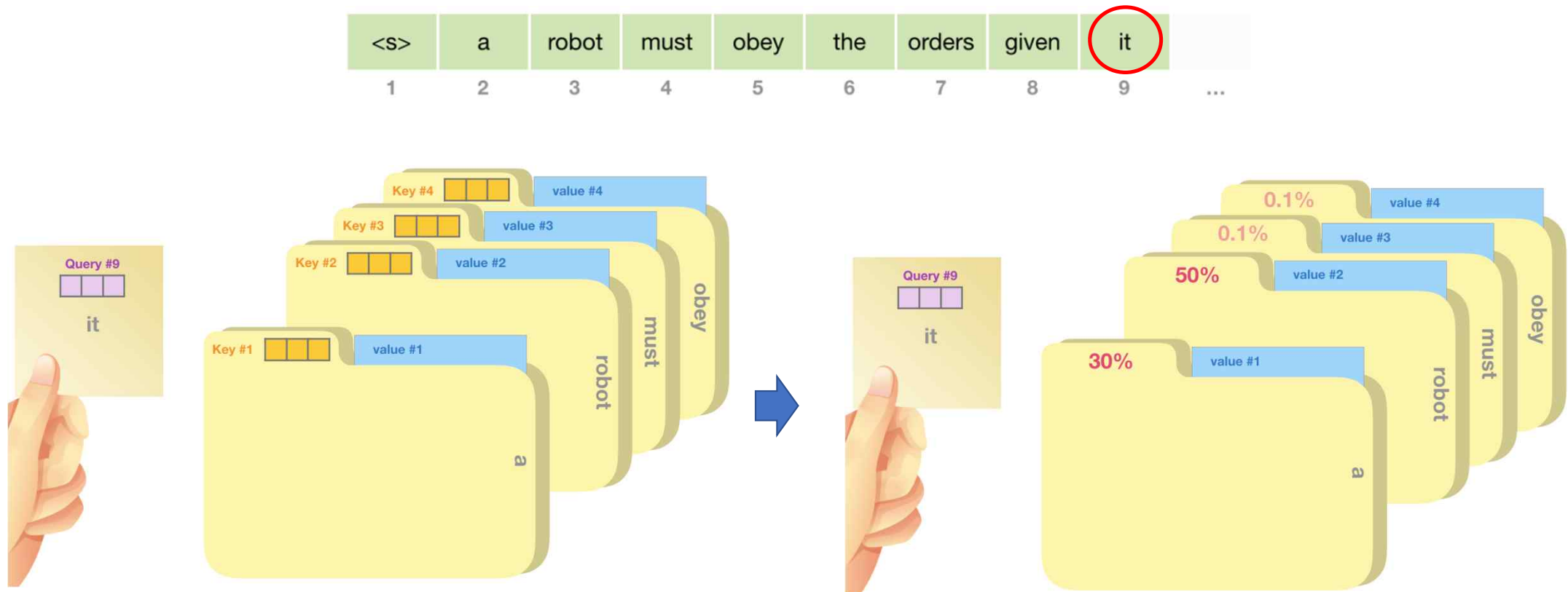
Self-attention 계산

3 개의 vector 필요 (훈련 과정에서 스스로 학습)

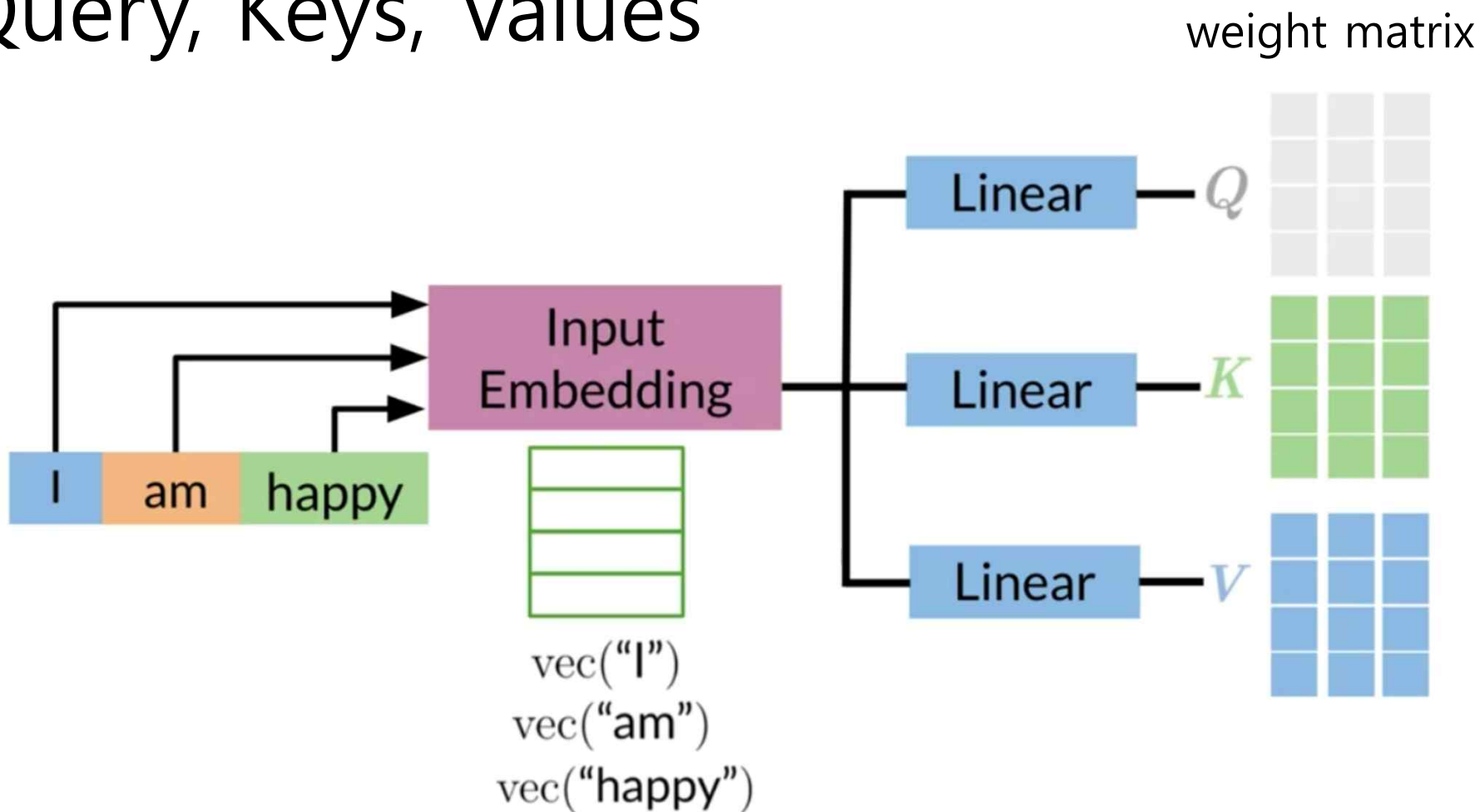
- Query Vector
- Key Vector
- Value Vector



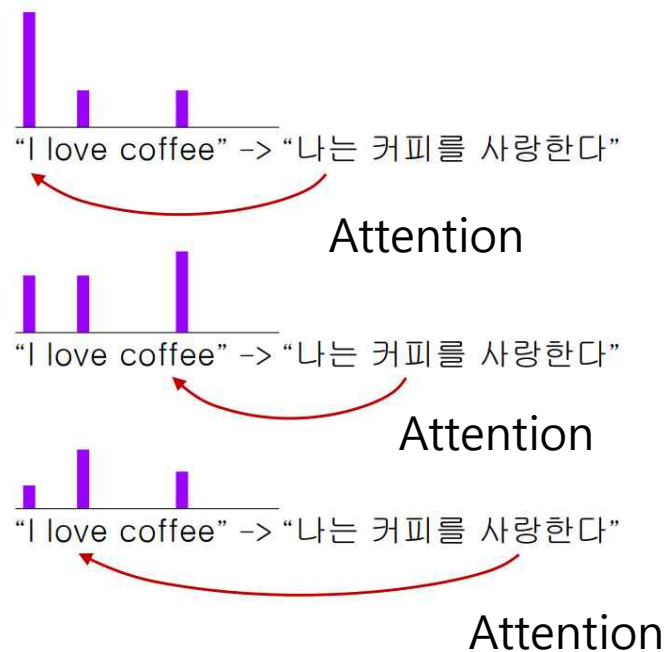
Self-Attention 과 Query, Key, Values



Query, Keys, Values



Simple Example of Attention



English:

How are you today?

한국어:

오늘 어떻게 지내세요?

Attention

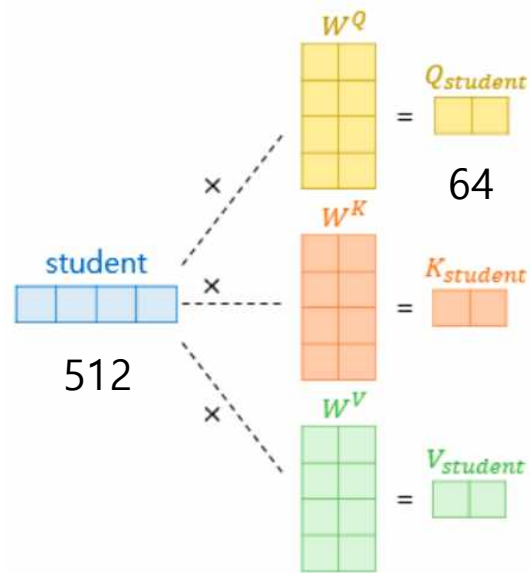
Attention

- 번역 시 각 target 단어(query)는 source 문장의 어떤 단어(key)에 집중할지 결정한다.
- 각 step마다 attention 분포가 다르게 나타난다 (왼쪽 그래프의 막대 분포).

Self-Attention 계산 (1~4 단계로 구성)

- 1 단계 - Q, K, V vector (projection vector) 생성 단계

각 단어 vector 를 $d_{model} / \text{num_heads}$ 차원의 vector 들로 변환 ($512 / 8 = 64$)



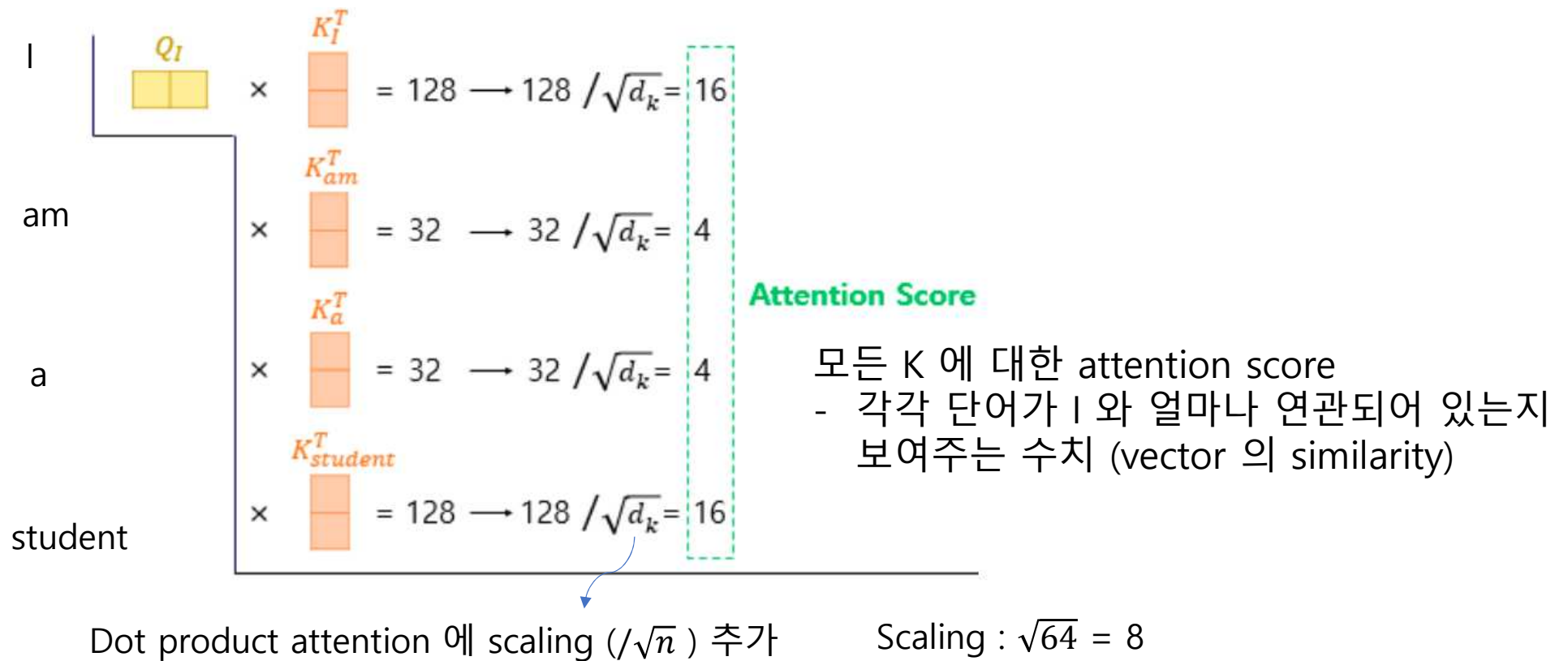
가중치 행렬 W_q, W_k, W_v 는 훈련과정에서 학습

Linear Layer

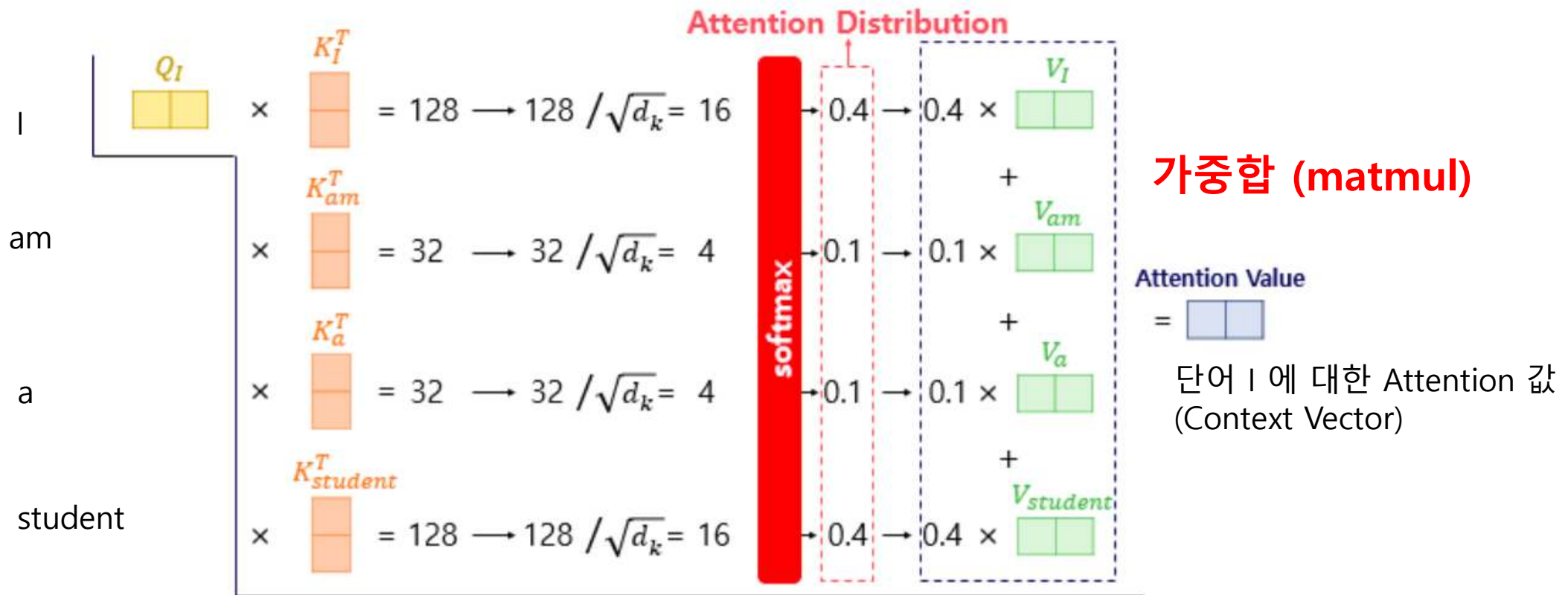
```
self.wq = tf.keras.layers.Dense(d_model)
self.wk = tf.keras.layers.Dense(d_model)
self.wv = tf.keras.layers.Dense(d_model)
```

- 2~4 단계 - Q, K, V 를 이용한 scoring 단계

- 2 단계 - 모든 K vector 에 대하여 attention score 를 구함
 - Scaled dot product Attention : $q \cdot k / \sqrt{n}$
 - 특정 단어에 대한 input sequence 내 모든 단어의 score 필요(얼마나 focus 할지 결정)

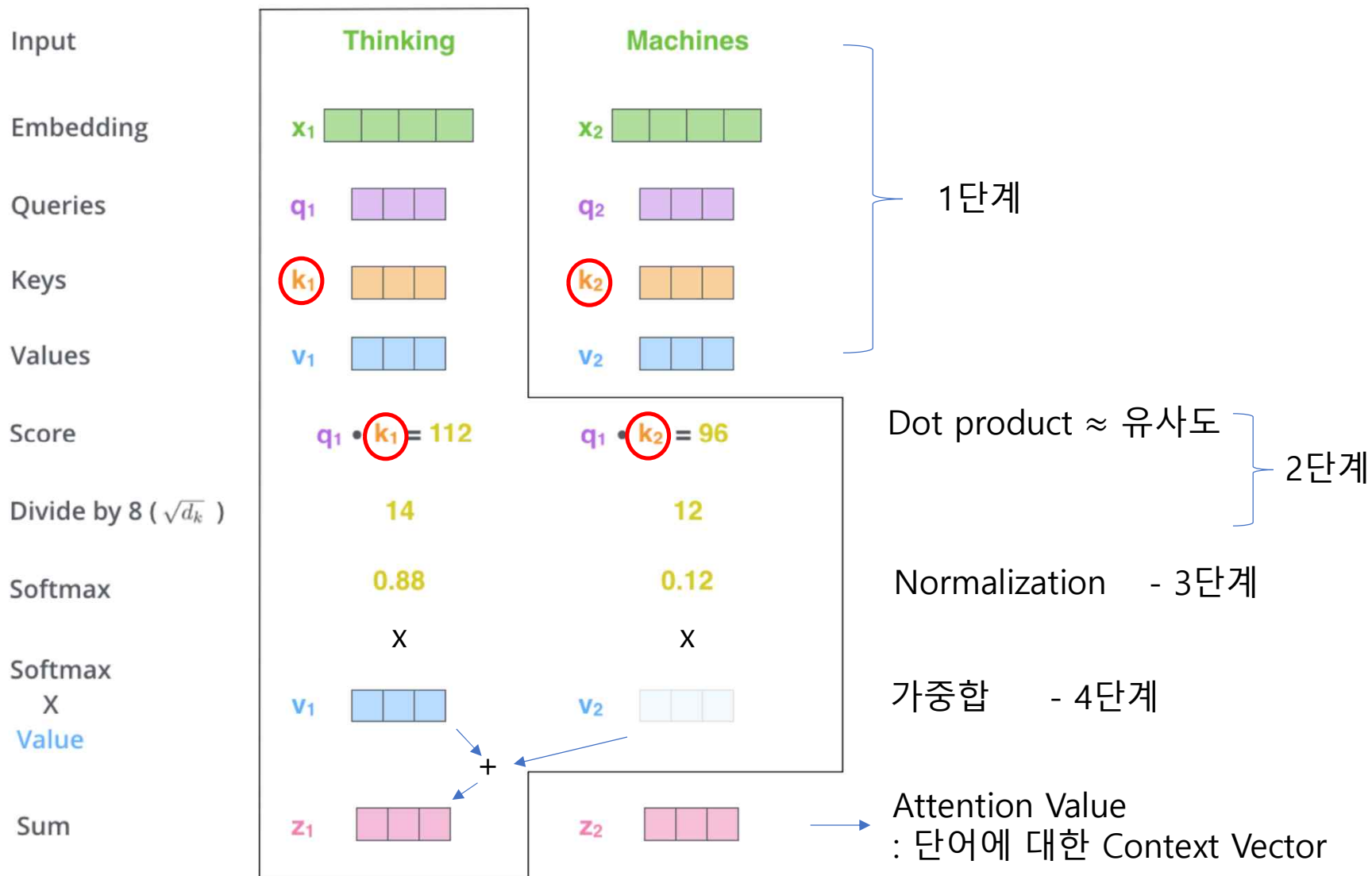


- 3 단계 - Softmax 로 Attention 분포를 구한 후
- 4 단계 - 이를 이용하여 V vector 를 가중합 하여 Attention Value 를 구함



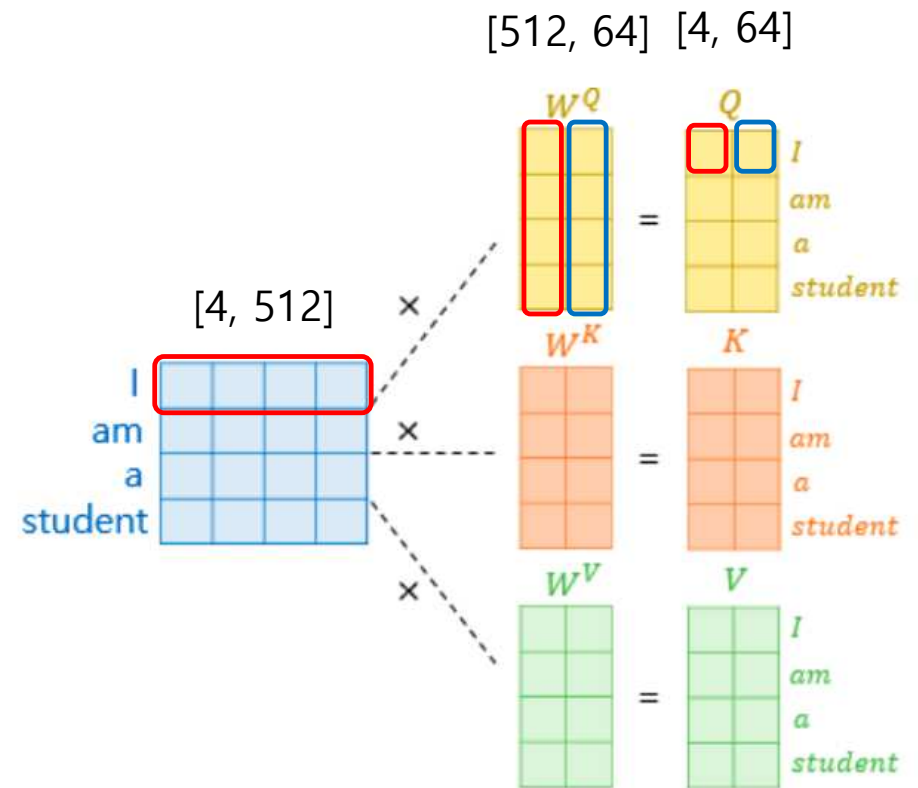
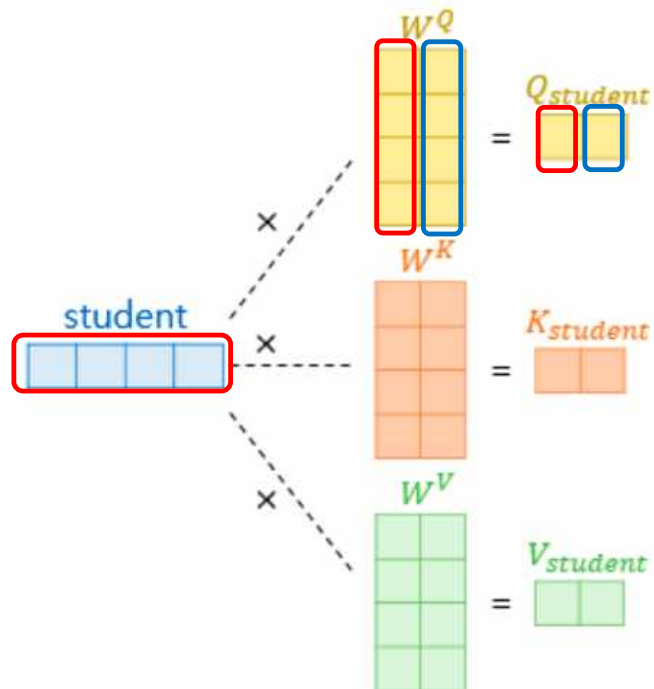
Self-Attention 계산

All-together

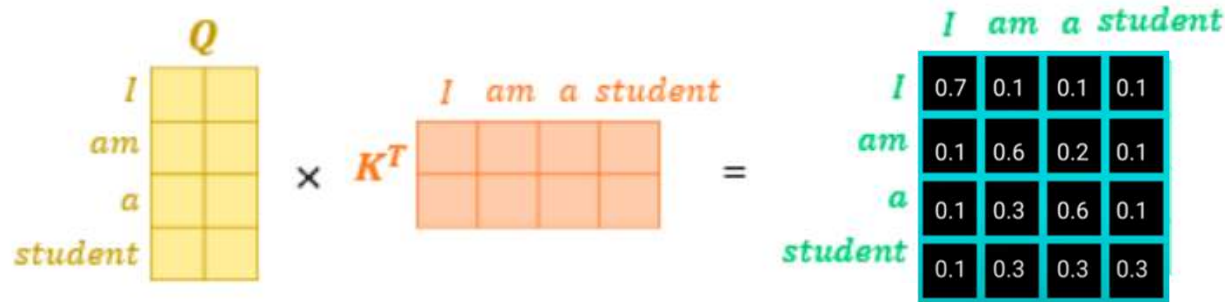


행렬 연산으로 일괄 처리

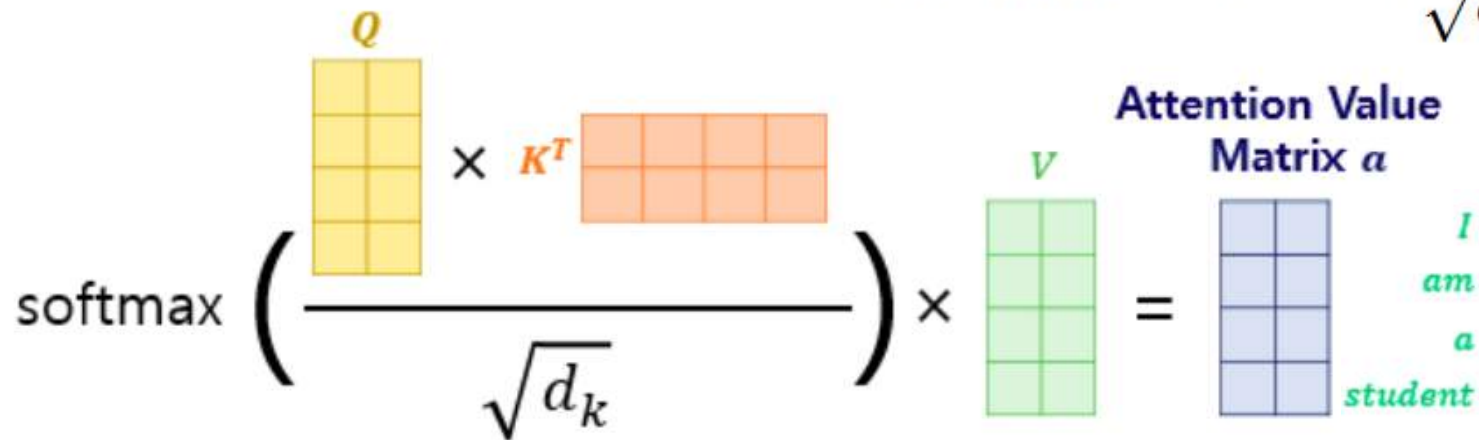
- 위의 과정을 Matrix 연산 처리



Scaled dot product attention formula



$$\Downarrow \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

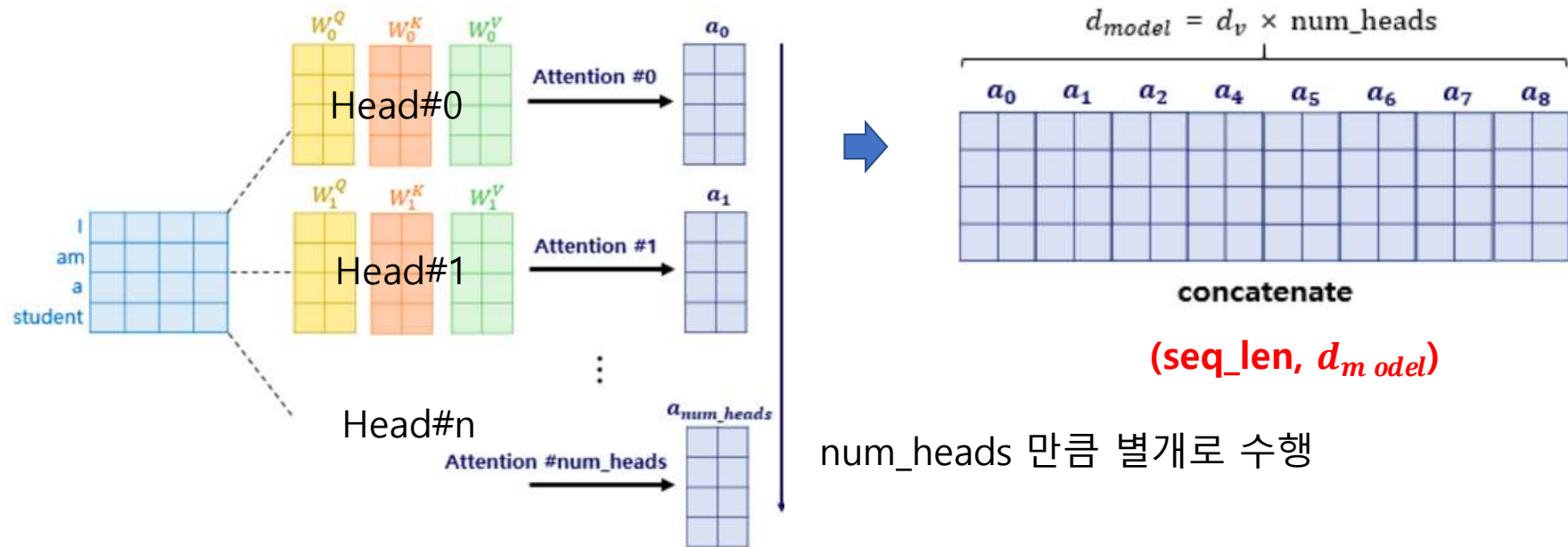


각 단어의 attention 값을
모두 가지는 attention 값
행렬 (seq_len, d_v)

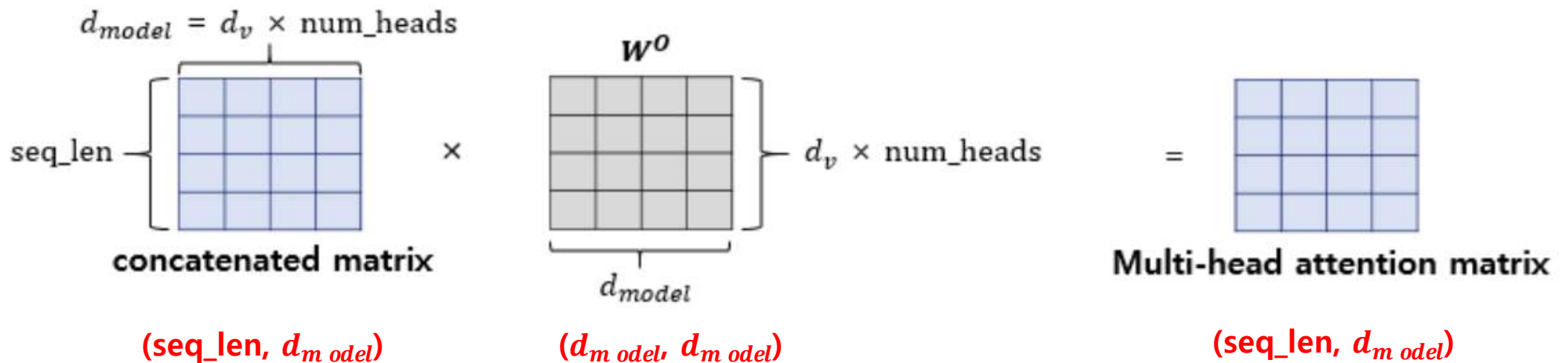
seq_len : 입력문장의 길이
 d_v - V vector 의 크기

Multi-Head Attention

- 여러 개의 attention 을 병렬로 사용한 후 Attention Head 를 연결
→ 다른 시각으로 단어 간의 상관 관계 파악
- 각각이 random 하게 초기화 되므로 training 후 다른 subspace 표시

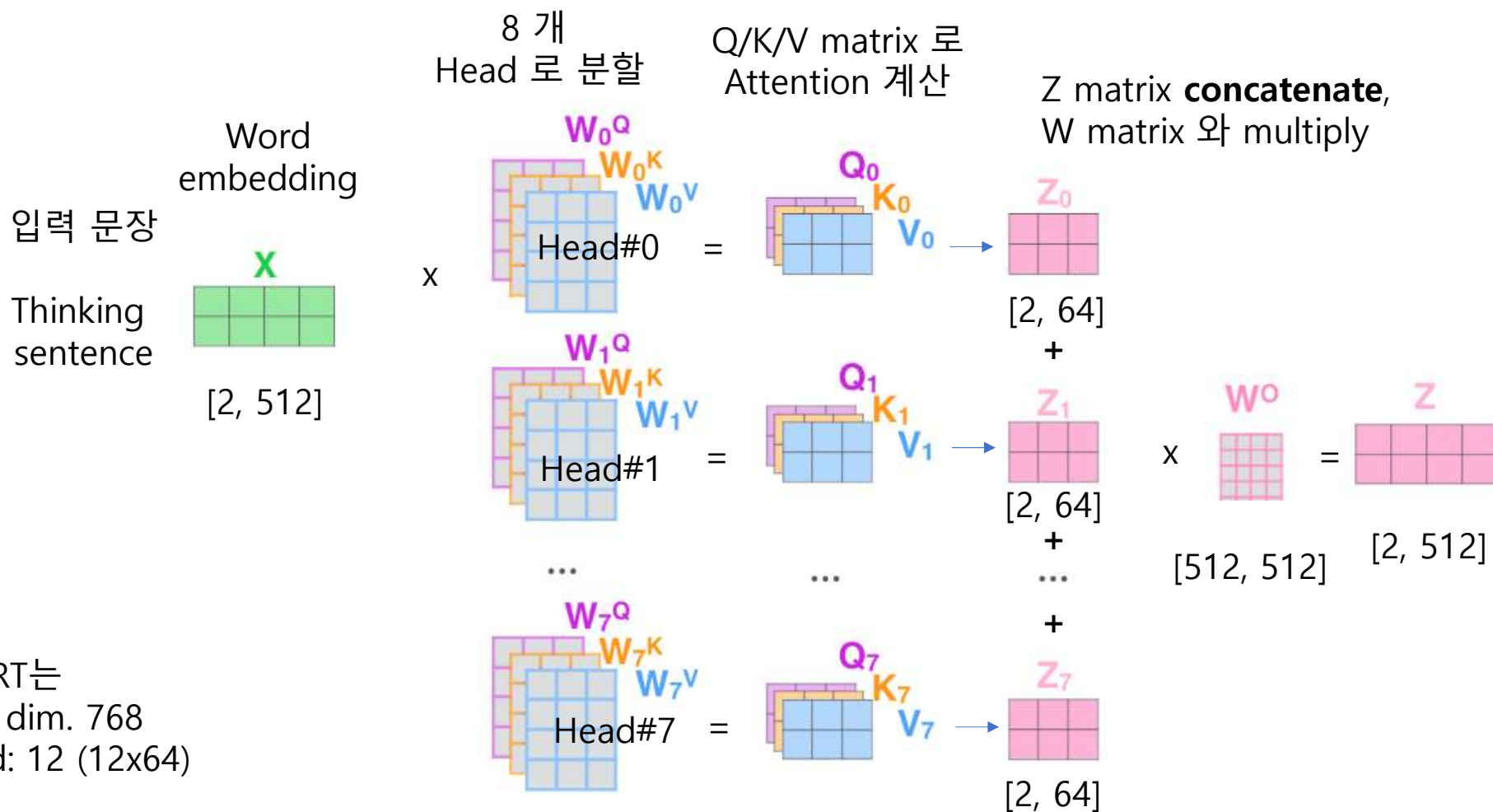


Multi-Head Attention output matrix



- Multi-Head Attention 의 크기는 Encoder 의 최초 입력의 크기와 동일
- Transformer 는 Encoder 를 6 개 쌓은 형태이므로, 입력의 크기가 출력에서 유지되어야 다음 Encoder 의 입력으로 사용 가능

All together

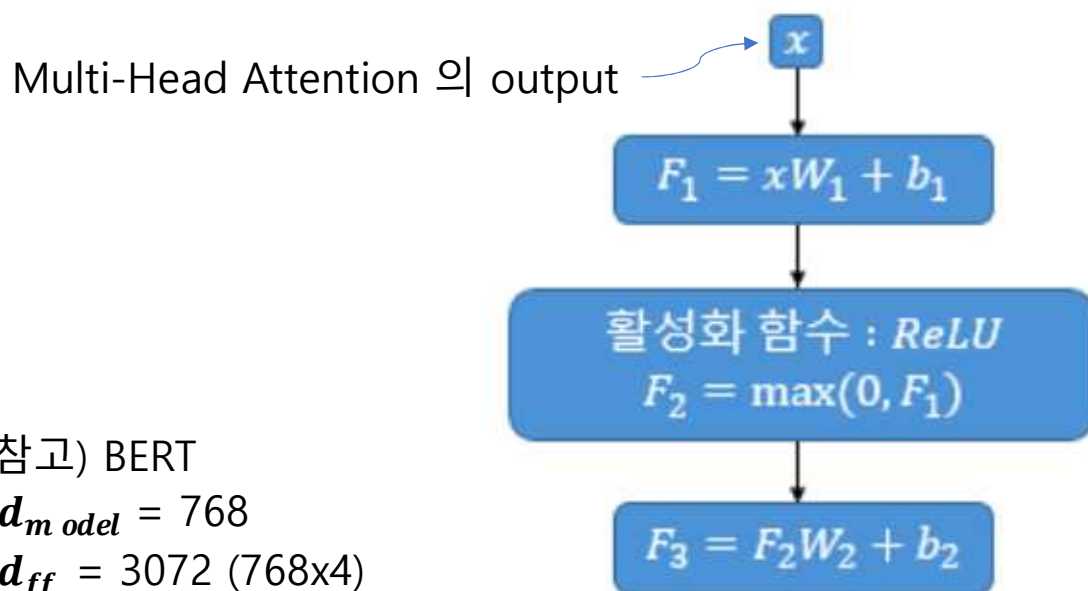


Position-wise Feed Forward NN

- Encoder 와 Decoder 의 각각의 layer 에서 보유
- Position 별 (단어별)로 별도로 적용하므로 position-wise

- Fully-Connected FFNN

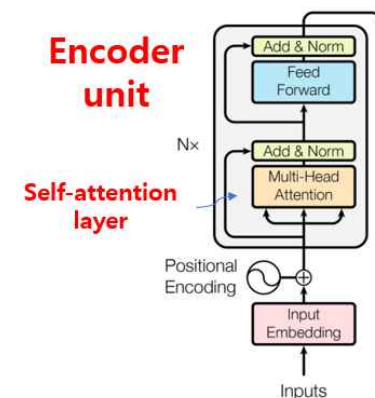
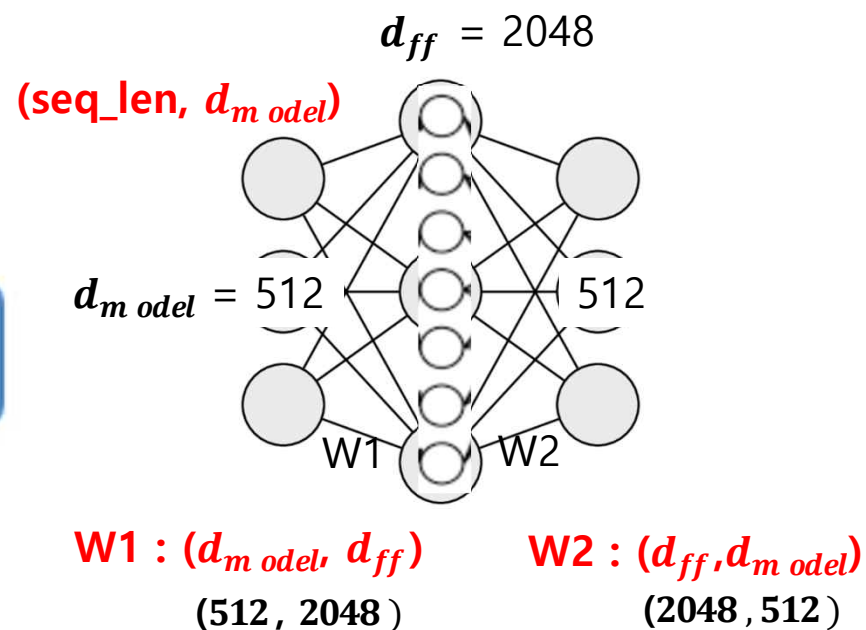
$$FFNN(x) = MAX(0, xW_1 + b_1)W_2 + b_2$$



참고) BERT

$d_{model} = 768$

$d_{ff} = 3072 (768 \times 4)$



(참고) Position-wise Feed Forward NN 구현

(batch_size, seq_len, dff)



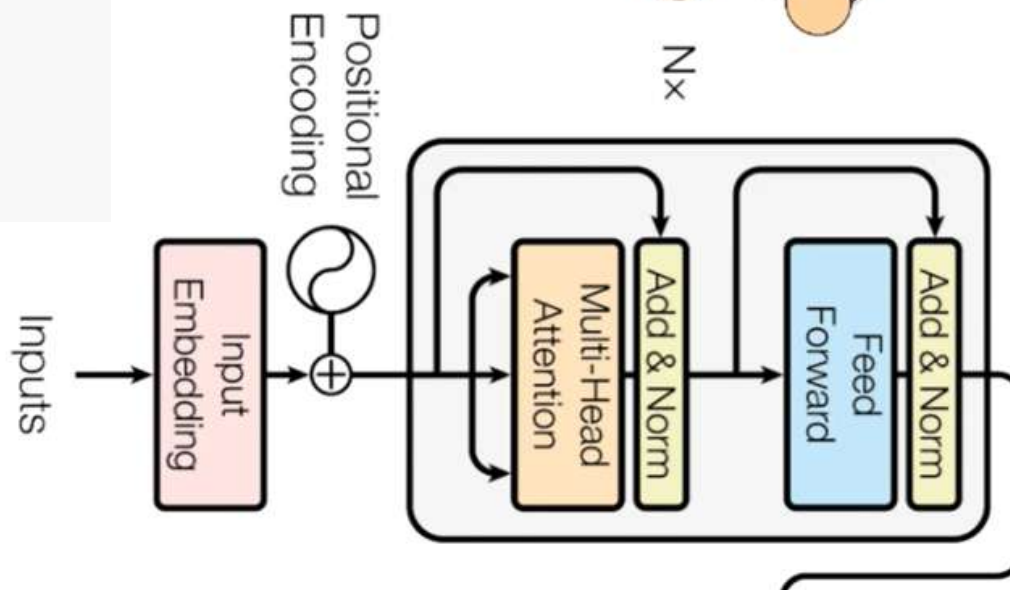
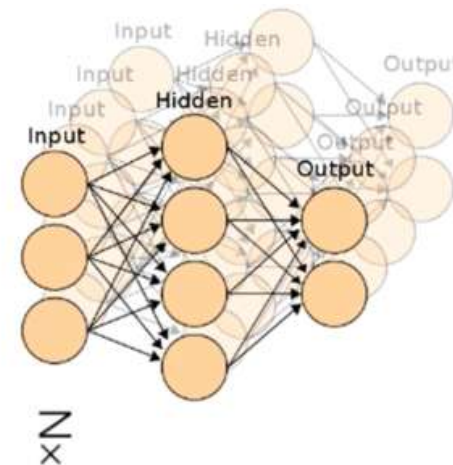
```
def point_wise_feed_forward_network(d_model, dff):  
    return tf.keras.Sequential([  
        tf.keras.layers.Dense(dff, activation='relu'),  
        tf.keras.layers.Dense(d_model)  
    ])
```



(batch_size, seq_len, d_model)

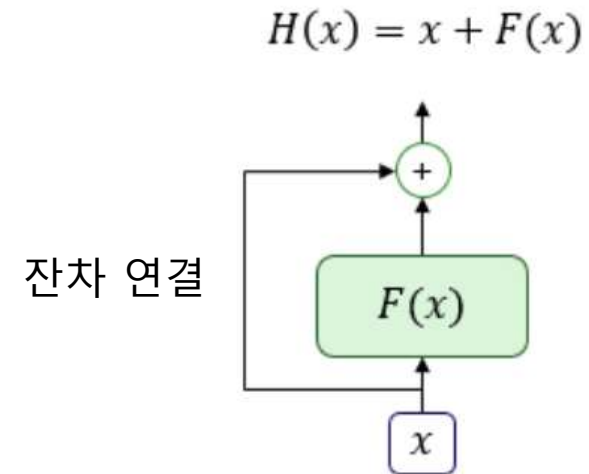
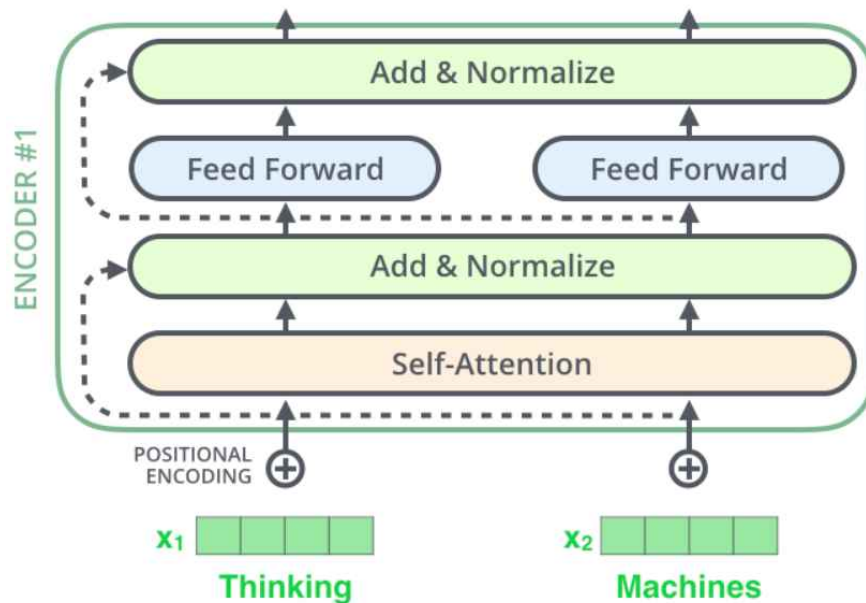
The big red dog

$\begin{bmatrix} 0.711 \\ 0.04 \\ 0.07 \\ 0.18 \end{bmatrix}$



잔차 연결 및 정규화 (Residual Connection and Layer Normalization)

- Add & Norm – 트랜스포머는 서브층의 입력과 출력이 동일한 차원을 유지하므로 잔차 연결 가능 → Vanishing Gradient 해결



Positional Encoding

- 위치 정보 특성 해결
- Embedding vector 에 positional encoding 값 추가

- $PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{model}})$ → 짝수 위치

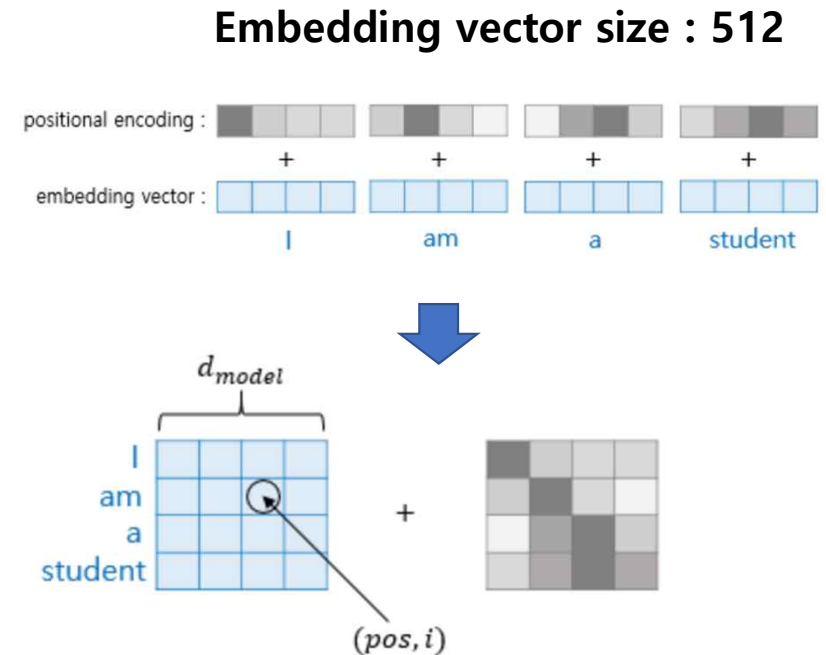
- $PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{model}})$ → 홀수 위치

pos – 입력문장 내의 embedding vector 의 위치

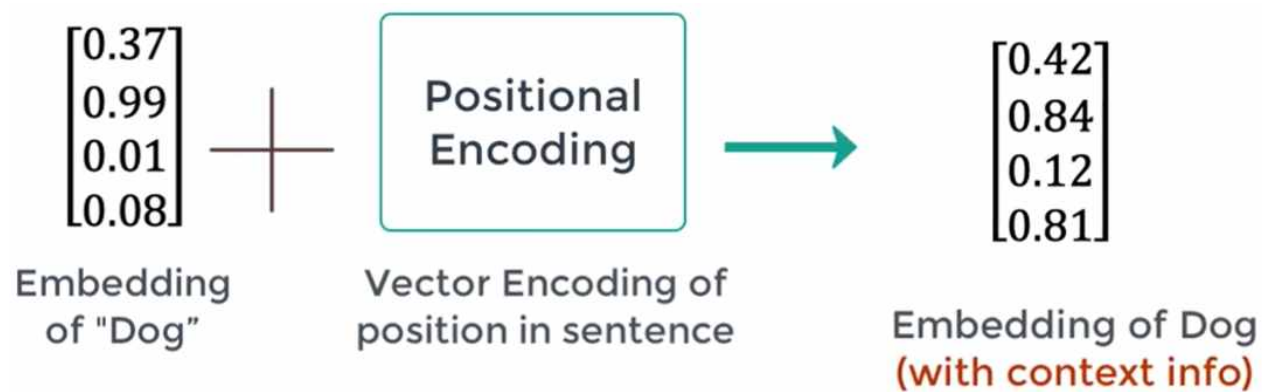
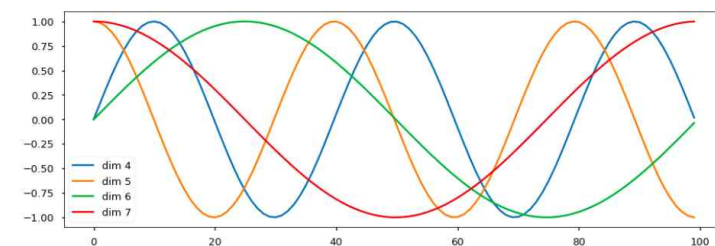
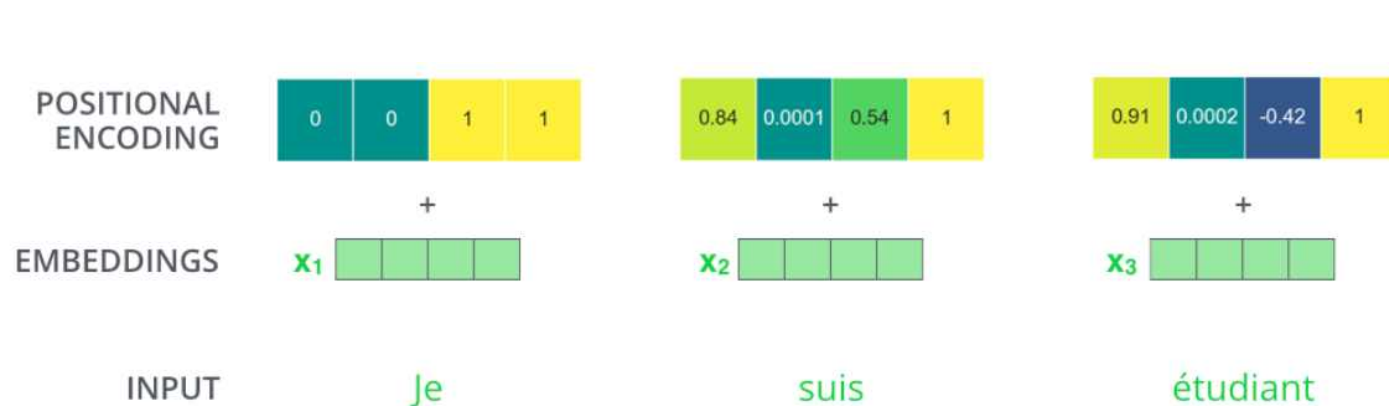
i – embedding vector 내의 차원의 index

- 같은 단어라도 문장 내의 위치에 따라서 Transformer의 입력으로 들어가는 Embedding vector 값이 달라진다 → 순서의 정보가 포함된 Embedding Vector
- 정현파는 일정한 cycle 로 값이 규칙적으로 반복되므로, 신경망이 상대적 위치 값을 쉽게 학습

[유튜브 설명 동영상](#)

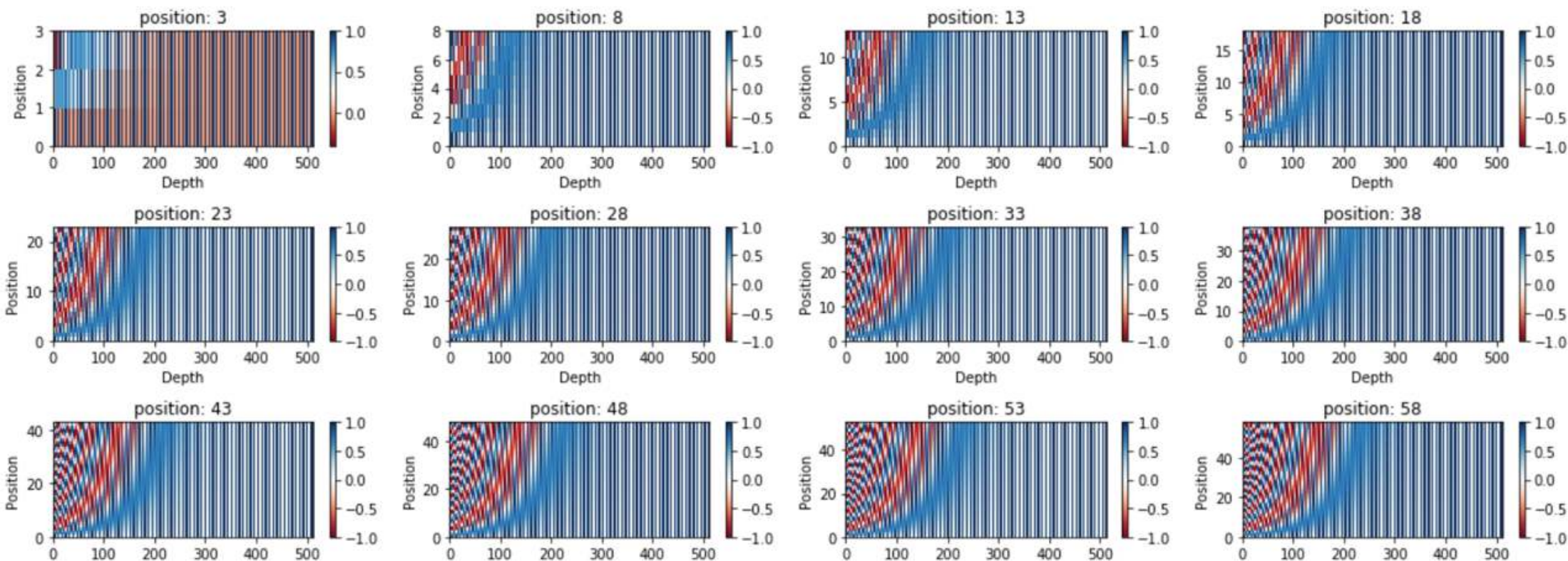


Positional Encoding example



참고) BERT
Learnable Positional
Embedding 사용

Positional Encoding Value 시각화

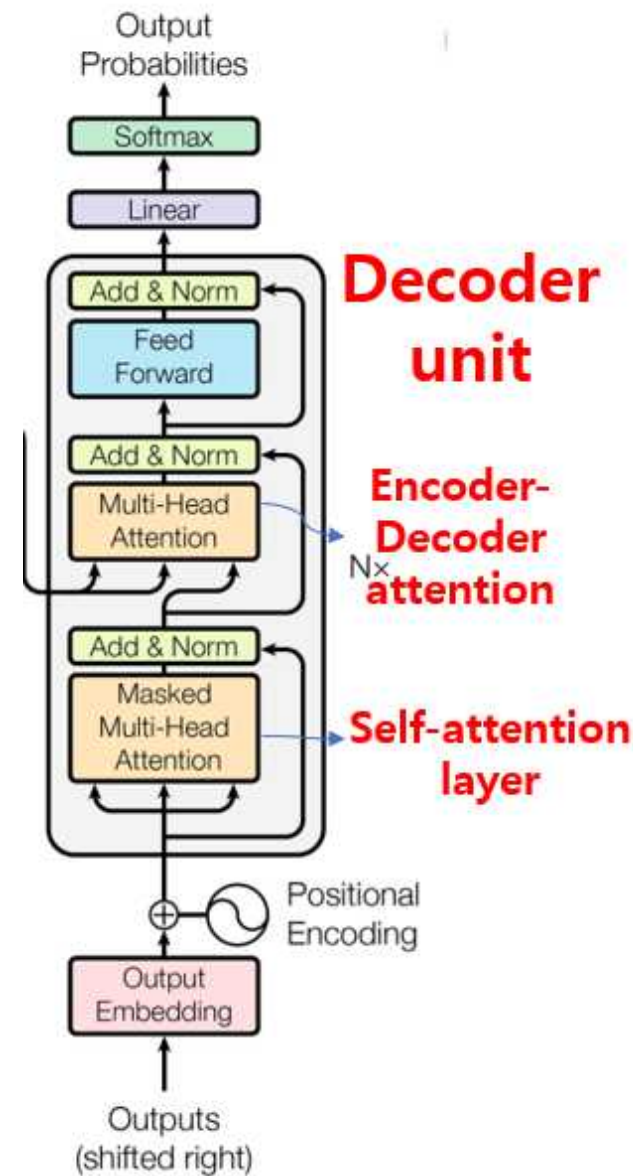


Transformer 의 주요 Hyper-Parameter

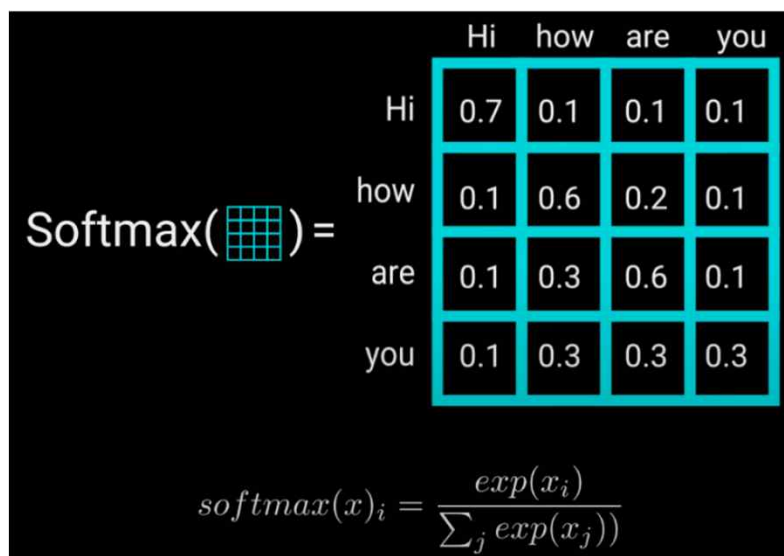
- $d_{model} = 512$
 - encode 와 decoder 의 정해진 입출력 크기
 - Embedding vector 의 크기
 - 모든 encoder, decoder 층에 공통
- num_layers = 6
 - Encoder 와 decoder 의 전체 층수
- num_heads = 8
 - 병렬처리 multi-head 갯수
- $d_{ff} = 2048$
 - 트랜스포머 내부 feed forward 신경망의 은닉층 크기

Decoder

- **Top Encoder** 의 **output** 이 **Decoder** 의 attention vector **Key, Value** 로 **transform** 되어 각 decoder 의 “encoder-decoder attention” layer 에서 사용
→ Input sequence 의 적절한 위치를 focus 하도록 함
- **Query** : training - ground truth label 사용
inference – previous predictions 사용
- Decoding result 는 encoder 경우처럼 위로 전달
→ 각 step 의 output 은 next time step 에서 bottom decoder 의 input 으로 사용
- Decoder input 에 positional encoding 을 추가해 줌

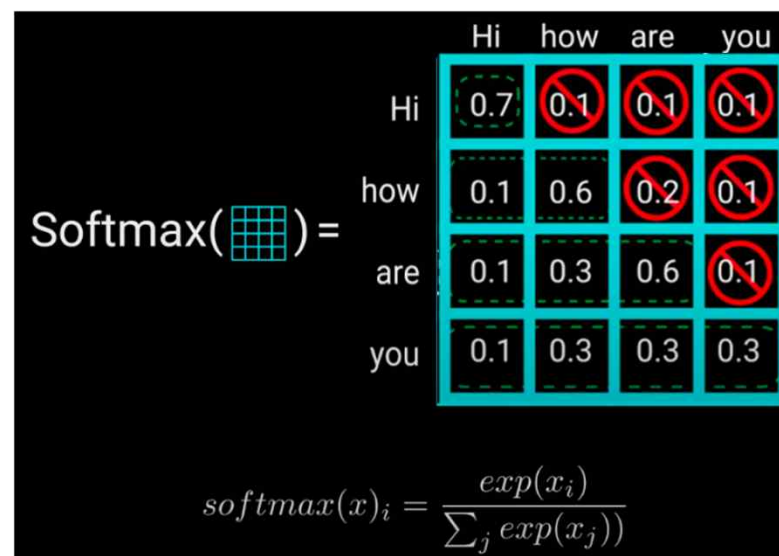


- Decoder 의 self-attention layer 는 output sequence 의 self-attention 계산을 위한 softmax step 전에는 **future position 을 masking** (-inf 로 setting) 하여 이전 위치만 attend 할 수 있도록 한다.



Taking the softmax of the scaled scores to get probability values

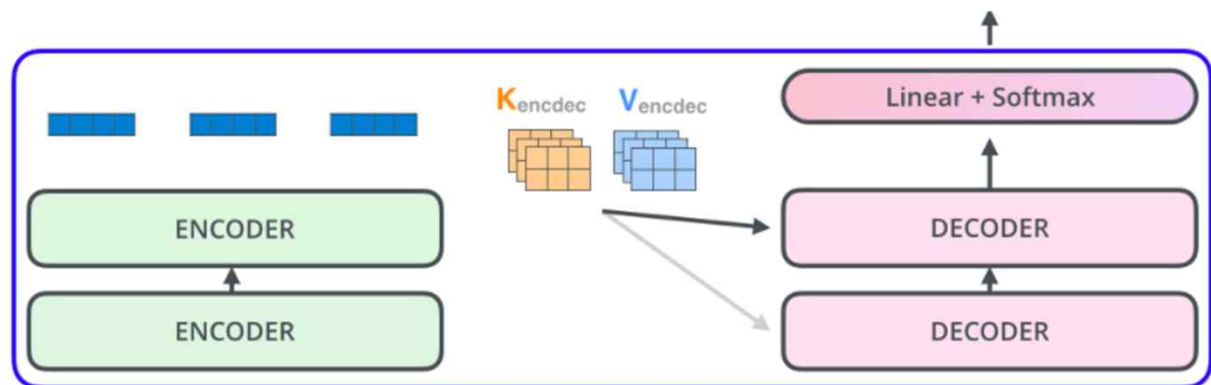
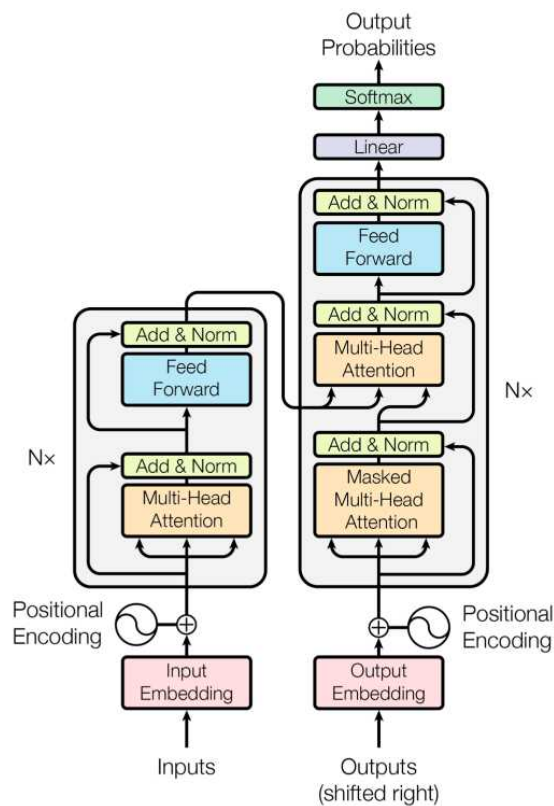
Encoder 의 attention score 계산



Taking the softmax of the scaled scores to get probability values

Decoder 의 attention score 계산

- Encoder-Decoder attention 은 Query 가 Decoder 의 아래쪽 layer 에서 생성되는 것 외에는 Encoder 의 multi-headed attention 과 동일하게 동작



Transformer 의 3 가지 Attention 정리

- key, value from encoder



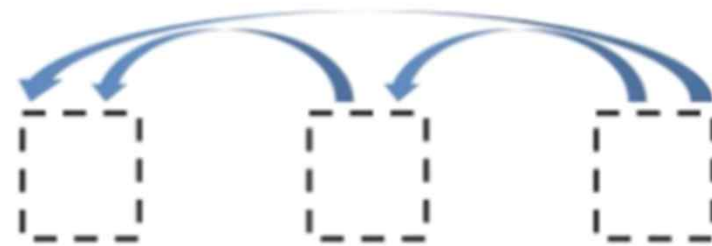
Encoder-Decoder Attention

- query, key, value
- Multi-head attention



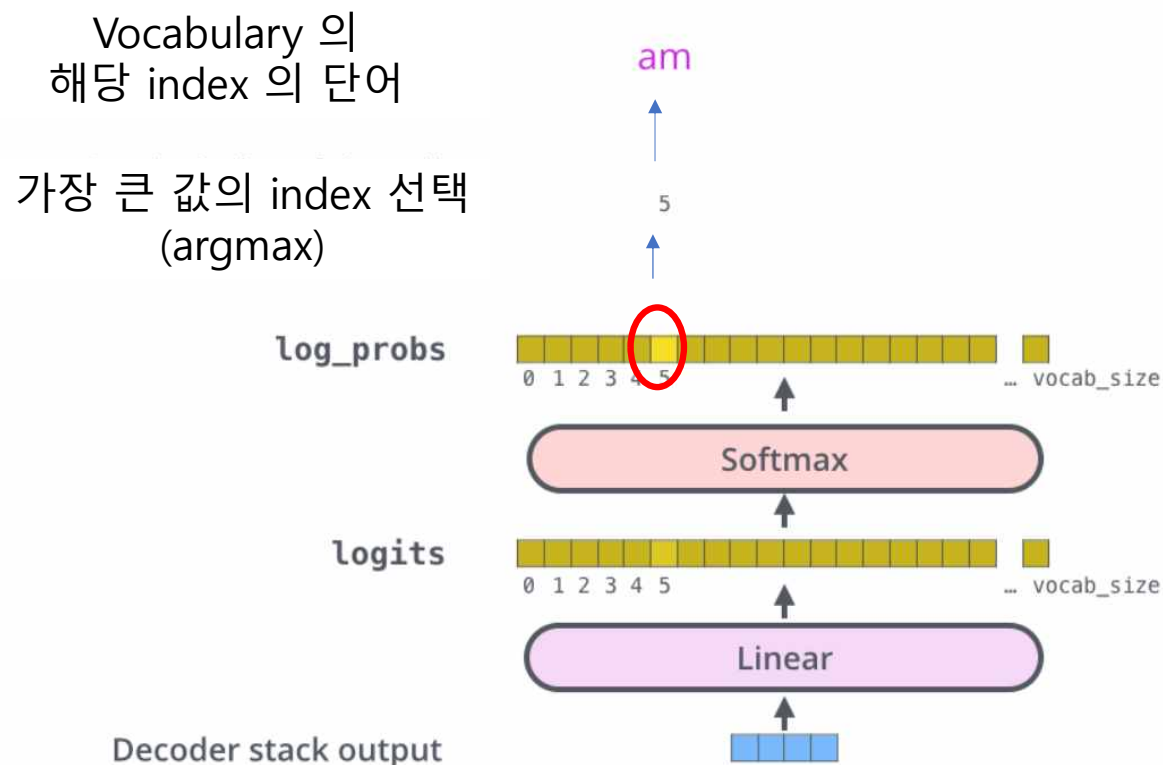
Encoder Self-Attention

- query, key, value
- Masked Multi-head attention



MaskedDecoder Self-Attention

Final Linear and Softmax Layer



Label Smoothing

– overfitting 방지를 위해 1 보다는 작고,
0 보다는 크도록 one-hot encoding 조정

Ex) Thank you
– 감사합니다. 고마워. 모두 정답

Training Dataset

- <https://huggingface.co/datasets/google/wmt24pp>

원 논문 "Attention Is All You Need" 학습 데이터

EN-DE (영어-독일어)

WMT 2014 | 약 450만 문장 쌍
BPE 37,000개 토큰 공유 vocabulary

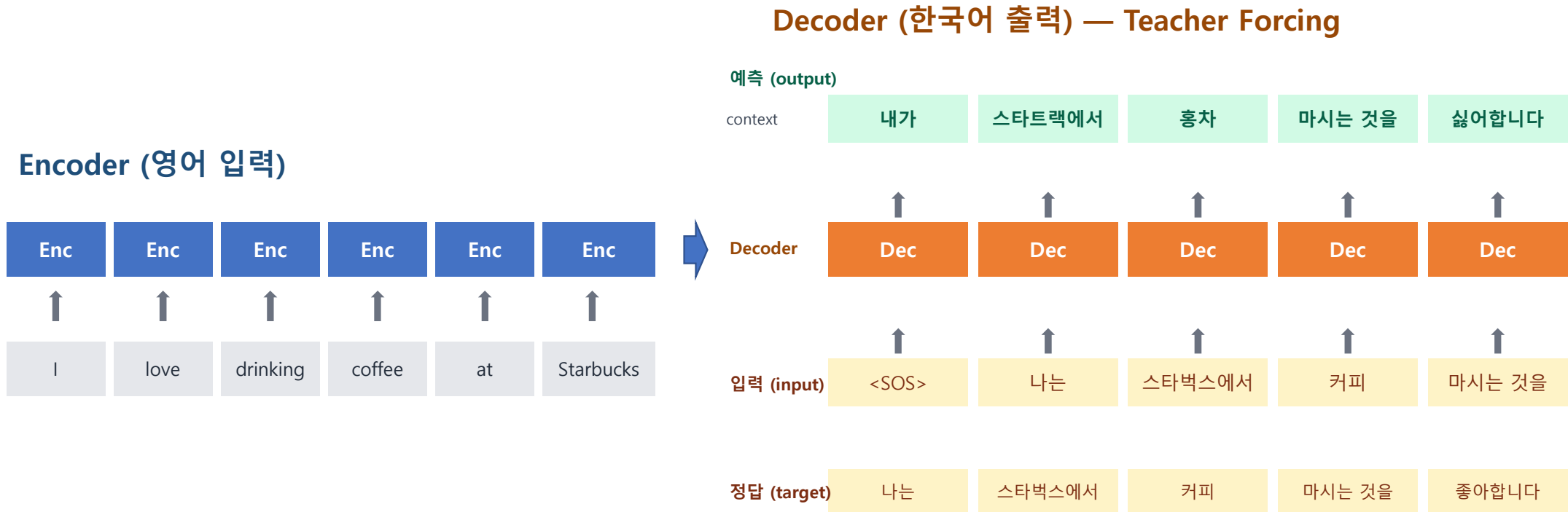
EN-FR (영어-프랑스어)

WMT 2014 | 약 3,600만 문장 쌍
Word-piece 32,000개 vocabulary

WMT (Workshop on Machine Translation)

매년 기계번역 연구용으로 공개하는 병렬 코퍼스. Europarl(유럽의회 회의록), Common Crawl, News Commentary 등 여러 소스를 합친 데이터셋.

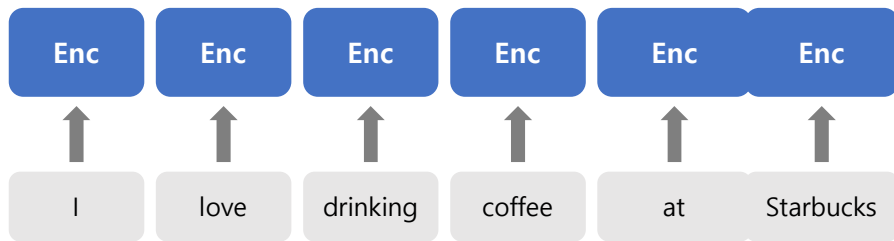
Teacher Forcing - 학습 (Training)



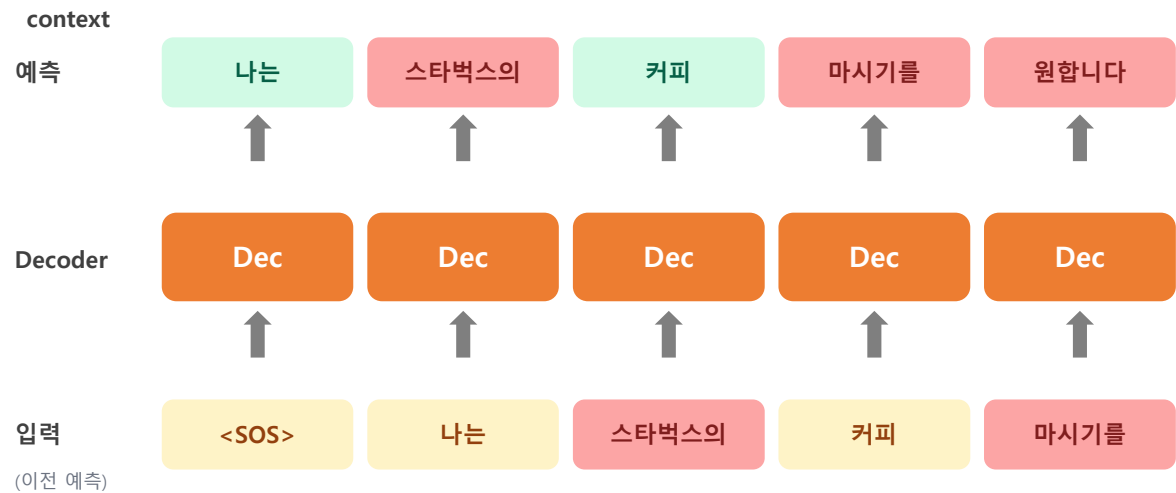
- 학습 시에는 정답 시퀀스를 미리 알고 있으므로, 매 step의 decoder 입력으로 이전 step의 예측이 아닌 **정답 토큰**을 넣어준다 (Teacher Forcing).
- 모든 step을 병렬로 학습할 수 있어 학습 속도가 빠르고, 초기 학습이 안정적이다.

Teacher Forcing - 추론 (Inference)

Encoder (영어 입력)



Decoder (한국어 출력) — Inference (예측 → 다음 입력)



⚠ 한 번 잘못된 예측이 다음 입력이 되어 오류가 누적됨 (Exposure Bias)

- 추론 시에는 정답이 없으므로, 매 step의 decoder 입력으로 **이전 step의 예측 토큰**을 사용한다.
- 학습-추론 불일치로 인해 한 번 어긋난 예측이 다음 입력으로 들어가 **오류가 누적**되는 문제가 발생한다.

실습: 130-Transformer model 이해

- From Google Tutorial
(<https://www.tensorflow.org/tutorials/text/transformer>)
- Portugal – English translation
- Text generation 및 Attention 이해 필수
- Encoder-Decoder model
- Transformer model 의 핵심은 Self-Attention

Transfer Learning of NLP

Transfer Learning

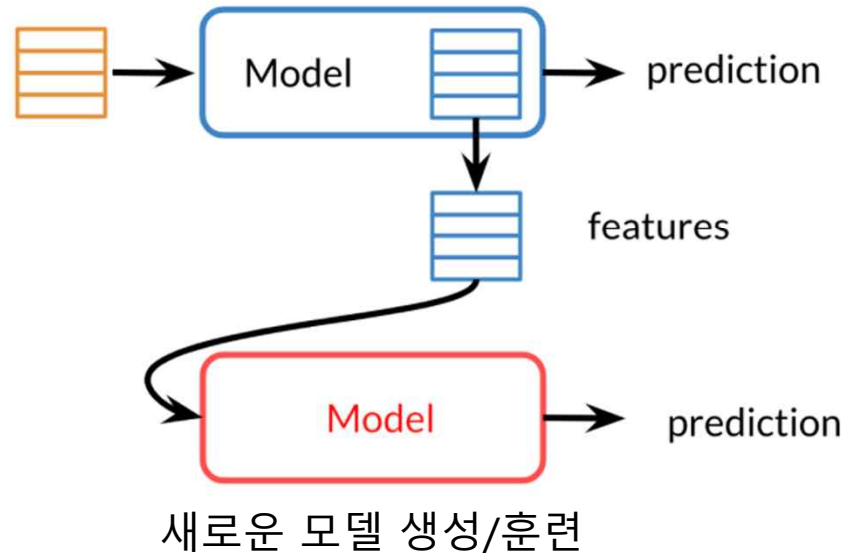
- Training Time 절감
 - GPT-3 와 같은 최첨단 model 은 train 에 약 2 개월 소요
- 예측 성능 향상
- Small dataset 으로 첨단 performance 구현

전이 학습 (Transfer Learning) 방법

Feature-based Transfer Learning

pre-trained model의 출력 feature를
new model의 입력으로 사용

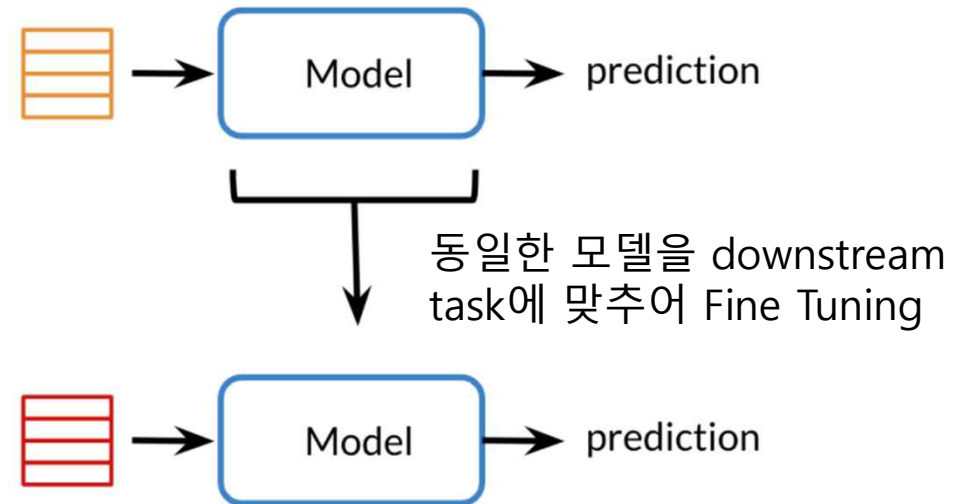
Pre-Train



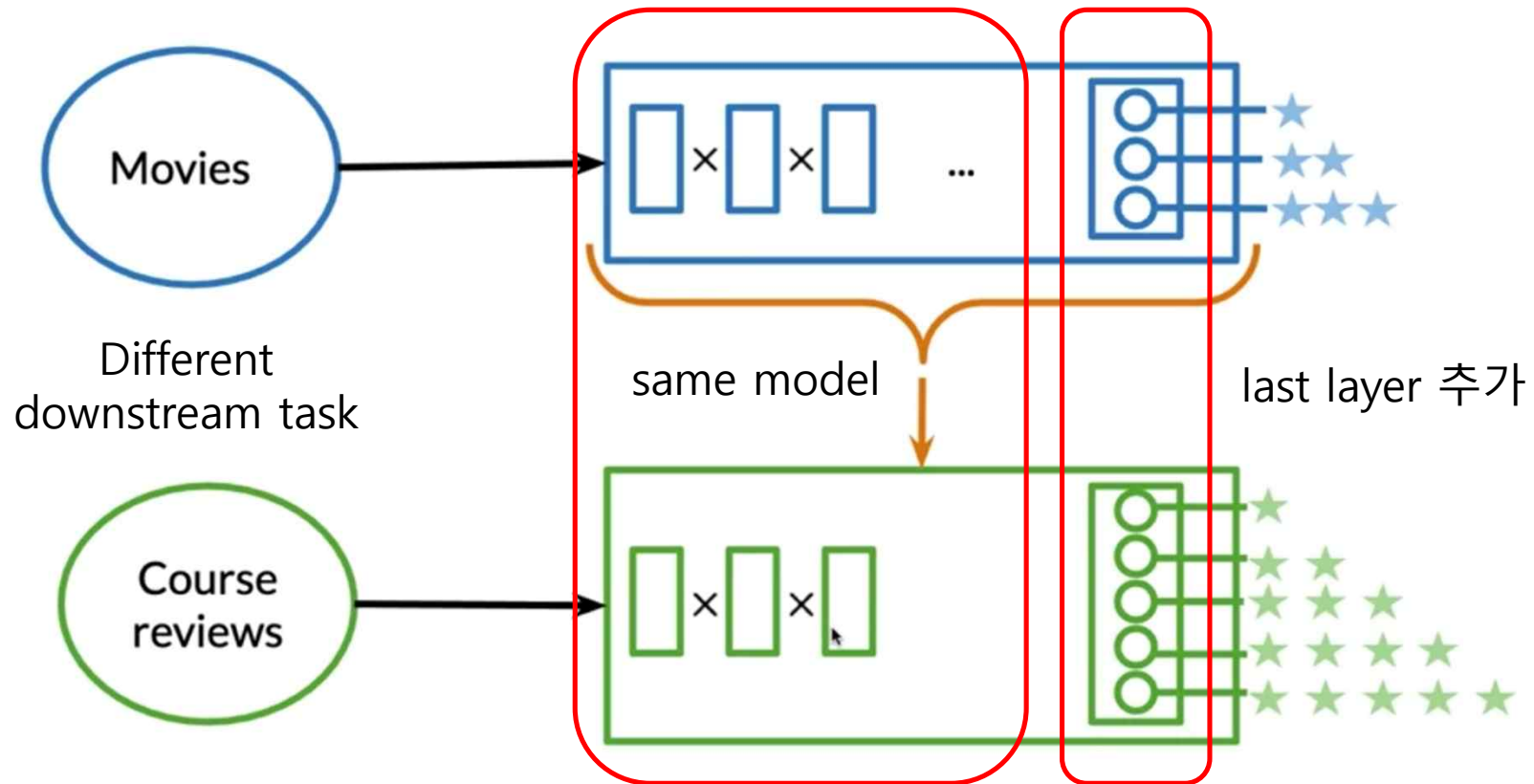
Fine-Tuning

pre-trained weight를
초기값으로 사용하여 동일
model fine-tuning

Pre-Train



Fine-tuning – last layer 추가



BERT

(Bidirectional Encoder Representations
from Transformers)

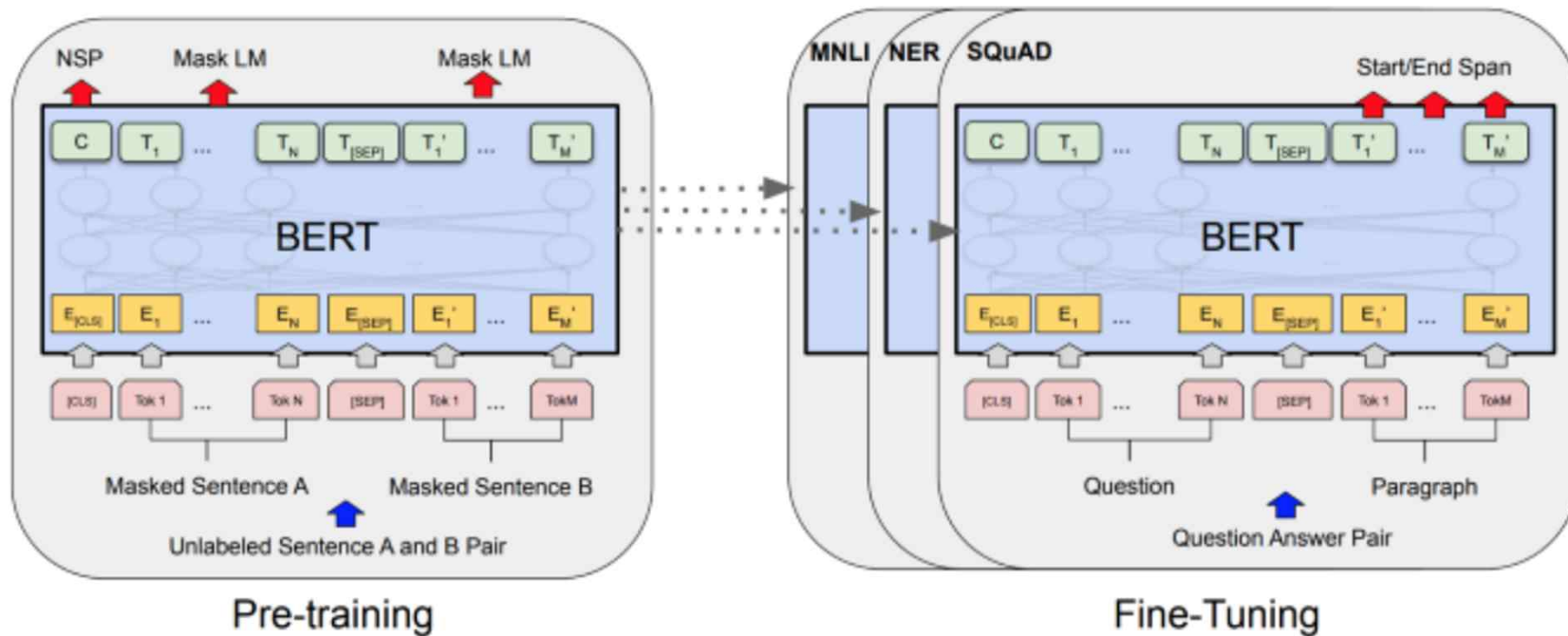
BERT

- 2018. Nov. 발표 (<https://github.com/google-research/bert>)
 - 104 개 multi-language model 동시 발표
 - ALBERT, RoBERTA, TinyBERT, DistilBERT, and SpanBERT 등 다양한 변종
- 질의 응답 시스템, 감성분석등의 다양한 GLUE task 에서 SOTA 달성
- Pre-trained token embedding (WordPiece) 사용
 - 30,000(영문 version) / 110,000(104개국어 version) subwords
- BERT-Base model : Transformer layer 12 개, Total parameters 110M
- BERT-Large model : Transformer layer 24 개, Total parameters 340M

BERT 의 접근 방식

- 1) 범용 Solution 을 (Transformer 의 encoder 이용)
- 2) Scalable 한 형태로 구현하여
- 3) 많은 머신 리소스로 훈련하여 성능을 높인다.
 - Pre-training 에 사용한 data
 - BooksCorpus (800M 단어)
 - English Wikipedia (2500M 단어)
 - BERT-Base : 64 TPU 4 days 소요
- 11 개의 NLP 과제에 하나의 pre-trained model 을 공통적으로 사용. 단, 각각의 과제에 대해 약간의 fine-tuning 적용
 - 발표 당시 SQuAD(Stanford Question Answering Dataset) 에서 human level 능가
- NLP 를 위한 AI solution 개발은 coding 능력을 갖춘 모든 사람에게 개방

BERT 전략



- No human labeling (비지도 학습)
- 대규모 corpus 사용
- 대규모 resource 사용

- 다양한 downstream NLP task 에 적용
 - MNLI(Multi-Genre Natural Language Inference)
 - Named Entity Recognition
 - SQuAD

BERT 적용 범위

- 모든 NLP 문제에 적용되지는 않는다

CAN DO

- Classification
- NER (Named Entity Recognition)
- POS Tagging
- Question Answering

CANNOT DO

- Language Model (next word 예측)
- Text Generation
- Translation

GLUE Benchmark

- GLUE (General Language Understanding Evaluation) 는 한개의 자연어 처리 모델에 대해 여러 태스크들을 훈련시키고, 그 성능을 평가 및 비교 분석하기 위한 데이터셋들로 구성
- 모델들의 **자연어 이해 능력**을 평가하기 위해 다양하고 해결하기 어려운 9개의 Task Dataset 으로 구성
- BERT와 같은 전이학습 모델들을 평가하기 위한 필수적인 벤치마크

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
BERT _{BASE}	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	91.1	94.9	60.5	86.5	89.3	70.1	81.9

GLUE Task 구성 – 9 개

- CoLA (The Corpus of Linguistic Acceptability)
 - 문장이 문법적으로 맞는지 분류 (이진분류)
- SST-2 (The Stanford Sentiment Treebank)
 - 영화평 감성 분류 (positive, negative, neutral)
- MRPC (Microsoft Research Paraphrase Corpus)
 - 문장B 가 문장A 의 다른 표현 (paraphrase) 인지 여부 (이진분류)
- STS-B (The Semantic Textual Similarity Benchmark)
 - 문장 A 와 B 는 얼마나 유사한가 ?
- QQP (Quora Question Pairs)
 - 두개의 질문이 서로 유사한가 ? (이진분류)

- MNLI (Multi-Genre Natural Language Inference)
 - 문장 B 가 문장 A 에 이어지는 문장인지, 반대되는 문장인지, 무관한 문장인지 여부
 - 문장 A (Hypothesis) [sep] 문장 B (Premise)
- QNLI (Question Natural Language Inference)
 - 문장 B가 문장 A의 질문에 대한 답을 포함하는지 여부 (이진분류)
- RTE (Recognizing Textual Entailment)
 - MNLI 와 유사하나, 상대적으로 훨씬 적은 데이터셋 (이진분류)
- WNLI (Winograd NLI)
 - 문장 B가 문장 A의 애매한 대명사를 정확한 명사로 대체하였는가 ? (이진분류)
- 기타 non-GLUE : SQuAD (Stanford NLP Group, Wikipedia 질의 응답)

Data Example of Question Answering

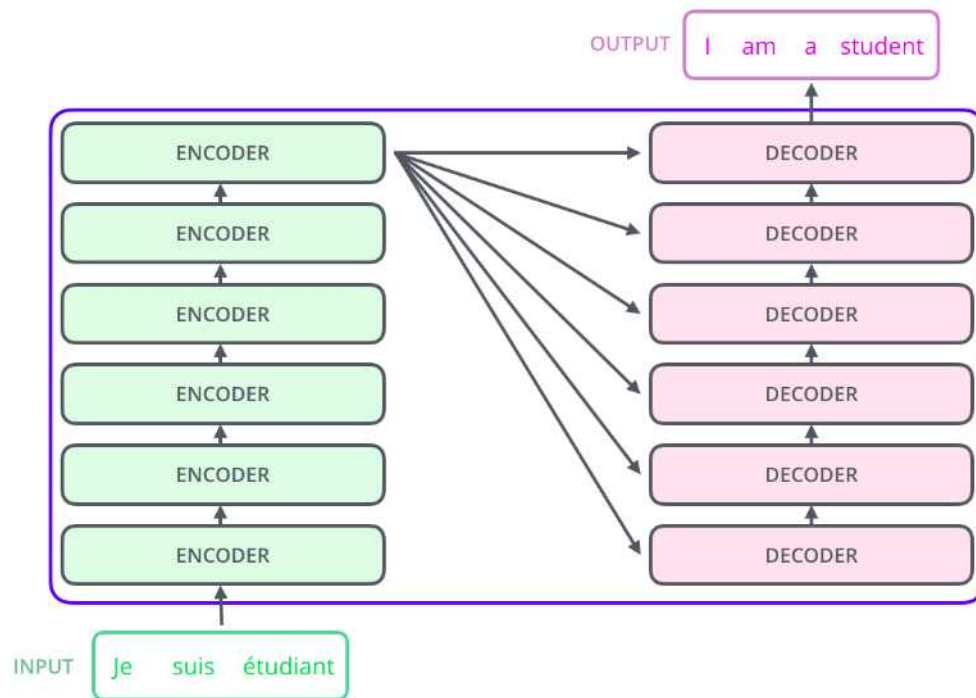
Question: What percentage of the French population today is non - European ?

Context: Since the end of the Second World War , France has become an ethnically diverse country . Today , **approximately five percent** of the French population is non - European and non - white . This does not approach the number of non - white citizens in the United States (roughly 28 – 37 % , depending on how Latinos are classified ; see Demographics of the United States) . Nevertheless , it amounts to at least three million people , and has forced the issues of ethnic diversity onto the French policy agenda . France has developed an approach to dealing with ethnic problems that stands in contrast to that of many advanced , industrialized countries . Unlike the United States , Britain , or even the Netherlands , France maintains a " color - blind " model of public policy . This means that it targets virtually no policies directly at racial or ethnic groups . Instead , it uses geographic or class criteria to address issues of social inequalities . It has , however , developed an extensive anti - racist policy repertoire since the early 1970s . Until recently , French policies focused primarily on issues of hate speech — going much further than their American counterparts — and relatively less on issues of discrimination in jobs , housing , and in provision of goods and services .

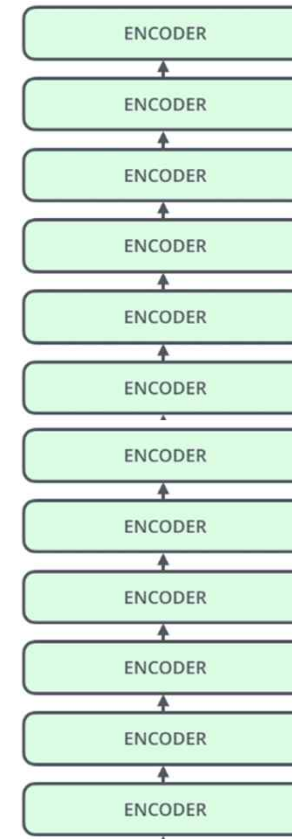
Target: **Approximately five percent**

BERT 구성

Transformer
6 encoders + 6 decoders

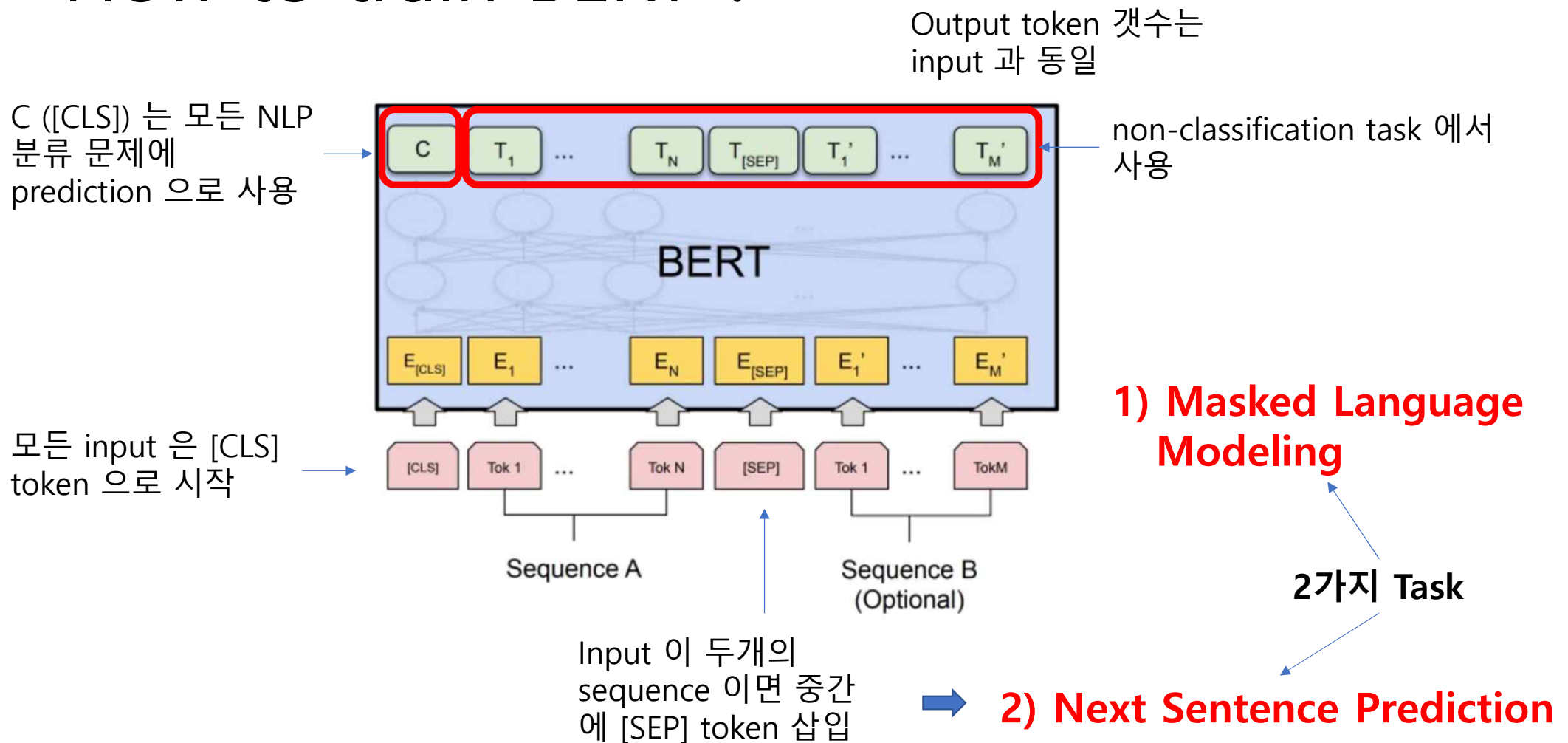


BERT
12/24 encoders



MLM, Next Sentence Prediction

How to train BERT ?

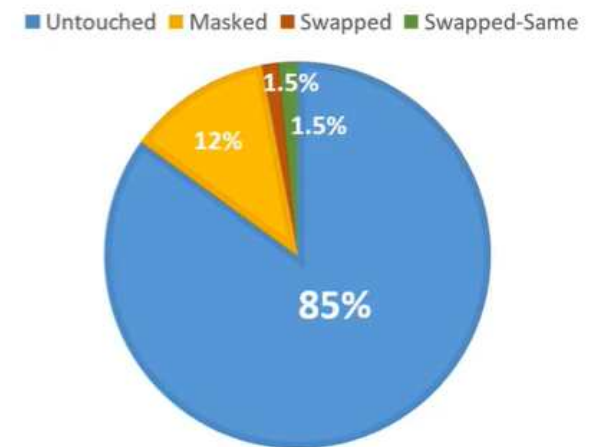


Train 1 : Masked Language Modeling (MLM)

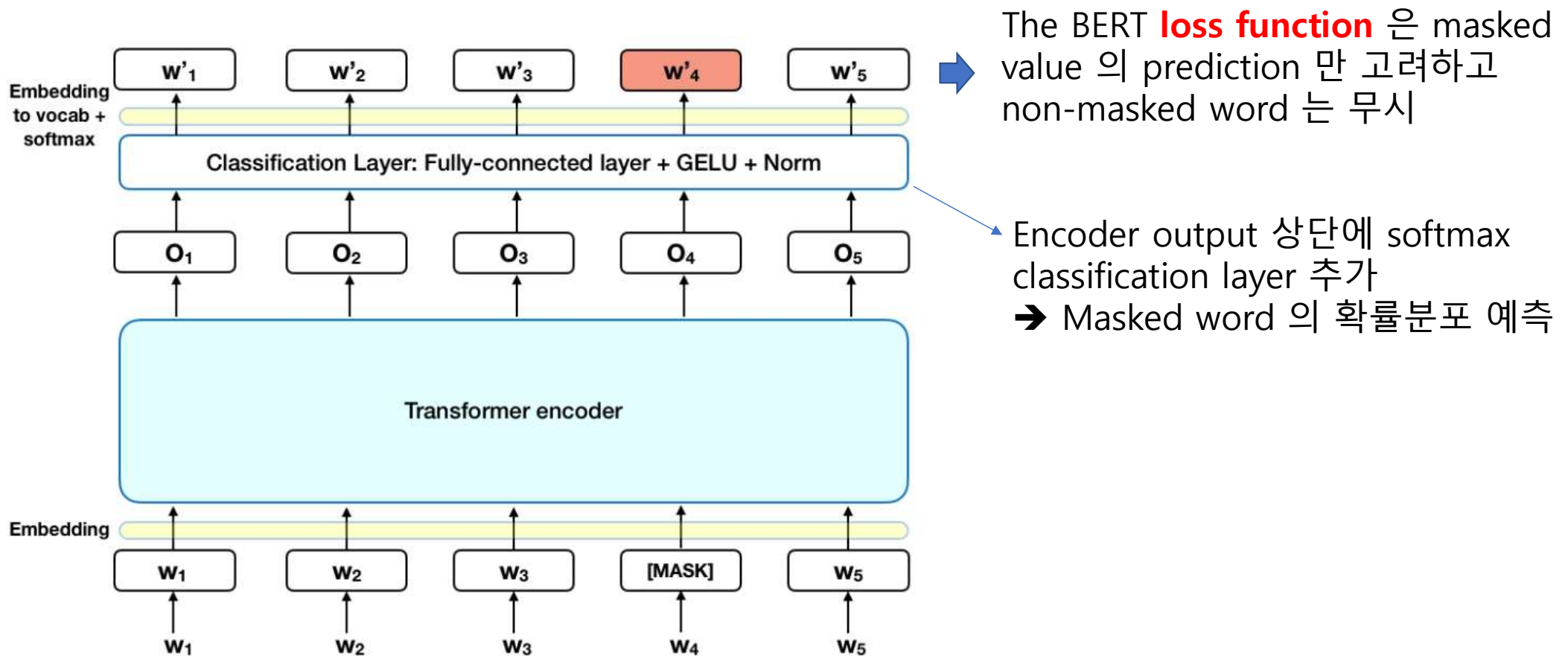
- Input 에서 random 하게 몇 개의 token 을 masking
 - **Input:** the man went to the [MASK] to buy a [MASK] of milk
 - Label:** store gallon

- Left, right context 만을 가지고 masked word 의 원래 단어를 예측

- 전체 학습 데이터 token 의 15% 를 masking.
- 이중 80% 는 [MASK] token 으로 replace
- 10% 는 random word 로 replace
- 10% 는 unchanged (tagging 만 바뀐 것으로 표시)



MLM training – 단어 예측



Train 2 : Next Sentence Prediction

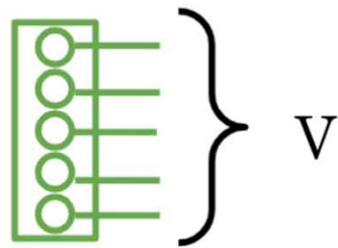
- Training data 로 sentence pair 사용
- 주요 목적은 output 의 C([CLS]) 를 train 시키는 것
- 예를 들어 100,000 개의 sentence 가 있고 BERT language model 을 pre-train 시키려면 50,000 개 pair 의 training data 존재
 - 50% 의 pair 는 second sentence 가 실제 다음 sentence (corpus 에서 연속된 문장)
 - 나머지 50% pair 는 second sentence 를 corpus 에서 random 선택 (사람의 개입 없음)
 - 첫째 경우의 label 은 'IsNext' (첫번 문장과 다음 문장이 관련 있음) 이고 둘째 경우는 'NotNext' (관련 없음)이다.

- 이어지는 문장이 앞선 문장과 연결되었는지 여부는 다음과 같이 train
 - 전체 input sequence 를 Transformer model 에 입력
 - [CLS] token 의 output 을 simple classification layer 를 이용하여 2×1 shaped vector 로 변환
 - Softmax 를 이용하여 IsNextSequence 예측
- BERT model train 시, **Masked LM 과 Next Sentence Prediction 은 동시에 train** 되고 goal 은 **combined loss function 을 minimize** 하는 것

BERT 의 목적 함수

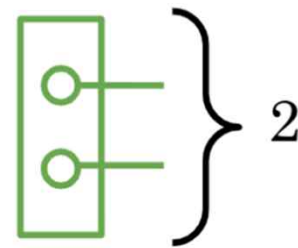
Objective 1:
Multi-Mask LM

Loss: Cross Entropy Loss



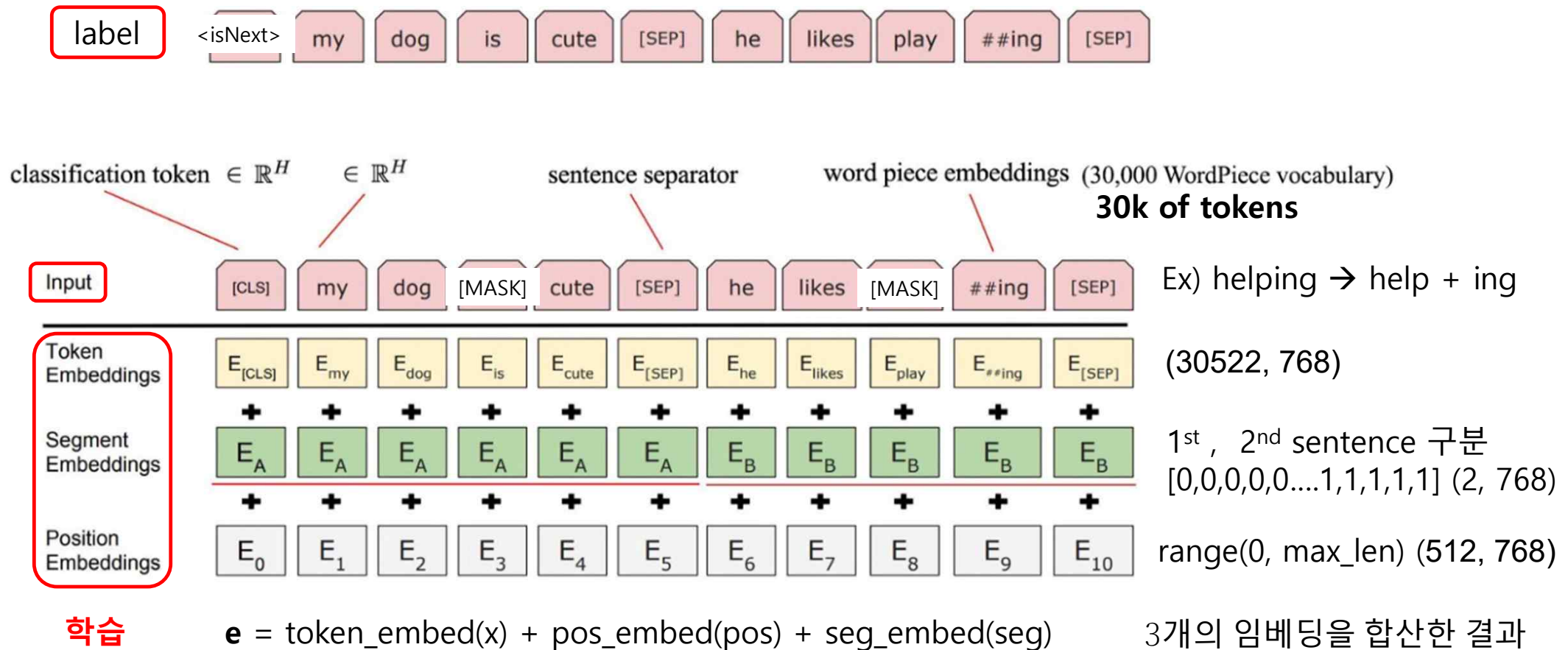
Objective 2:
Next Sentence Prediction

Loss: Binary Loss



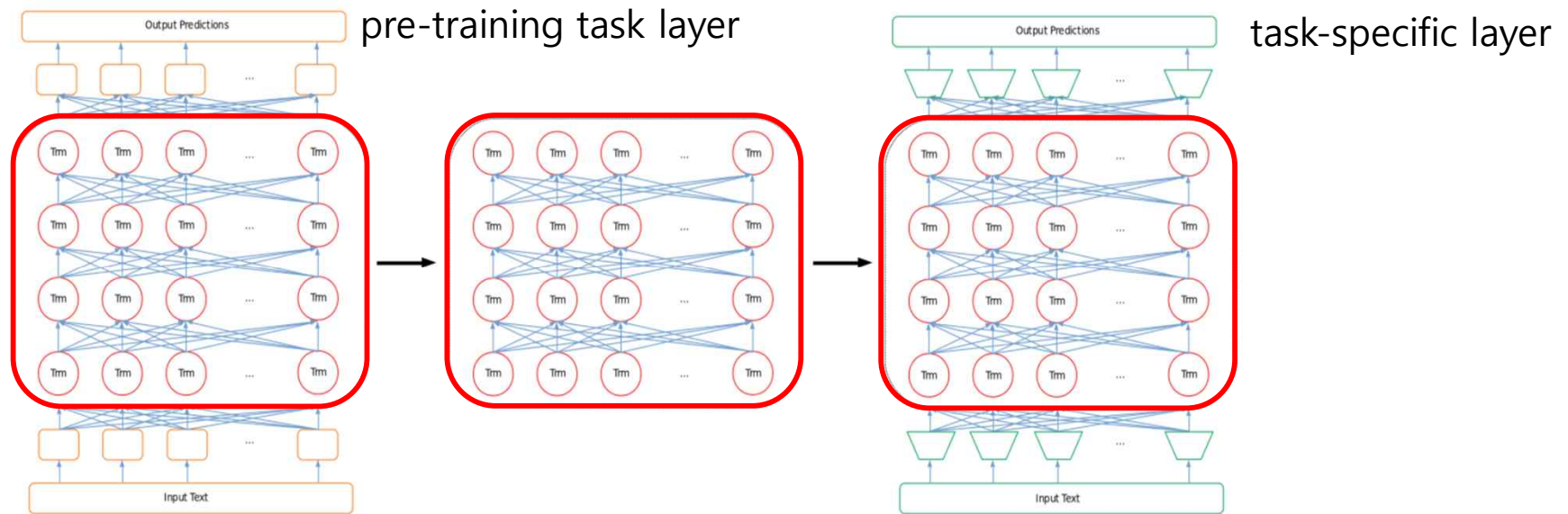
두 가지 목적함수를 합산

BERT 의 Input Embeddings – All Together

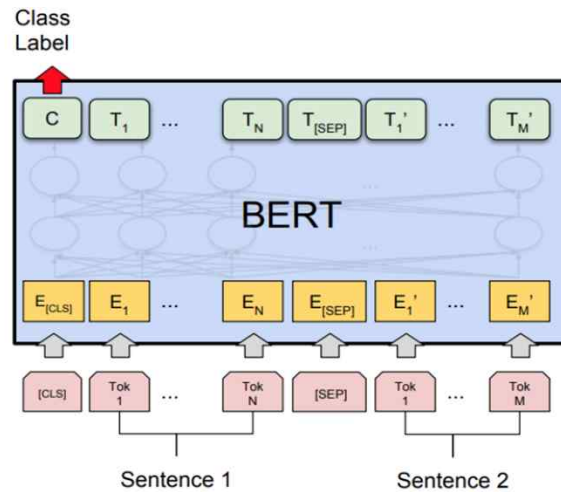


Fine-tuning process of BERT

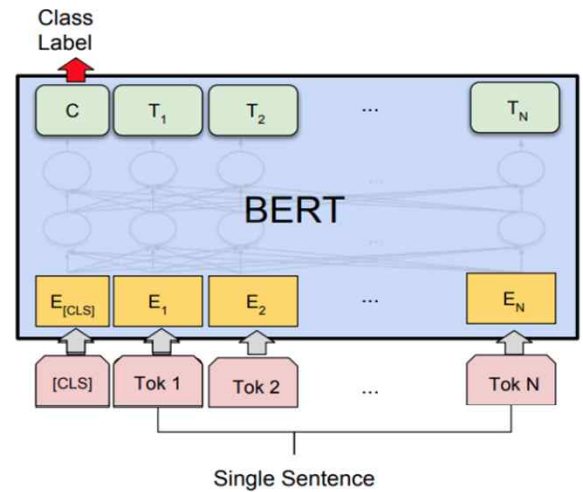
- 위 과정을 거쳐 변형된 모델을 n번의 epoch 동안 재학습(Fine-Tuning)
- 이때 사전학습 모델에서 차용한 모델 중간부는 자연어를 잘 '이해'하는 파라미터를 지니고 있기 때문에 우리가 풀고자 하는 문제 해결에 큰 도움이 됨



Classification Task



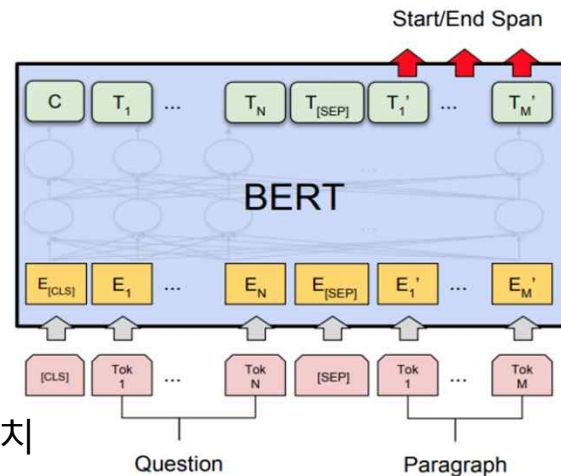
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



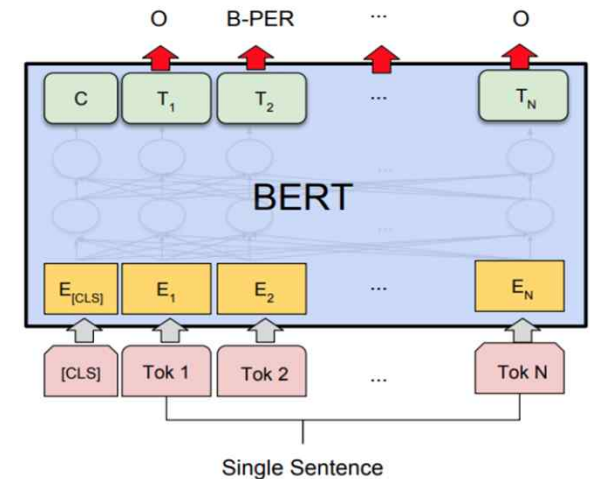
(b) Single Sentence Classification Tasks:
SST-2, CoLA

Non-Classification Task

Question – 질문
Paragraph – 본문
Start – 본문 중 정답 시작 위치
End – 본문 중 정답 끝 위치



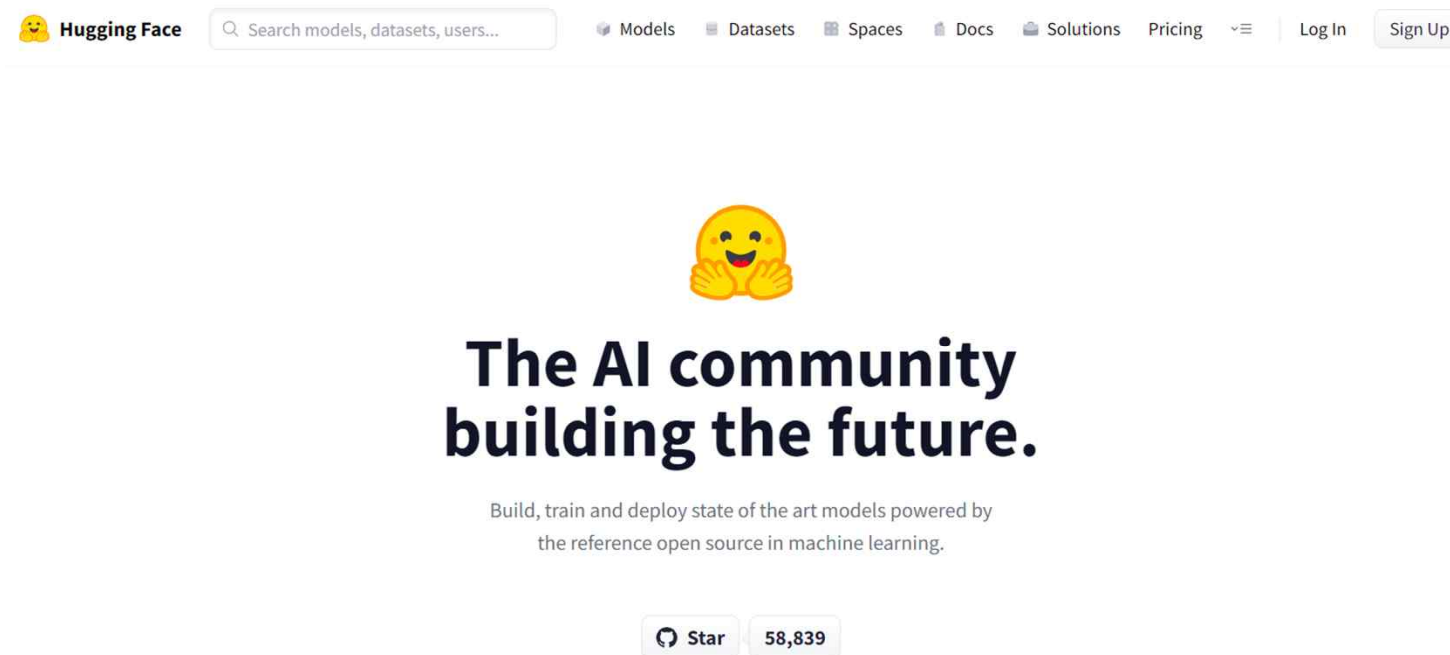
(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

HuggingFace transformers library (<https://huggingface.co/>)

- NLP 기술을 많은 이들이 편히 이용할 수 있도록 기술민주화를 하는 그룹



Hugging Face (<https://github.com/huggingface>)

- NLP 와 관련된 다양한 패키지를 제공

Search or jump to... Pull requests Issues Marketplace Explore

 **Hugging Face**
The AI community building the future.
NYC + Paris <https://huggingface.co/> Verified

Overview Repositories 52 Packages People 26 Projects Sponsoring 4

Pinned

 **transformers**
🤖 Transformers: State-of-the-art Natural Language Processing for Pytorch, TensorFlow, and JAX.
Python ☆ 50.4k 🍴 12k

 **datasets**
🤖 The largest hub of ready-to-use datasets for ML models with fast, easy-to-use and efficient data manipulation tools
Python ☆ 8.8k 🍴 1.1k

 **tokenizers**
🚀 Fast State-of-the-Art Tokenizers optimized for Research and Production
Rust ☆ 4.8k 🍴 375

 **knoockknoock**
🔔 Knock Knock: Get notified when your training ends with only two additional lines of code
Python ☆ 2.2k 🍴 190

 **accelerate**
🚀 A simple way to train and use PyTorch models with multi-GPU, TPU, mixed-precision
Python ☆ 1.8k 🍴 89

 **huggingface_hub**
all the open source things related to the Hugging Face Hub.
Python ☆ 196 🍴 34

Hugging Face

- Transformers
 - Transformer 기반 (masked) language model 알고리즘 제공
 - 기 학습된 (pretrained) 모델 배포
- Tokenizers
 - transformers package 에서 사용할 수 있는 subword 토크나이저 학습
 - transformers 와 분리되어 다른 목적에도 이용할 수 있다.
- dataset → tokenizer → models 의 언어 모델 학습에 필요한 전 과정을 지원

Hugging Face Trainer

- Transformers 라이브러리에서 제공하는 고수준 API로
- 사전 학습된 모델의 미세 조정(fine-tuning)과 평가를 간편하게 수행
- 복잡한 학습 루프를 직접 구현할 필요 없이, 모델 학습에 필요한 주요 구성 요소들을 설정하여 효율적으로 모델 훈련
- 주요 구성 요소
 1. 모델 (model): 사전 학습된 모델 객체 (예: AutoModelForSequenceClassification)
 2. 학습 인자 (TrainingArguments): 출력 디렉토리, 에폭 수, 배치 크기, 평가 전략 등 정의
 3. 데이터셋 (train_dataset, eval_dataset): 학습 및 평가에 사용할 데이터셋
 4. 토큰라이저 (tokenizer): 텍스트 데이터를 모델 입력 형식으로 변환하는 도구
 5. 데이터 콜레이터 (data_collator): 배치 단위로 데이터를 처리하는 함수.
 6. 평가지표 함수 (compute_metrics): 모델의 성능을 평가하기 위한 함수

실습 : 140-HuggingFace Sentiment Fine Tuning

사전학습 BERT 를 NAVER 영화 리뷰로 fine-tuning — 전이학습의 전 과정

1 NAVER 리뷰로 BERT Fine-Tuning

데이터 + 사전학습 모델

NSMC(네이버 영화 리뷰) 로
사전학습 BERT 를 우리 과제에 맞게 조정

2 다국어 Tokenizer 로 Encoding

bert-base-multilingual-cased

한국어 포함 100여 개 언어 지원
문장 → input_ids · attention_mask

3 Trainer 로 모델 학습

HuggingFace Trainer

TrainingArguments 설정
Trainer.train() 한 줄로 학습 수행

4 Fine-Tuning 모델로 예측

softmax → argmax

model.eval() · no_grad 추론
새 리뷰의 긍정 / 부정 판별

전이학습(transfer learning) : 이미 언어를 아는 모델을 재활용 → 적은 데이터·짧은 시간에 높은 성능

NER (Named Entity Recognition) Task

- **정의:** 문장에서 사람(PER), 장소(LOC), 조직(ORG), 날짜(DATE), 숫자 등 미리 정의된 개체명 (Named Entity)을 식별하고 분류하는 Sequence Labeling 과제
- **입력·출력:** 토큰 단위 입력 → 각 토큰에 entity tag 부여 (token classification)
- **태깅 방식:** BIO 스킴이 표준 — B-(개체 시작), I-(개체 내부), O(개체 아님)
- **예시:** "Steve Jobs는 1976년 Apple을 설립했다"
→ Steve/B-PER, Jobs/I-PER, 는/O, 1976/B-DATE, 년/O, Apple/B-ORG, 을/O, 설립했
다/O
- **활용 분야:** 정보 추출, 검색·질의응답, 챗봇, 의료·법률 문서 분석, 금융 뉴스 모니터링 등

실습 : 150-HuggingFace NER Fine Tuning

klue/bert-base 를 *KLUE-NER* 로 *fine-tuning* — 토큰마다 라벨을 매기는 *NER*

1 KLUE-NER 로 BERT Fine-Tuning

token classification

한국어 NER 벤치마크 데이터셋
토큰마다 BIO 태그 부여

2 Sub-word 단위 Label Alignment

klue/bert-base Tokenizer

글자 라벨 → sub-word 토큰 정렬
word_ids() · 이어지는 조각은 -100

3 Token Classification 학습

AutoModelForTokenClassification + Trainer

DataCollatorForTokenClassification
seqeval entity 단위 F1 평가

4 개체명 예측

pipeline('ner') 추론

PS · LC · OG · DT · TI · QT
aggregation_strategy='simple'

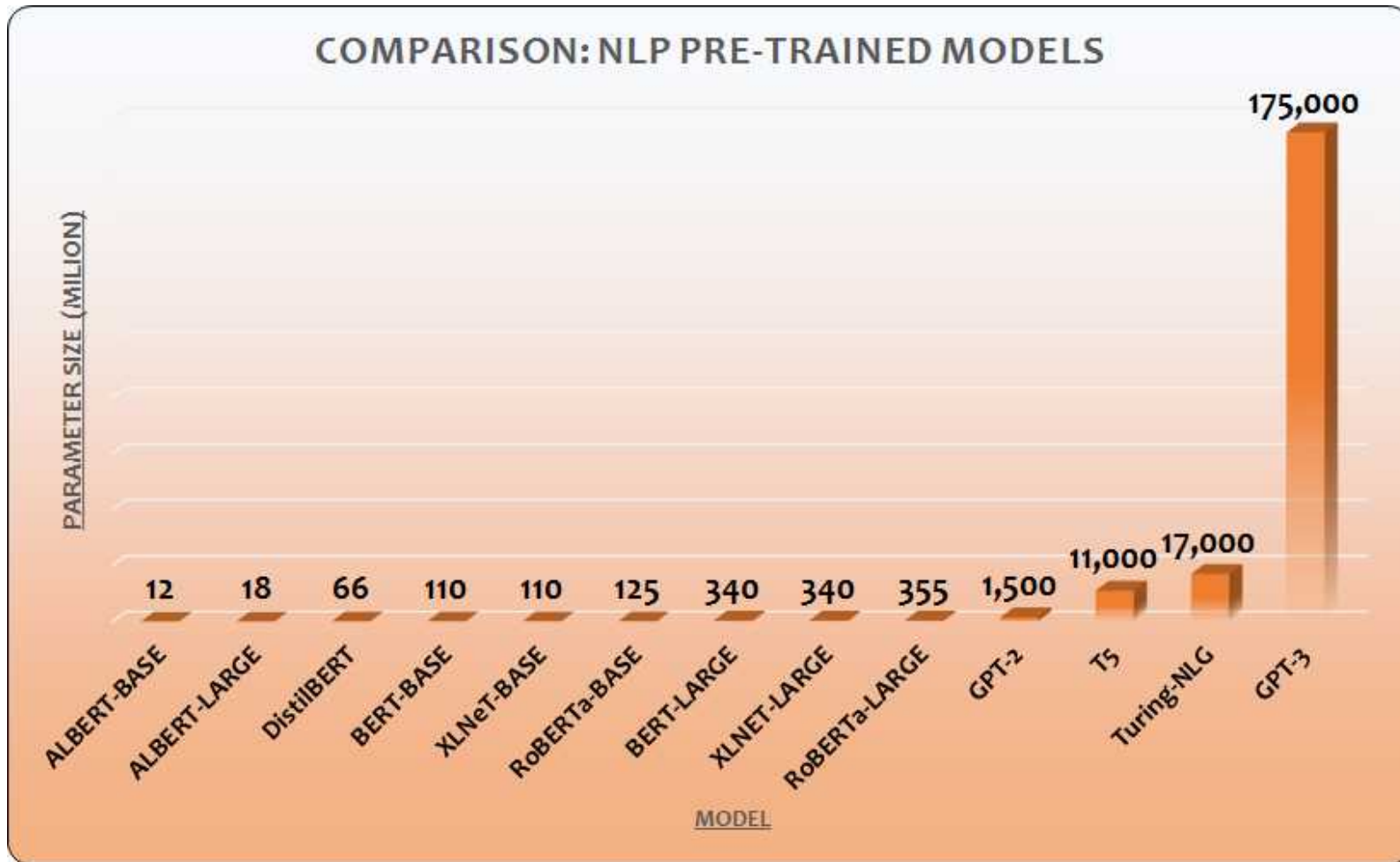
NER의 핵심 : 글자 단위 라벨을 sub-word 토큰에 맞추는 alignment 단계

GPT-3

What is GPT-3 ?

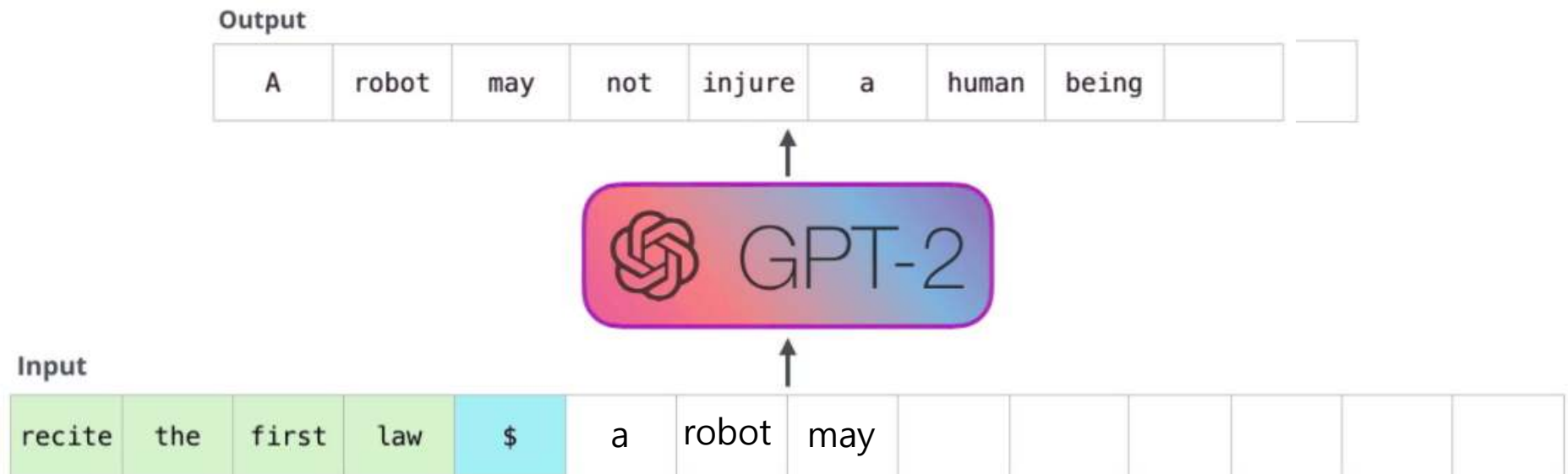
- Transformer의 decoder 부분을 가능한 높이 쌓은 구성(96 layers)
- 대용량 dataset(300 billion tokens of text)과 computing 자원을 이용하여 transformer 기반 language model 학습
 - language model – next word prediction model (ex) 스마트폰의 자판 입력기
 - GPT-3 training cost는 Tesla V100 cloud 사용 기준 1회 당 약 \$4.6M
- fine-tuning 없이 사용하는 model
 - Parameter 수를 1.5 billion 에서 175 billion 으로 확장
 - GPT-3 가 생성한 가짜 기사 구분 by human – 52% (인간은 구분 못함)

GPT-3 : parameter size – 175 Billions

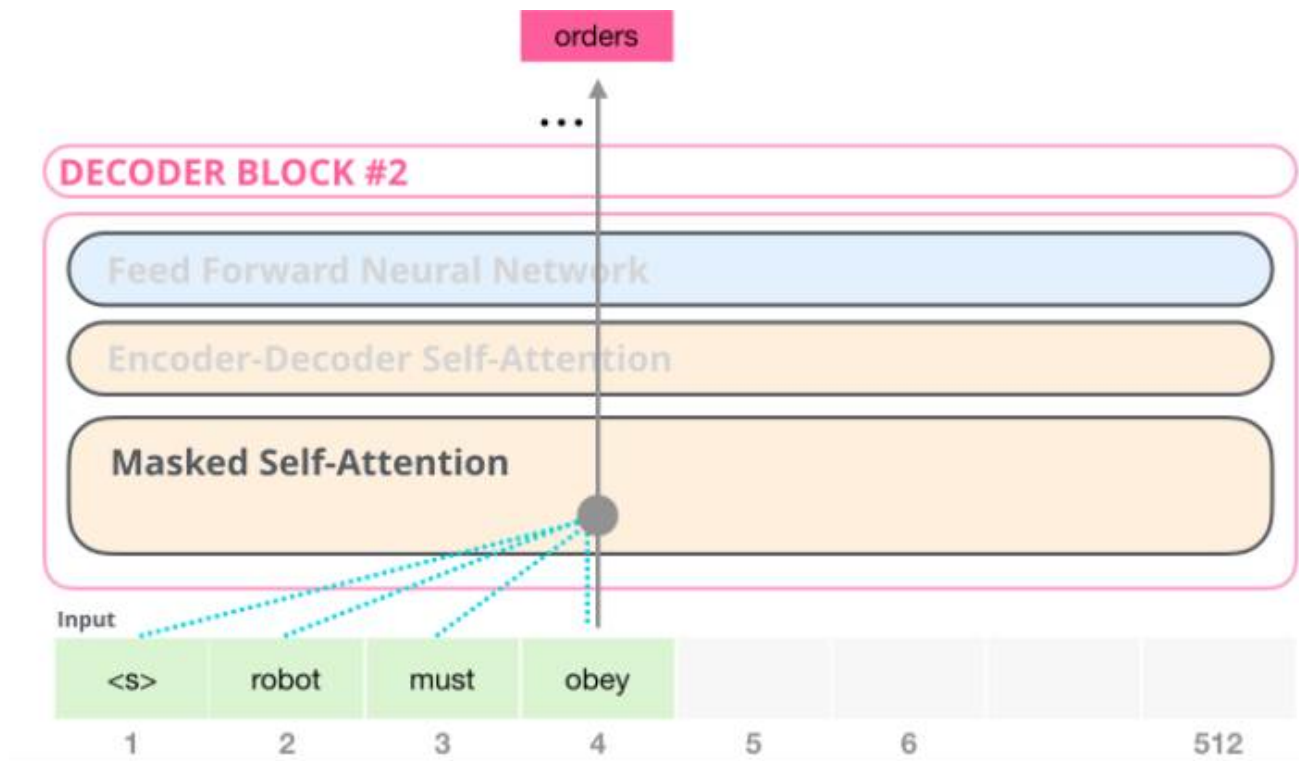


GPT 는 auto-regressive 한 입력 구조

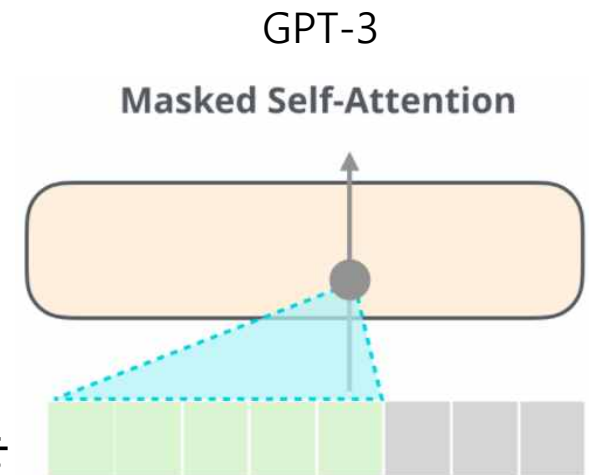
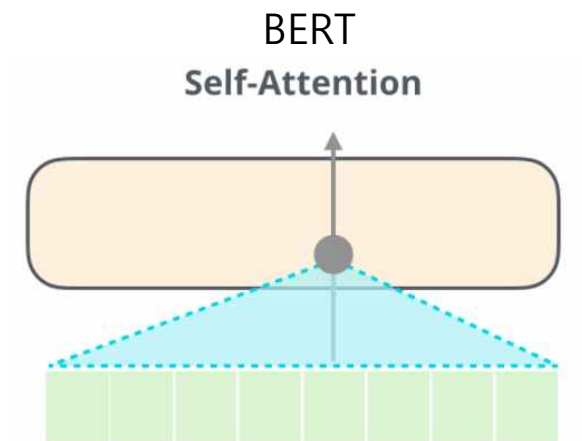
- 각 토큰이 생성 된 후 해당 토큰이 입력 시퀀스에 추가
- 그 새로운 시퀀스는 다음 단계에서 모델에 대한 입력이 된다



Masked Self-Attention

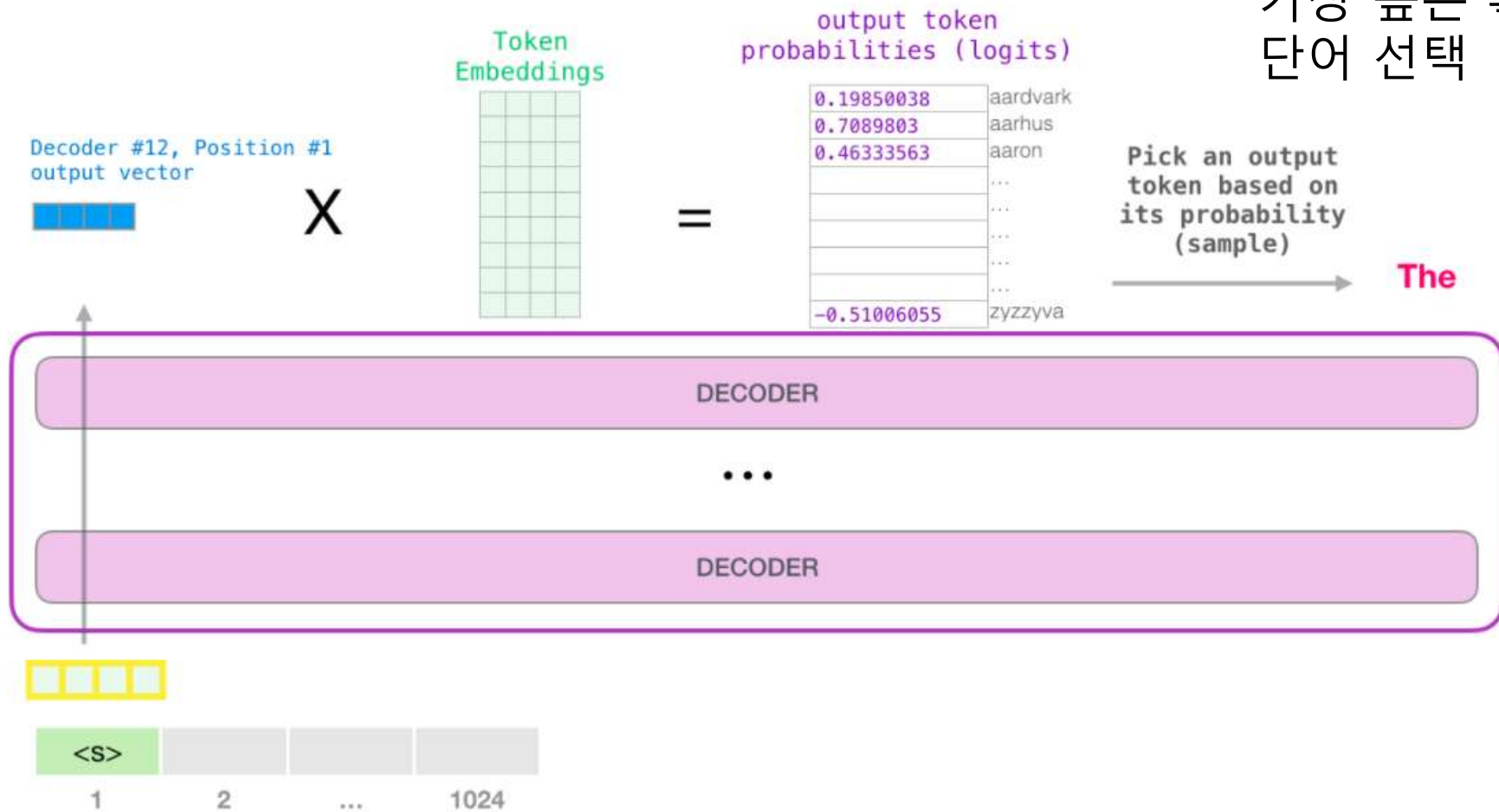


Next token 예측을 위해 현재 및 이전 토큰만 참조 가능



Output token probability

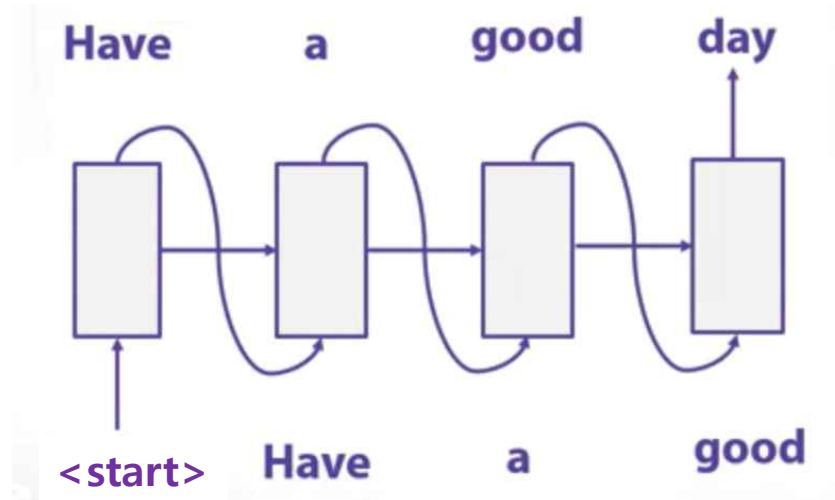
가장 높은 확률의
단어 선택



Language Model

●언어 생성 모델의 작동 원리

→이전 time step 의 output 을 next time step 의 input 으로 feed 하고, 각 step 에서 가장 높은 확률의 다음 단어를 선택 (greedy selection)하거나 혹은 확률 분포에 따라 sampling → Auto-regressive 방식



실습 160: Autoregressive 문장 생성 — HyperCLOVA X

핵심 개념 — Autoregressive: 이전 토큰을 기반으로 다음 토큰을 하나씩 예측 (GPT 계열 구조)
사용 모델: naver-hyperclova/HyperCLOVAX-SEED-Text-Instruct-0.5B (한국어 특화 대화형 LLM)

- 1 모델 로드 — `AutoModelForCausalLM + AutoTokenizer`
- 2 입력 구성 — chat 형식(role/content) → `apply_chat_template()` → 토큰화/디코딩 확인
- 3 자동 생성 — `model.generate()`
- 4 수동 생성 — Forward → logits → argmax → 이어 붙임 반복 (원리 직접 구현)
- 5 후처리 — 결과 디코딩 후 종료 토큰 제거

ChatGPT

OpenAI GPT

● ChatGPT vs GPT-3 비교

	ChatGPT	GPT-3
발표	2022년 11월	2020년 6월
매개변수 숫자	비공개 (GPT-3.5 기반)	1,750억개
Fine Tuning	'인간 feedback을 이용한 강화학습' 방법 이용	모든 OpenAI 언어 모델의 기본형
학습 Dataset	대화형 데이터	일반 문서, 서적, 기사 등

RLHF (Reinforcement Learning from Human Feedback)

OpenAI GPT

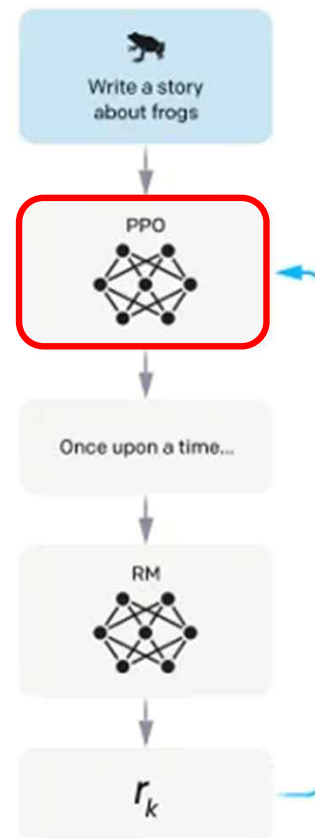
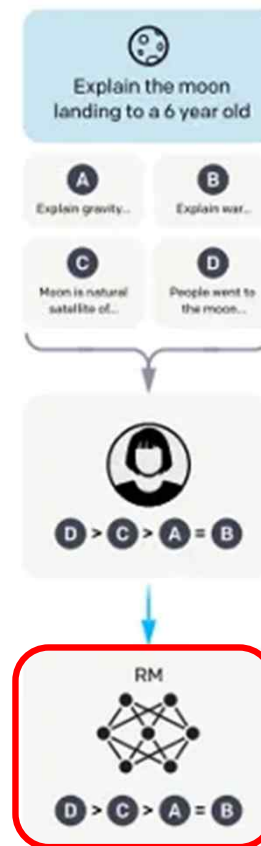
● ChatGPT Fine Tuning 과정

- ➔ 1단계 : Supervised Fine Tuning(SFT) model 작성
 - ▶ GPT-3를 초기 model로 사용
 - ▶ 40 명의 계약직 labeler가 생성된 output 에 점수 부여
 - ▶ 이렇게 얻어진 데이터를 이용하여 지도학습 방법으로 Fine Tuning
- ➔ 2단계 : Reward Model 작성
 - ▶ 1단계에서 Fine Tuning된 SFT 모델의 출력에 labeler가 ranking 부여
 - ▶ 이 데이터를 이용하여 reward model train
- ➔ 3단계 : 강화학습 모델 생성

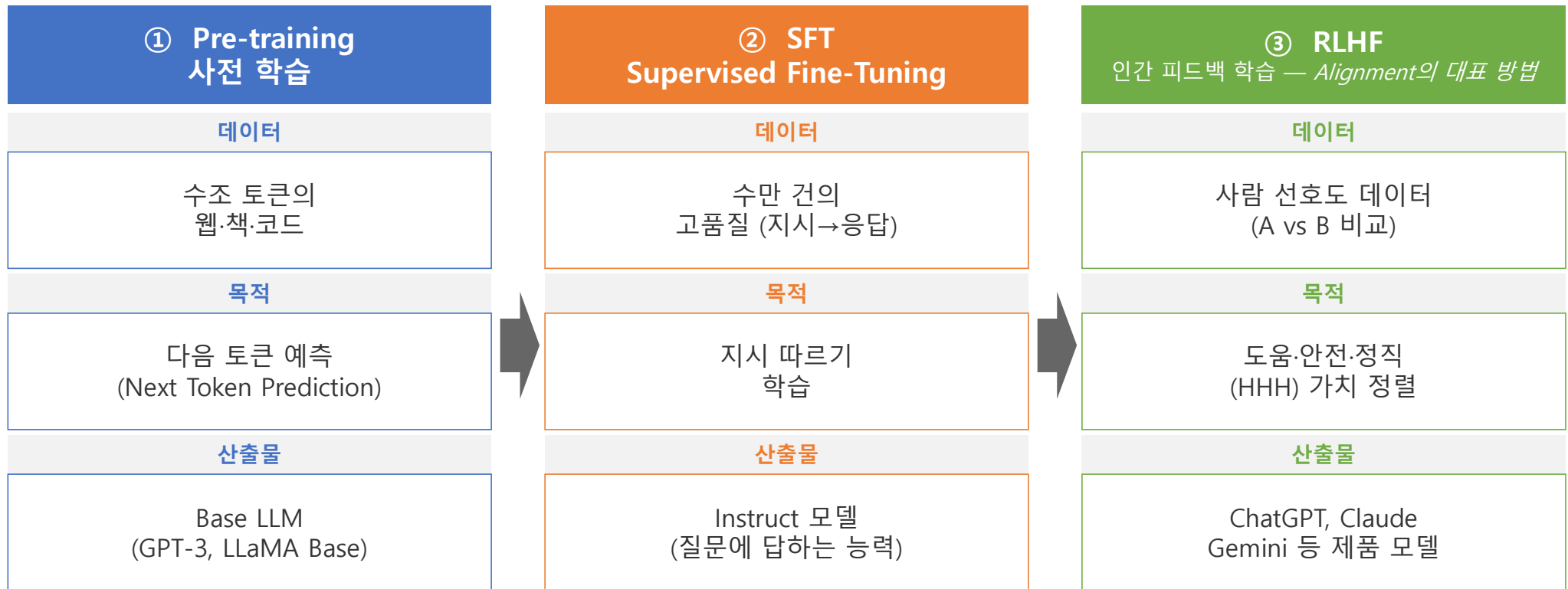
ChatGPT Fine Tuning 과정

RLHF(Reinforcement Learning from Human Feedback)

- 1단계 : **Supervised Fine Tuning(SFT) model** 작성
 - GPT-3를 초기 model로 사용
 - 인간이 의도하는 정책을 학습 시키기 위해 인간 labeler가 모델이 생성한 output 에 점수 부여
 - 이렇게 얻어진 데이터를 이용하여 지도학습 방법으로 Fine Tuning
- 2단계 : **Reward Model** 작성(인간 선호도 모방)
 - 1단계에서 Fine Tuning된 SFT 모델의 여러 답변 후보에 인간 labeler가 ranking을 매긴 데이터셋 수집
 - 이 데이터셋을 이용하여 새로운 reward model 학습
- 3단계 : **SFT 모델을 강화학습**



LLM 학습 3단계 파이프라인 — ChatGPT/Claude는 이렇게 만들어진다



핵심: Pre-training은 "아는 것" / SFT는 "답하는 법" / RLHF는 "사람이 좋아하는 답"

HHH 원칙 — Alignment가 추구하는 3가지 가치

Anthropic이 제시한 안전한 AI의 기준 — "좋은 AI는 어떻게 행동해야 하는가?"



Helpful (도움이 되는)

사용자의 요청을 정확히 이해하고
실질적으로 유용한 답을 제공

예) 명확한 질문 ↑
모호한 요청은 재확인

❌ "잘 모르겠어요"만 반복 (회피)



Harmless (해를 끼치지 않는)

위험·차별·불법·자해 콘텐츠를 거부
취약 사용자를 보호

예) 폭탄 제조법 거부
혐오 표현 차단

❌ "네 알겠습니다" 무조건 따름 (jailbreak)



Honest (정직한)

모르는 것은 모른다고 말하고
사실과 추측을 구분

예) 출처·근거 제시
불확실성 표현 ("~로 알려짐")

❌ 그럴듯한 거짓말 = 환각(hallucination)

⚖ 세 가치는 서로 충돌할 수 있다 (예: 정직 vs 무해) → Alignment 학습은 이 trade-off의 균형점을 찾는 과정

PEFT — 적은 자원으로 LLM 길들이기 (LoRA · QLoRA)

Parameter-Efficient Fine-Tuning — 전체 파라미터의 0.1~1%만 학습하고도 비슷한 성능

Full Fine-tuning vs PEFT

❌ Full Fine-tuning (전체 학습)
7B 모델 = 28GB VRAM · A100 80GB 필요 · 체크포인트 14GB/개

✅ PEFT (Parameter-Efficient FT)
원본 가중치 고정 + 작은 어댑터만 학습 · 소비자 GPU로 가능 · 어댑터 수 MB

LoRA — Low-Rank Adaptation

원리: 가중치 업데이트 ΔW 를 두 개의
저랭크 행렬 곱으로 근사 ($W + B \cdot A$)

- 원본 W 는 동결, 작은 $A \cdot B$ 만 학습
- 학습 파라미터 $\approx 0.1 \sim 1\%$
- 추론 시 원본에 병합 가능 (지연 X)
- 여러 어댑터를 task별로 교체 사용

🔧 7B 모델 fine-tuning: A100 1장 (40GB) — 메모리 1/3 절감

QLoRA — Quantized LoRA

원리: LoRA + 원본 가중치 4-bit 양자화
(NF4 + Double Quantization)

- 원본 W 를 16bit $\rightarrow$ 4bit로 압축
- 메모리 사용량 추가 $\sim 75\%$ 절감
- 성능 손실 거의 없음 (논문 검증)
- 24GB GPU 1장으로 65B 모델 학습 가능

🔧 65B 모델 fine-tuning: RTX 3090/4090 1장 — 가정용 GPU로!

💡 라이브러리: 🧑‍🎤 PEFT (peft.LoraConfig) · bitsandbytes (4-bit) · TRL (SFTTrainer / DPOTrainer)

OpenAI GPT

● 생성 모델의 환각 효과 Hallucination Effect

➔ "인공지능(AI) 챗봇에 발생하는 환각(할루시네이션) 현상은 여전한 AI 걸림돌입니다. 발생 이유는 아무도 모릅니다. 그래서 명확한 해결책도 없습니다. 사회에 AI를 안전하게 뿌리내리려면 환각 현상을 어떻게든 해결해야 합니다." – 선다 피차이 (구글 CEO)



문재인 대통령의 미국 방문 때 바이든 대통령과 몇차례의 회담을 했었나?



문재인 대통령은 2023년 5월 20일부터 22일까지 미국을 방문했습니다. 이 기간 동안 문재인 대통령은 조 바이든 미국 대통령과 총 2차례의 회담을 가졌습니다. 첫 번째 회담은 20일 오후 워싱턴 백악관에서 열렸으며, 두 번째 회담은 22일 오전 워싱턴 국무부에서 열렸습니다. 두 차례의 회담에서 양국 정상은 한미 동맹 강화, 북핵 문제, 경제 협력 등 다양한 분야에 대해 논의했습니다.



윤석열 대통령의 미국 방문 때 바이든 대통령과 몇차례의 회담을 했었나?

다른 답안 보기



윤석열 대통령은 2023년 4월 20일부터 22일까지 미국을 방문했습니다. 이 기간 동안 윤석열 대통령은 조 바이든 미국 대통령과 두 차례의 회담을 했습니다. 첫 번째 회담은 4월 20일 백악관에서 열렸고, 두 번째 회담은 4월 21일 백악관에서 열렸습니다. 두 차례의 회담에서 윤석열 대통령과 조 바이든 대통령은 한미동맹 강화, 북한 핵 문제, 경제안보 등 다양한 분야에 대해 논의했습니다.

환각 현상 발생 원인

1. 훈련 데이터의 한계

AI 모델은 훈련 데이터에 의존하며, 데이터에 포함되지 않은 정보나 잘못된 정보로 인해 오류가 발생할 수 있다.

2. 맥락 이해 부족

모델이 모든 맥락을 완벽하게 이해하지 못하고, 일관되지 않은 출력을 생성할 수 있다.

3. 확률적 특성

모델이 확률적으로 다음 단어를 예측하기 때문에, 잘못된 단어를 선택할 가능성이 있다.

→ 환각 문제는 아직 해결되지 않았으며 AI 모델 활용의 가장 큰 도전 과제

생성형 LLM 파인 튜닝

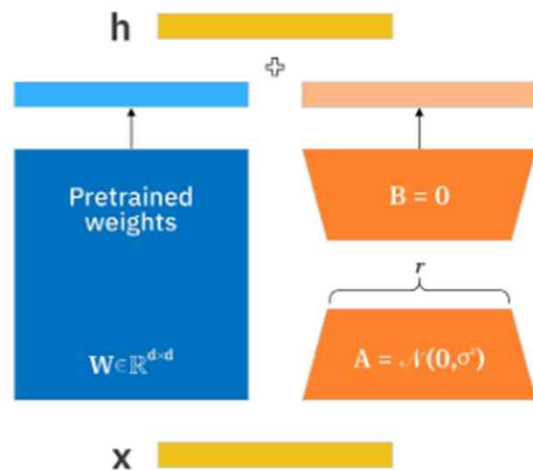
생성형 LLM 파인 튜닝이란?

- **프롬프트·RAG로는 한계가 있다** — 질문에 답하게 하는 것은 프롬프트 엔지니어링이나 RAG로도 가능하지만, 일관된 톤·포맷·정확도까지는 보장하기 어려움
- **출력 형식·스타일 고정** — JSON 스키마, 회사 말투, 답변 길이 등 "원하는 형태로 항상 출력"되도록 모델 자체를 학습시켜 결과 안정성과 재현성을 확보
- **도메인 지식 내재화** — 의료·법률·금융 등 전문 영역의 용어, 규정, 사례를 모델 가중치에 미리 학습시켜 매번 프롬프트로 설명하지 않아도 자연스럽게 활용 가능
- **프롬프트 길이·비용 절감** — few-shot 예시·시스템 지시문을 모델이 이미 "알고 있는" 상태로 만들어 토큰 사용량과 추론 비용·지연 시간을 줄임
- **안전성·정책 정렬** — 사내 가이드라인, 금지어, 응답 거절 규칙을 모델에 직접 학습시켜 프롬프트 우회(jailbreak)에도 일관된 행동을 유지

파인 튜닝 효율 개선

Parameter-Efficient Fine Tuning (PEFT)

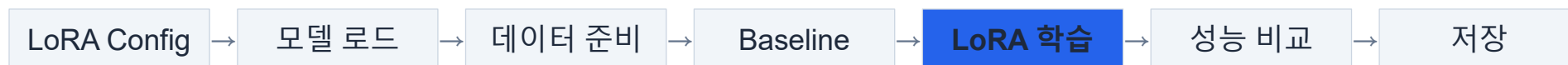
- 기존 모델의 대부분의 Parameter를 고정(freeze) 해 둔 상태에서 일부 weights만 학습하는 방식
- Low-Rank Adaptation(LoRA) 라는 기법이 주로 쓰임



$W: d \times d$ (고정, 학습 안 함)
 $A: r \times d$ (랜덤 초기화, 학습)
 $B: d \times r$ (0으로 초기화, 학습)

$r=16, d=4096$ 이면 학습 파라미터가
 $2 \times 4096 \times 16 = 131K$ 로, full FT의
16.7M 대비 약 0.8%만 학습

실습 : 200_LoRA_KorQuAD Finetuning



실습 설정

모델	Qwen2.5-0.5B (5억 파라미터)
데이터	KorQuAD 1.0 — 5,000개
LoRA r / α	32 / 32 (rank / scale)
Target	all-linear (모든 Linear 레이어에 LoRA 적용)
Epochs	3
Batch	$8 \times \text{grad_accum } 2 = 16$
LR	3e-4, warmup 10%
Precision	fp16

Labels Masking

답변 부분만 loss 계산 → 효율적 학습

-100 (프롬프트: loss 무시) 토큰 ID (학습)

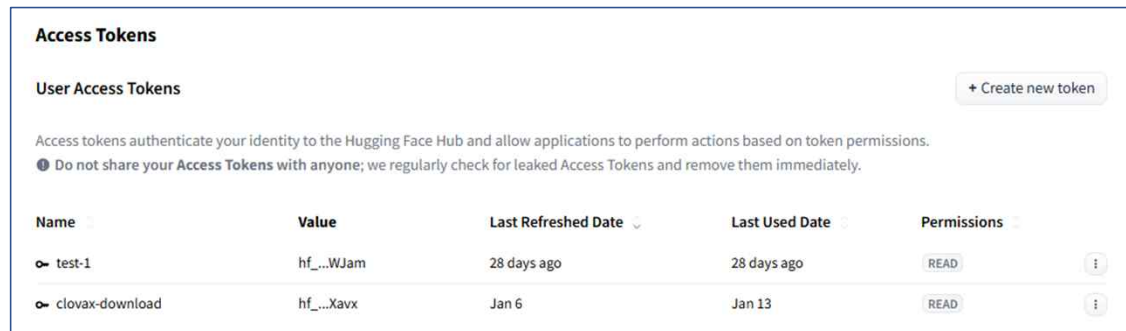
평가 지표 (학습 전 → 후)

- **Perplexity** (모델 혼란도) 높음 → 낮음
- **Loss Curve** 꾸준히 하강

HuggingFace Access Token 생성 <https://huggingface.co/settings/tokens>

1 토큰 페이지 진입

로그인 후 우상단 [+ Create new token] 클릭



2 Token 정보 입력

< Create new Access Token

Token type

Fine-grained **Read** Write

This cannot be changed after token creation.

Token name

test-2

This token has read-only access to all your and your orgs resources and can make calls requests and comment on discussions.

Create token

Token type (Read / Write / Fine-grained) 선택 후 이름 지정

3 토큰 복사 & 보관

Save your Access Token

Save your token value somewhere safe. You will not be able to see it again after you close this modal. If you lose it, you'll have to create a new one.

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 **Copy**

Name	Permissions
test-2	READ

Done

[Create token] 클릭 → 생성된 토큰 [Copy] (한 번만 표시됨)

LoRA, 왜 실무에서 기대만큼 안 되는가?

1 하이퍼파라미터 민감도

r, alpha, target modules 조합이
태스크마다 최적점이 달라 underfitting 위험

2 데이터 품질 문제 증폭

적은 파라미터에 신호 집중 →
garbage in, garbage out이 더 극적

3 Task-Rank 미스매치

스타일 변환은 OK, 도메인 지식 주입은
low-rank의 근본 한계에 부딪힘

4 Base model 의존성

delta 학습이라 base에 없는 능력을
LoRA로 만들어내긴 어려움

5 평가 기준의 문제

벤치마크 점수 ≠ production 체감 품질
perplexity ↓ ≠ 실사용 품질 ↑

6 Merge 후 성능 열화

base에 merge + quantization 시
섬세한 delta가 날아감

핵심: LoRA = "메모리 효율적 fine-tuning" ≠ "마법처럼 적은 데이터로 큰 성능 향상"

LLM (Large Language Model)

활용 방법

LLM 개요

- LLM(Large Language Model)은 대규모 데이터셋을 학습하여 자연어를 이해하고 생성할 수 있는 딥러닝 모델
- 일반적으로 Transformer 기반의 신경망 구조를 사용하며, 사전 훈련(pre-training)과 미세 조정(fine-tuning)을 통해 다양한 자연어 처리(NLP) 작업을 수행
- **LLM의 특징**
 - 방대한 텍스트 데이터를 학습하여 다양한 언어적 패턴을 익힘
 - 질문 응답, 번역, 문서 요약, 코드 생성 등 다양한 작업 자연어 작업 수행 가능
 - Few-shot & Zero-shot Learning - 제한된 예제만으로도 새로운 작업을 수행 가능
 - 멀티모달 - 텍스트뿐만 아니라 이미지, 음성, 동영상과 같은 다양한 입력을 처리

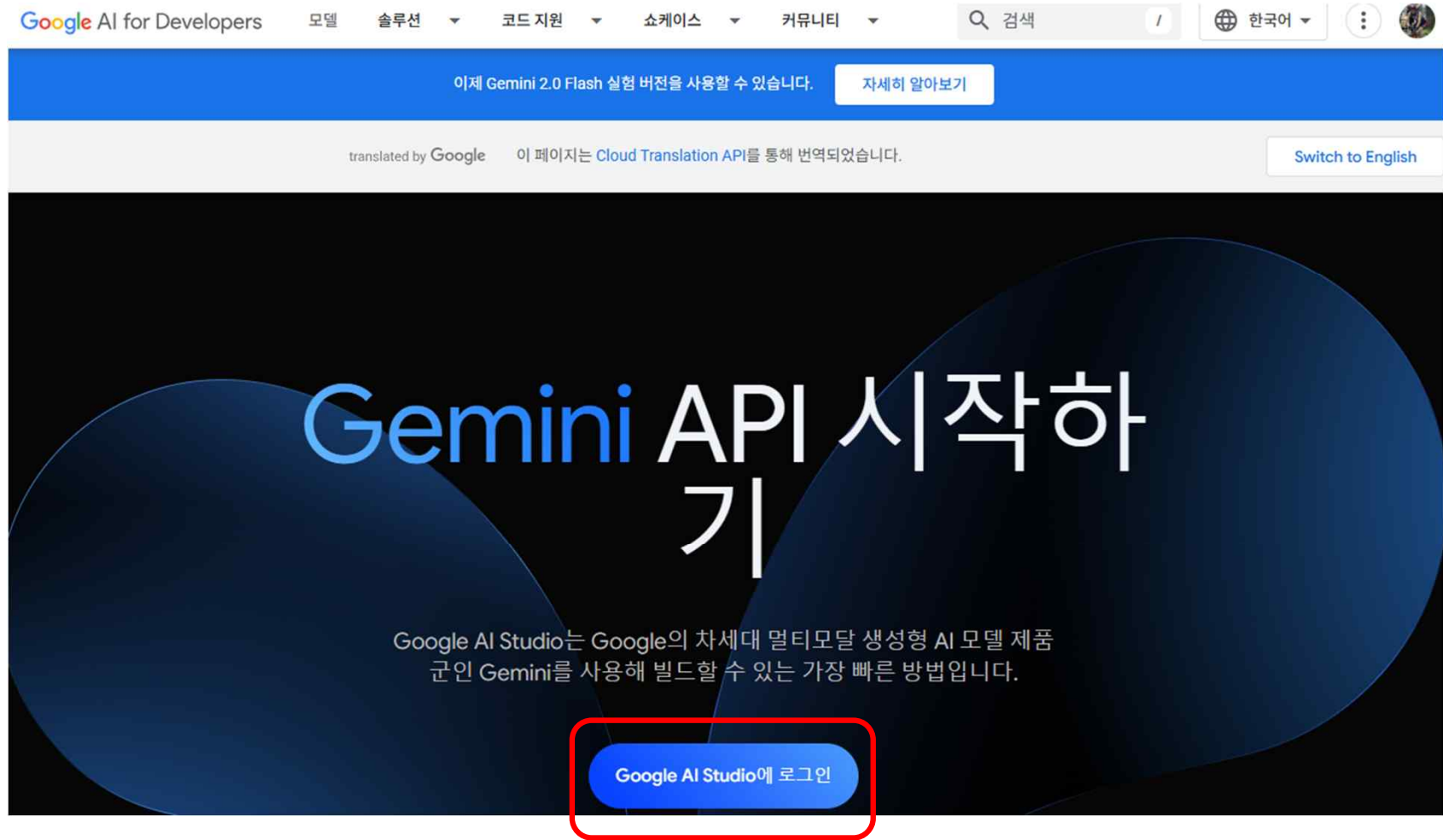
최신 LLM 모델 비교

모델명	발표	주요 특징	특징
GPT-5.5	2026 OpenAI	- 다중 모달 지원 (텍스트, 이미지, 음성) - 빠른 응답	AI 챗봇(ChatGPT)
Claude 4.7 Opus	2026 Anthropic	- 안전성을 강화한 AI 모델 - 강력한 추론 및 논리적 사고 능력	AI 윤리 및 법률 도구 교육 및 연구 보조 데이터 분석
Gemini 3 Pro	2026 Google	- 멀티모달 처리 지원 - 1백만 토큰 이상의 컨텍스트 윈도우 지원	복잡한 대화형 AI 시스템 다국어 번역 및 문서 생성
Llama 4	2025 Meta	- 오픈소스 LLM	경량화된 AI 모델 연구
Grok 4.2	2026 xAI	-복잡한 수학·추론 문제에 강점	소셜 미디어 통합형 + 실험적 모델
DeepSeek	2025 DeepSeek	- 오픈 소스 LLM (MIT 라이선스) - 혼합전문가(Mixture of Experts) 구조와 강화학습으로 모델 개선	개선저비용 고효율 추구

LLM API (Application Programming Interface)

- API란 응용 프로그램 간의 상호 작용을 위한 인터페이스
- 복잡한 모델을 직접 구축하지 않고도 LLM의 기능 활용 가능
- 최신 모델의 기능을 손쉽게 통합 가능
- 주요 LLM API 제공 업체 - OpenAI, Anthropic, Google DeepMind 등
- API 활용 시 고려사항
 - 요금 구조
 - 요청 제한
 - 보안 및 개인정보 보호

Gemini API KEY 생성



Google AI Studio

< Dashboard

API 키

프로젝트

Usage and Billing

로그 및 데이터 세트

변경 로그



프로젝트

📖 프로젝트 이해

+ 새 프로젝트 만들기

🔗 프로젝트 가져오기

🔍 프로젝트 검색

프로젝트

키

생성일

할당량 등급

Demo-1

phonic-studio-467301-n0

키 2개

2025. 7. 28.

결제 설정

무료 등급



새 프로젝트 만들기



프로젝트 이름 지정

취소

프로젝트 만들기

Google AI Studio

Get API key

Create new prompt

New tuned model

My library

Allow Drive access

Getting started

Documentation

Prompt gallery

Gemini cookbook

Discourse forum

Build with Vertex AI on Google Cloud

API 키 가져오기

API 키

아직 프로젝트가 없으면 새 프로젝트를 만들거나 기존 프로젝트에 API 키를 추가할 수 있습니다. 모든 프로젝트에는 새 프로젝트를 만들 때 동의하는 [Google Cloud Platform 서비스 약관](#)이 적용되며, Gemini API 및 Google AI Studio 사용 시에는 [Gemini API 서비스 약관](#)이 적용됩니다.

API 키는 안전하게 사용합니다. 다른 사람이 볼 수 있는 코드에 키를 공유하거나 삽입하지 마세요.

결제가 사용 설정된 프로젝트에서 Gemini API를 사용하는 경우 [사용한 만큼만 지불하는 가격 책정 방식](#)이 적용됩니다.

키 API 키 만들기

API 키는 아래와 같습니다. 또한 Google Cloud에서 프로젝트와 API 키를 확인하고 관리할 수 있습니다.

프로젝트 번호	프로젝트 ID	API 키	생성일	요금제
...4627	Generative Language Client	...Pwxg	2024. 7. 18.	무료 결제 설정 사용 데이터 보기

.env 에 저장

GOOGLE_API_KEY=created-api-key

단일 프로젝트에 대해 API key 설정 방법

- .env – API 키가 포함된 로컬 파일 생성

```
# .env 파일  
OPENAI_API_KEY=sk-vvDtl*****XiilEpdjLhBJaH0f
```

- .gitignore 에 .env 파일 포함

```
# .env 파일을 git에서 무시  
.env
```

- Python code

```
pip install python-dotenv
```

```
from dotenv import load_dotenv, find_dotenv
```

```
_ = load_dotenv(find_dotenv()) # local .env file을 읽어서 os.environ 에 OPENAI_API_KEY 추가
```

```
from openai import OpenAI
```

```
client = OpenAI() # os.environ.get("OPENAI_API_KEY")을 default 로 사용하여 API Key 이용
```

LLM의 입력 구조 — Message Role

System	User	Assistant
모델의 역할·성격·제약 정의 전체 대화의 가이드라인 설정 예: "친절한 한국어 튜터입니다."	사용자의 실제 질문·요청 태스크의 핵심 입력 예: "이 문장을 영어로 번역해줘"	모델이 생성한 응답 Few-shot 예시에서는 정답 역할 다음 turn의 context로 전달

💡 세 메시지가 순서대로 모델에 전달되며, 전체가 하나의 *prompt*로 합쳐져 다음 *token*을 예측한다

In-context Learning — Zero / One / Few-shot

모델 가중치를 바꾸지 않고, prompt에 예시를 몇 개 주느냐로 성능을 끌어올린다

Zero-shot 예시 0개

다음 리뷰의 감성을
긍정/부정으로 분류해줘.

리뷰: "배송이 빠르고
품질이 좋아요"
→ ?

✓ 가장 간단
X 정확도 낮을 수 있음

One-shot 예시 1개

리뷰: "별로예요" → 부정

리뷰: "배송이 빠르고
품질이 좋아요"
→ ?

✓ 형식·톤 빠르게 학습
△ 한 예시에 편향 위험

Few-shot 예시 2~수십 개

리뷰: "별로예요" → 부정
리뷰: "최고!" → 긍정
리뷰: "그저 그래요" → 중립

리뷰: "배송이 빠르고
품질이 좋아요" → ?

✓ 일관성·정확도 우수
X 토큰 비용 증가

Chain-of-Thought (CoT) — 단계적 추론 유도

정답만 요구하는 대신 "풀이 과정을 단계적으로 보여달라"고 유도하면 복잡한 추론 정확도가 크게 향상된다

❌ Standard Prompt

Q: 카페에 사과 23개가 있다.
20개를 점심에 쓰고
6개를 더 사왔다면
지금 사과는 몇 개?

A: 27개

→ 단순 추측, 자주 틀림

✅ Chain-of-Thought

Q: 카페에 사과 23개가 있다.
20개를 점심에 쓰고
6개를 더 사왔다면?
단계적으로 풀어줘.

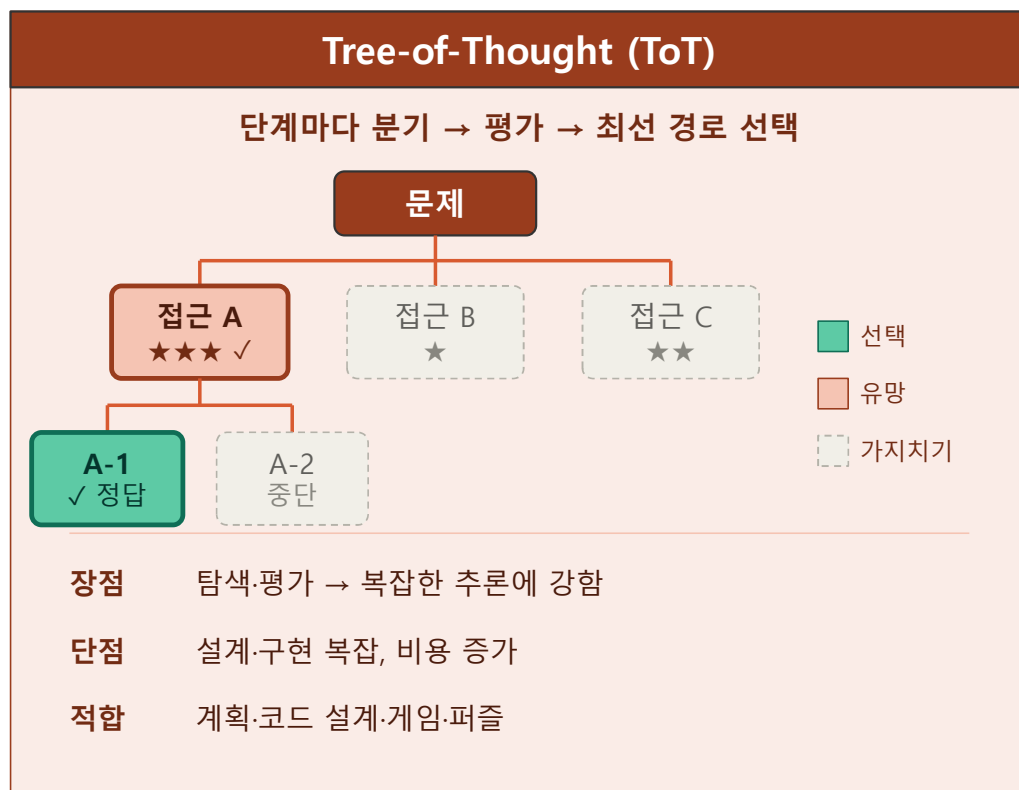
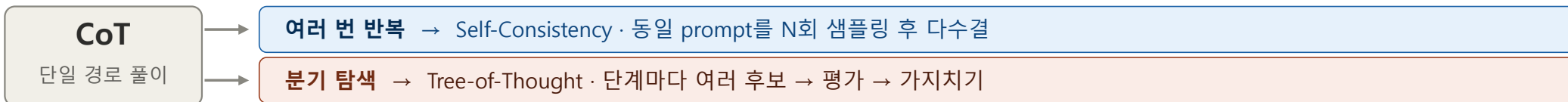
A: 처음 23개 - 점심 20개 = 3개
3개 + 추가 6개 = 9개
∴ 9개

→ 추론 과정 노출, 정확도 ↑

🔑 트리거 문구: "Let's think step by step" · "단계적으로 풀어줘" · "풀이 과정을 보여줘"

고급 추론 — Self-Consistency & Tree-of-Thought

CoT를 "여러 번 반복"하거나 "분기 탐색"하여 더 안정적인 답에 도달하는 기법



Structured Output — JSON Schema & Pydantic

자연어 응답 대신 미리 정의된 스키마에 맞춘 JSON 객체를 받아 — 후처리 없이 바로 코드에서 사용

JSON Schema 방식

```
response_schema = {
  "type": "object",
  "properties": {
    "sentiment": {
      "type": "string",
      "enum": ["positive",
               "negative",
               "neutral"]
    },
    "confidence": {
      "type": "number"
    }
  },
  "required": [
    "sentiment",
    "confidence"
  ]
}
```

Pydantic 방식 (Python)

```
from pydantic import BaseModel
from typing import Literal

class ReviewResult(BaseModel):
    sentiment: Literal[
        "positive",
        "negative",
        "neutral"]
    confidence: float
    keywords: list[str]

# OpenAI · Gemini SDK 모두
# response_format / response_schema
# 인자로 직접 전달 가능
result: ReviewResult = ...
```

💡 장점: 파싱 에러 제거 · 타입 검증 · API 응답을 그대로 함수 인자로 사용 (Function Calling 기반)

효과적 프롬프트 작성 — 4가지 원칙

막연한 "잘 해줘" 대신 — 누가, 무엇을, 어떤 제약 하에, 어떤 형태로 답해야 하는지 명시한다

역할 (Role)

모델에 페르소나를 부여

예: "당신은 10년 경력의 Python 코드 리뷰어입니다"

컨텍스트 (Context)

배경·목적·대상 청중 제공

예: "이 코드는 대학생을 위한 교육용이며, 가독성 우선"

제약 (Constraints)

길이·형식·금지사항 명시

예: "3문장 이내, JSON 형식, 외부 라이브러리 금지"

예시 (Examples)

원하는 입출력 형태를 시연

예: "입력 X → 출력 Y 형식으로 동일하게 답해주세요"

실습 : 300 - 5가지 prompt 기법 비교 + Hugging Face 호출

 공통 태스크: 영화 리뷰의 감성 분류 + 그렇게 판단한 근거 키워드 추출

기법	Prompt 핵심	관찰 포인트
Zero-shot	이 리뷰의 감성을 분류해줘.	기본 정확도 측정 (baseline)
Few-shot	예시 3개를 보여준 뒤 동일 형식으로 답하게 함	예시가 정확도·일관성을 얼마나 올리는가
CoT	"단계적으로 판단 근거를 설명한 후 결론"	추론 노출이 오류를 줄이는가
Self-Consistency	동일 CoT prompt를 5회 호출 → 다수결	단발성 오류 제거 효과
Structured Output	Pydantic 스키마로 sentiment / confidence / keywords 반환	후처리 코드 없이 바로 사용 가능한가

실습 : 301 - 5가지 prompt 기법 비교 (Gemini API 버전)

동일한 5가지 기법을 그대로 — 모델 백엔드만 로컬 HyperCLOVAX → Gemini API 로 교체

핵심 차이점 — 300 (로컬 HyperCLOVAX) vs 301 (Gemini API)		
항목	300 — HyperCLOVAX (로컬)	301 — Gemini API
모델 / 실행	HyperCLOVAX-SEED-1.5B 로컬 다운로드 Colab GPU 권장 · HF_TOKEN 필요	gemini-2.5-flash-lite REST API 호출 GPU 불필요 · GOOGLE_API_KEY 만 있으면 OK
Structured Output	프롬프트로 JSON 형식 지시 → 가끔 형식 어김 · try/except 필요	response_schema(Pydantic) 로 형식 강제 → native JSON 보장 · 후처리 코드 불필요
Self-Consistency	동일 CoT prompt 5회 호출 → 다수결	무료 API 호출 절약 — 3회 호출로 축소
비용 / 제약	한 번 다운로드 후 무료 · GPU 메모리 필요	무료 등급 RPM·일일 한도 → throttle + 429 자동 재시도 로직

- Structured Output 의 차이 — 로컬 소형 모델은 형식 안정성이 떨어져 후처리가 필요했지만, Gemini 는 스키마 자체를 강제한다.

실습 : 310-Project_Movie_감성분석

딥러닝 기반 영화 리뷰 감성 분석 — 사전학습 모델 두 가지로 같은 문제 풀기

1 영화 리뷰 데이터

네이버 영화 리뷰 (NSMC)

한국어 영화평 데이터셋
결측치 제거 등 최소 전처리

2 감성 분류 목표

이진 분류 (binary)

긍정(1) / 부정(0) 두 클래스로 판별
fine-tuning 없이 사전학습 모델만 사용

3 방법 1 — Hugging Face Pipeline

BERT 기반 전용 모델

pipeline('sentiment-analysis')
별점(1~5) 출력 → 긍/부정 변환

4 방법 2 — ClovaX

HyperCLOVAX LLM 기반

프롬프트로 "긍정/부정" 지시
자유 텍스트 → 키워드 파싱

전용 BERT Pipeline vs 범용 LLM 프롬프트 — 같은 문제, 두 가지 도구의 강점 비교

자율 실습 : 311 영화 리뷰 감성분석 — Pipeline vs ClovaX

학습(fine-tuning) 없이 사전학습 모델만으로 같은 문제 풀기 — 두 가지 접근 비교

방법 1 : Hugging Face Pipeline

감성분석 전용 사전학습 모델을 그대로 호출
nlptown/bert-base-multilingual-uncased-sentiment

방법 2 : ClovaX (생성형 LLM)

범용 LLM 에게 프롬프트로 분류 작업 지시
HyperCLOVAX-SEED-Text-Instruct-1.5B

과제 1

데이터 준비

NSMC 테스트 1,000건 표본
공정 비교용 5문장 추출

과제 2

Pipeline 감성분석

pipeline('sentiment-analysis')
별점(1~5 stars) 출력

과제 3

Pipeline 평가

별점 $\geq 4 \rightarrow$ 긍정 변환
정확도 계산

과제 4

ClovaX 프롬프트

chat 템플릿 + system/user
자유 텍스트 출력 파싱

과제 5

ClovaX 평가

동일한 5문장으로 평가
공정한 비교 위함

과제 6

두 방법 비교

정확도 · 속도 · 유연성
어떤 도구가 언제 좋은가

핵심 : 정해진 작업엔 전용 Pipeline · 유연성이 필요할 땐 프롬프트 기반 LLM — 문제에 맞는 도구 선택

LLM의 한계와 해결책

LLM의 한계

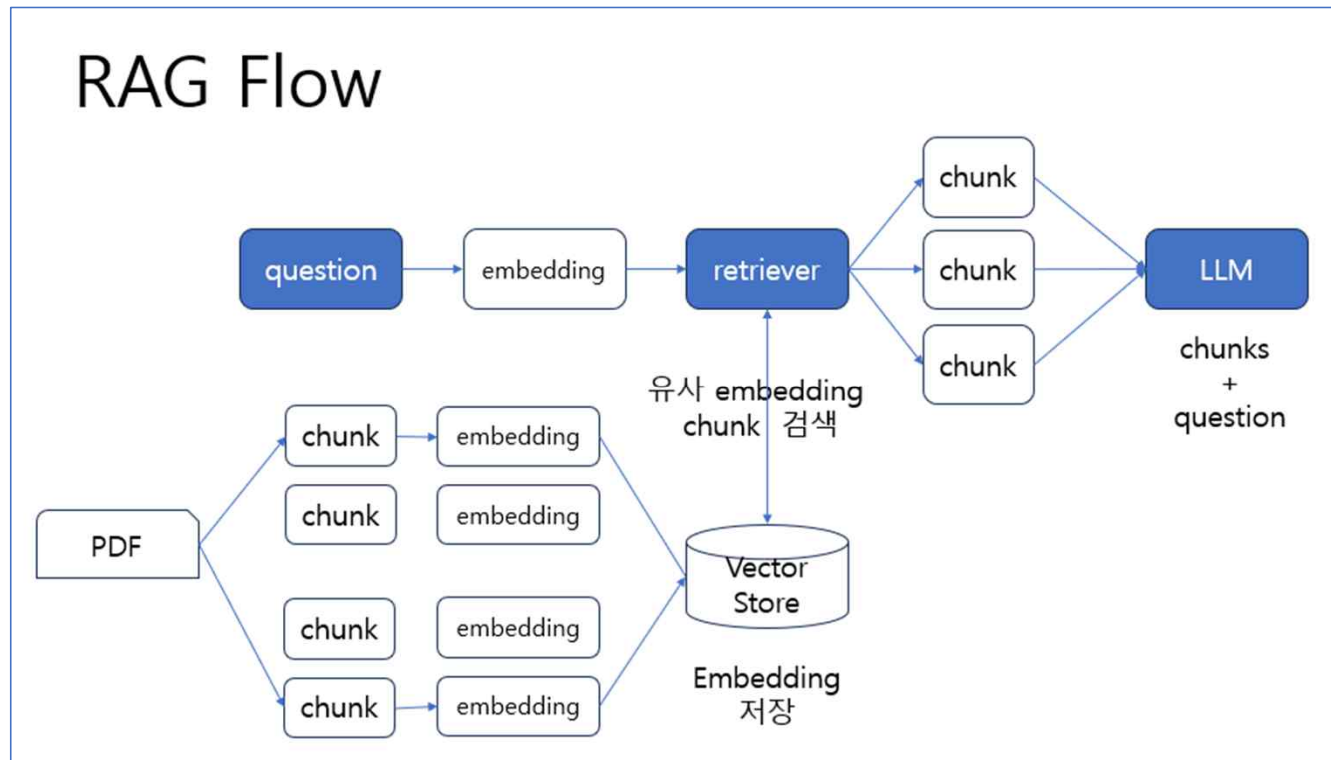
- 학습 시점 이후의 최신 정보를 알지 못함 (Knowledge Cutoff)
- 사내 문서·도메인 지식 등 비공개 데이터에 접근 불가
- 그럴듯하지만 사실이 아닌 답변 생성 (Hallucination)
- 답변의 출처를 확인하기 어려움

해결 아이디어

- 모델을 다시 학습시키는 대신, **필요한 정보를 외부에서 검색해 함께 입력**
- Retrieval(검색) + Generation(생성) = **RAG**

RAG (Retrieval Augmented Generation)

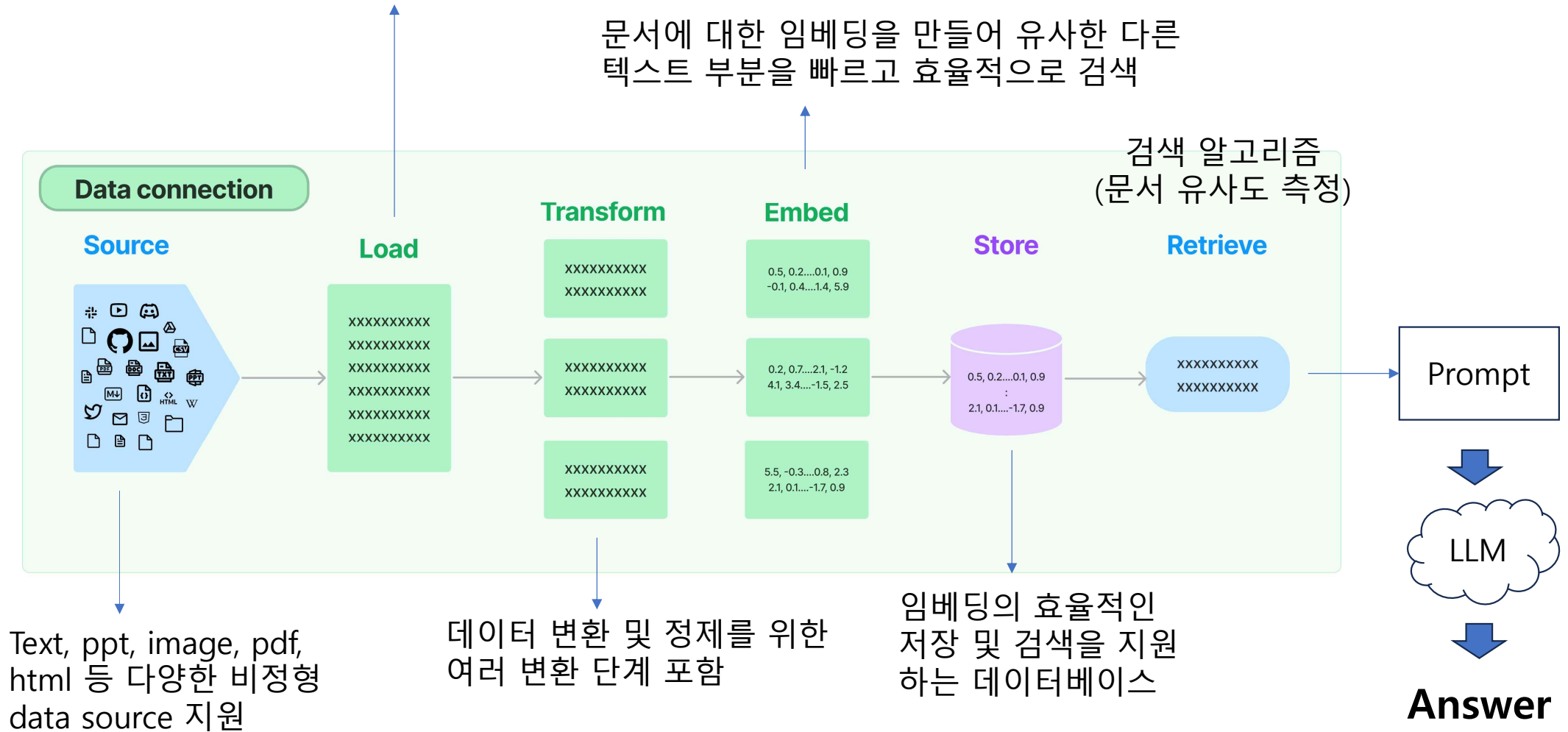
- LLM이 내부 지식만으로 답하는 방식이 아니라, 질문과 관련된 문서를 추가로 제공받아 그것을 근거로 답변 생성



다양한 소스(Web Site, DB, YouTube 등)에서 문서 (HTML, PDF, JSON, Word, ppt, 코드 등) 로드

문서에 대한 임베딩을 만들어 유사한 다른 텍스트 부분을 빠르고 효율적으로 검색

검색 알고리즘
(문서 유사도 측정)



Chunking — 문서를 어떻게 자르느냐가 RAG 품질의 절반

청크가 너무 크면 → 노이즈 ↑, 너무 작으면 → 문맥 단절. 4가지 전략과 트레이드오프

Fixed-size	Recursive	Semantic	Document-aware
고정 길이 (512~1024 토큰) Overlap: 50~100	단락→문장→토큰 계층적 분할	임베딩 유사도 기반 의미 단위 분할	Markdown / HTML 구조 섹션·헤더 기반
✓ 단순 · 빠름	✓ 구조 보존	✓ 문맥 정확	✓ 최고 품질
✗ 문장 중간 절단	✗ 튜닝 필요	✗ 비용 · 느림	✗ 포맷 의존

실전 가이드라인

- 청크 크기: 200~500 토큰 (한국어 기준 400~1000자) — 너무 길면 검색 정확도 ↓
- Overlap: 청크 길이의 10~20% — 경계에서 문맥이 잘려도 인접 청크가 메꿈
- 메타데이터 동봉: 제목·섹션·페이지·날짜 → 검색 후 필터링·표시에 활용
- LangChain RecursiveCharacterTextSplitter / LlamaIndex SentenceSplitter가 표준

💡 첫 RAG는 Recursive + 500토큰/100overlap으로 시작 → 검색 품질 평가 후 Semantic으로 업그레이드

RAG 파이프라인 — 6단계 흐름

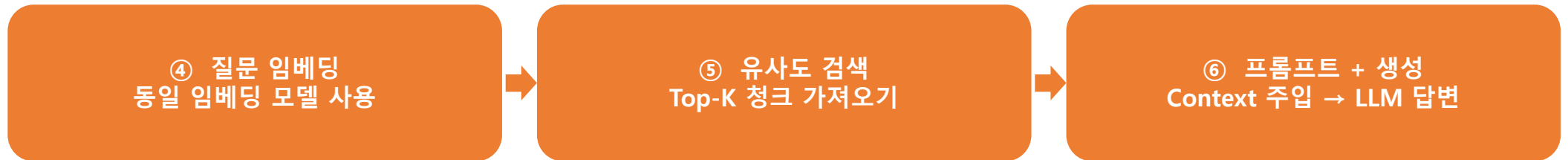
질문이 들어오면 → 관련 문서를 찾고 → LLM에 함께 넣어 답을 생성한다

■ Indexing (오프라인 — 사전 작업)



사용자 질문 도착

■ Querying (온라인 — 질문 시점)



💡 핵심: ① ~ ③은 한 번만 (Indexing) — ④ ~ ⑥은 질문마다 매번 (Querying)
임베딩 품질 = ② 단계, 검색 품질 = ⑤ 단계, 답변 품질 = ⑥ 단계가 좌우

임베딩 모델 — 텍스트를 벡터로 바꾸는 핵심

같은 의미의 문장은 벡터 공간에서 가깝게 — 코사인 유사도로 측정

OpenAI text-embedding-3-small

1536d · 다국어 · 유료
범용 표준 — 가성비 ↑

BGE-M3

1024d · 다국어 · 오픈소스
한국어 성능 우수

KURE-v1 (NCSoft)

1024d · 한국어 특화
국내 RAG 1순위 추천

Cohere embed-multilingual-v3

1024d · 100+ 언어
엔터프라이즈 안정성

📌 임베딩 모델 선택이 RAG 품질에 미치는 영향

- 한국어 RAG는 한국어 학습이 충분한 모델 필수 → 영어 전용 모델은 한국어 검색 정확도 30~50% ↓
- 차원 수가 클수록 표현력 ↑ but 저장·검색 비용 ↑ — 보통 768~1536이 균형점
- 인덱싱 시 사용한 모델과 질의 시 모델이 동일해야 함 — 모델 교체 = 전체 재인덱싱

💡 한국어 중심 프로젝트 → KURE-v1, 다국어 혼용 → BGE-M3, 운영 안정성 → OpenAI/Cohere

벡터 데이터베이스 — 수십억 벡터에서 Top-K를 빠르게

ANN(Approximate Nearest Neighbor) 검색 — 정확도를 약간 양보하고 속도를 100배 ↑

벡터 DB	형태	강점	약점	추천 시나리오
FAISS	라이브러리 (로컬)	Meta 제작 · 매우 빠름 · 무료	단일 머신 · 메타데이터 빈약	실습/프로토타입/PoC
Chroma	임베디드 DB	Python 친화 · 메타필터 지원 · 가벼움	대규모 분산 한계	소~중규모 RAG 앱
Pinecone	Managed 클라우드	운영 무료 · 스케일링 자동	유료 · Vendor lock-in	프로덕션 SaaS
Weaviate	오픈소스 + 클라우드	하이브리드 검색(BM25+벡터)	운영 복잡도 ↑	엔터프라이즈 자체호스팅
pgvector	PostgreSQL 확장	기존 RDB와 통합 · SQL	초대규모는 전용 DB만 못함	기존 PG 인프라 활용

💡 강의 실습은 FAISS — 무료·로컬·빠름. 운영은 Pinecone(쉬움) vs pgvector(통합) 사이에서 선택

실습: 320/321. RAG 예제

- ClovaX/Gemini API + KURE-v1 사용
- RAG(Retrieval Augmented Generation)의 전체 흐름을 처음부터 끝까지 직접 구현
 - 문서 준비 (Source)
 - 문서 분할 (Transform – Chunking)
 - 임베딩 생성 (Embed)
 - 벡터 저장소 구성 (Store)
 - 유사도 검색 (Retrieve)
 - RAG 핵심 로직 (Retrieve + Generate) 구현
 - 일반 LLM vs RAG 비교

AI Agent — LLM에서 자율 행동으로

LLM에서 Agent로 — 응답에서 행동으로의 진화

LLM은 "무엇을 답할까"를 생성, Agent는 "무엇을 할까"를 결정·실행한다

LLM (지금까지)

- 입력 → 텍스트 출력 (1회성)
- 외부 세계 정보 접근 불가
- 계획·재시도 능력 없음
- "날씨 어때?" → 학습 시점 일반론만
- Stateless: 매 호출이 독립적

AI Agent (지금부터)

- 입력 → 사고(Reason) + 도구 호출 + 관찰 반복
- API·검색·DB 등 외부 도구 사용 가능
- 실패하면 다시 계획·재시도
- "날씨 어때?" → 위치 파악 → 날씨 API 호출 → 결과 요약
- Stateful: 대화·메모리 유지

 **핵심: Agent = LLM + 도구(Tools) + 루프(Loop) + 메모리(Memory)**
→ LLM은 "두뇌"일 뿐, Agent는 "손과 발을 가진 두뇌"

Function Calling — LLM이 도구를 호출하는 표준 방식

LLM이 직접 함수를 "호출"하지는 않음 — JSON Schema에 맞춰 "호출 의도"를 출력하고, 앱이 실제 실행

① 도구 정의

JSON Schema로
함수 시그니처 등록

② 의도 출력

LLM이 호출할 함수와
인자를 JSON으로 반환

③ 실제 실행

앱(클라이언트)이 함수
실행 → 결과 획득

④ 결과 주입

결과를 다시 LLM에
전달 → 최종 답변

도구 정의 예시 (OpenAI Function Calling Schema)

```
{
  "name": "get_weather",
  "description": "특정 도시의 현재 날씨를 조회",
  "parameters": {
    "type": "object",
    "properties": { "city": { "type": "string", "description": "도시 이름" } },
    "required": ["city"]
  }
}
```

 Agent의 출발점 = Function Calling. 모든 도구 사용 Agent는 이 메커니즘 위에 동작

ReAct 패턴 — Thought · Action · Observation 루프

Reasoning + Acting — LLM이 "왜 이걸 하는지"를 매 단계 말로 풀어내며 행동하는 가장 보편적인 Agent 패턴



↻ Observation의 결과를 새 Thought로 → 종료 조건(최종 답변) 전까지 반복

📄 실제 trace 예시 — "서울과 도쿄 중 어디가 더 따뜻해?"

Thought: 두 도시의 현재 온도를 비교해야 한다. 먼저 서울부터 조회.
Action: `get_weather(city="Seoul")`
Observation: { "temp": 12, "unit": "°C" }
Thought: 이제 도쿄도 조회한다.
Action: `get_weather(city="Tokyo")`
Observation: { "temp": 17, "unit": "°C" }
Thought: 도쿄가 5도 더 높다. 최종 답변 준비.
Action: 최종 답변 → "도쿄가 17°C로 서울(12°C)보다 5도 더 따뜻합니다."

💡 ReAct = "생각을 말로 적고 행동하기" — 디버깅 쉬움, 해석 가능, 대부분의 Agent 프레임워크의 기본 패턴

MCP — 도구 연결의 표준 프로토콜 (Model Context Protocol)

Anthropic이 2024년 11월 공개한 오픈 표준 — "AI를 위한 USB-C"라고 불리는 도구·데이터 연결 규약

❌ MCP 이전 — N×M 통합 지옥

- 모델 N개 × 도구 M개 = N×M 통합 코드
- 도구마다 직접 API 클라이언트 작성
- 프레임워크별로 다른 도구 등록 방식
- 도구 추가/삭제 시 모든 Agent 재배포

✅ MCP — 표준 인터페이스

- Client(모델) ↔ Server(도구) 1:1 표준 통신
- Server 한 번 만들면 모든 MCP 호환 모델이 사용
- 도구·리소스·프롬프트의 3종 제공자
- Claude Desktop, Cursor, VS Code 등 점진 채택 중

📦 MCP 3대 요소

Tools
함수 호출 (Function Calling)

Resources
파일·DB 등 컨텍스트 데이터

Prompts
재사용 가능한 프롬프트 템플릿

💡 MCP는 "도구의 생태계"를 만든다 — 누군가 만든 MCP Server를 그대로 우리 Agent에 꽂아 쓰는 시대

Agent 4대 구성요소 — Memory · Planning · Tools · Reflection

LLM은 Agent의 "두뇌" — 진짜 Agent를 완성하는 4가지 부속

Memory 기억

- 단기: 현재 대화 컨텍스트
- 장기: Vector DB + 임베딩으로 영속화
- 예) 사용자 선호도, 과거 작업 이력

Tools 도구

- Function Calling으로 외부 기능 호출
- 검색 API, 코드 실행, DB 쿼리, 파일 I/O
- MCP 표준으로 도구 생태계 확장 가능

Planning 계획

- 큰 작업을 단계로 분해 (Task Decomposition)
- Chain-of-Thought / Plan-and-Execute
- 예) "보고서 작성" → 자료 수집 → 초안 → 검토

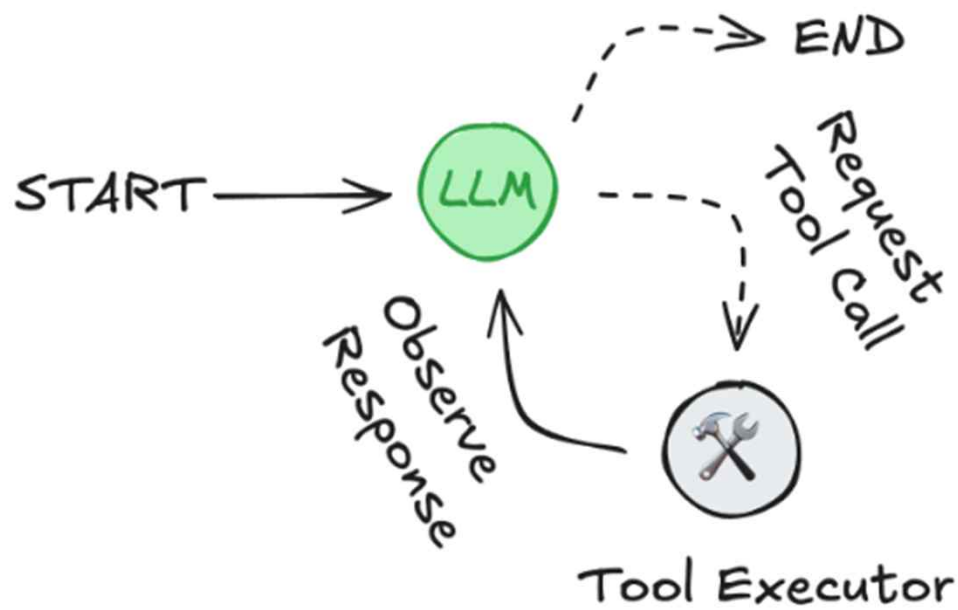
Reflection 성찰

- 결과를 스스로 평가·재시도 (Self-Critique)
- 실패한 trace를 분석해 다음 시도 개선
- 예) 코드 실행 실패 → 에러 읽고 수정

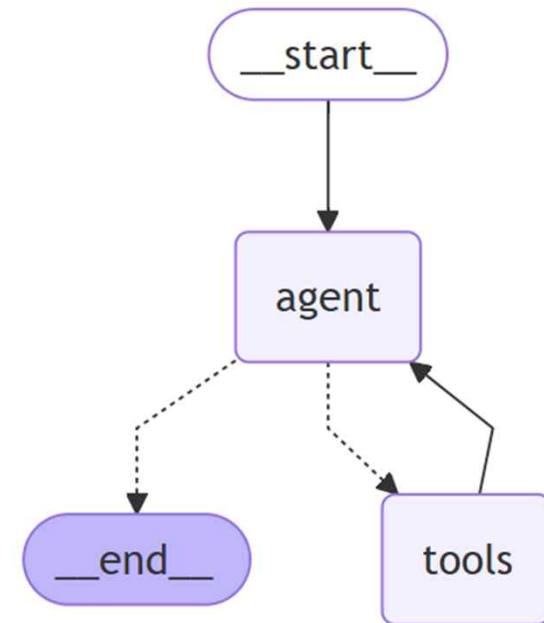
 Agent = LLM(두뇌) + Memory(기억) + Planning(계획) + Tools(손발) + Reflection(거울)

에이전트 루프(Agent loop)

- LLM이 도구들을 선택하고, 그 도구들의 출력을 활용하여 사용자의 요청을 수행



Graph



주요 Agent 프레임워크 — 어떤 도구를 언제 쓸까

2025년 기준 가장 활발히 쓰이는 5가지 — 각각의 강점과 적합한 상황을 한눈에 비교

프레임워크	특징	강점	적합한 상황
LangGraph	그래프 기반 상태 머신	복잡한 워크플로우·분기·반복	Production-grade Agent
OpenAI Agents SDK	Handoff 중심 경량 SDK	직관적·OpenAI 생태계 통합	OpenAI API 위주 개발
Anthropic Claude SDK	MCP·Tool Use 일등 시민	장문 컨텍스트·신뢰성 ↑	Claude 기반 Agent
AutoGen (MS)	Multi-Agent Conversation	Agent 간 대화·토론 패턴	연구·실험적 Multi-Agent
CrewAI	Role-based Multi-Agent	캐릭터 정의 직관적	빠른 PoC, 데모

- 학습 추천 순서: ① OpenAI Agents SDK (개념 파악) → ② LangGraph (실무 능력) → ③ MCP·Multi-Agent로 확장
- 프레임워크는 "수단" — 핵심은 ReAct 루프, Function Calling, 상태 관리의 이해

💡 단순 Tool Use는 SDK 한 줄, 분기·메모리·재시도가 필요해지면 LangGraph로 — 복잡도에 맞게 도구 선택

실습 : 400 - LangChain Tools & ReAct Agent 직접 만들기

도구 정의 → 에이전트 생성 → 실행까지, LLM이 외부 도구를 스스로 호출하는 ReAct 에이전트를 구현

① 도구(Tools) 정의

✂ 함수를 LLM이 쓸 수 있는 도구로 변환하는 3가지 방법

- @tool 데코레이터 + Type Hints + Docstring
- 이름.설명 속성 커스터마이징
- Pydantic args_schema 고급 스키마

② 에이전트 생성·실행

✂ LangChain으로 프로덕션용 ReAct 에이전트 구성

- create_agent(model, tools)
- agent.invoke({"messages": [...]})
- system_prompt으로 행동 방식 제어

③ 예제 도구로 검증

✂ 3가지 도구로 에이전트 동작 확인

- search_db : 고객 DB 레코드 조회
- calculator : 산술 계산
- get_weather : Open-Meteo 실시간 기온 조회

실습 : 410 - RAG Agent (검색 증강 생성)

외부 지식 소스(웹 문서)를 검색해 LLM 응답을 보강하는 RAG 에이전트를 직접 구현

① 인덱싱 (Indexing)

✂ 웹 문서를 임베딩해 벡터 스토어에 저장

- WebBaseLoader: 블로그 페이지 수집
- RecursiveCharacterTextSplitter (chunk 1000 / overlap 200)
- Gemini 임베딩
- InMemoryVectorStore에 저장

② 검색 & 생성 두 가지 방식

✂ 체인 vs 에이전트 방식 비교

- 체인 기반 RAG
항상 검색 후 답변 - 단순·일관적
- 에이전트 기반 RAG
필요할 때만 검색 도구 호출
유연함 ↔ 지연 시간 ↑

③ RAG 에이전트 구성

✂ 검색을 도구로 쓰는 에이전트 구현

- 검색 도구: @tool로 래핑
→ retrieve_context(query)
- create_agent(model, tools, system_prompt)
- agent.stream(...)으로 과정 확인

실습 : 420 - MCP Agent 표준 프로토콜로 도구 연결

MCP (Model Context Protocol) — LLM·에이전트를 외부 도구에 연결하는 오픈 표준, "AI를 위한 USB-C"

① MCP 핵심 개념

✂ 도구 제공자와 사용자 분리

- Client — 모델·에이전트 쪽 Server에 접속해 도구 가져옴
- Server — 도구 제공 프로세스 Tools / Resources / Prompts
- 한 번 만든 Server를 어디서나 재사용 가능

② 실습 흐름

✂ Server 구축 → Client 연결

- FastMCP + @mcp.tool()로 add · multiply 도구 정의
- streamable-http로 서버 기동
- MultiServerMCPClient로 서버 URL 등록
- await client.get_tools() → LangChain 도구로 변환

③ @tool(400) vs MCP(420)

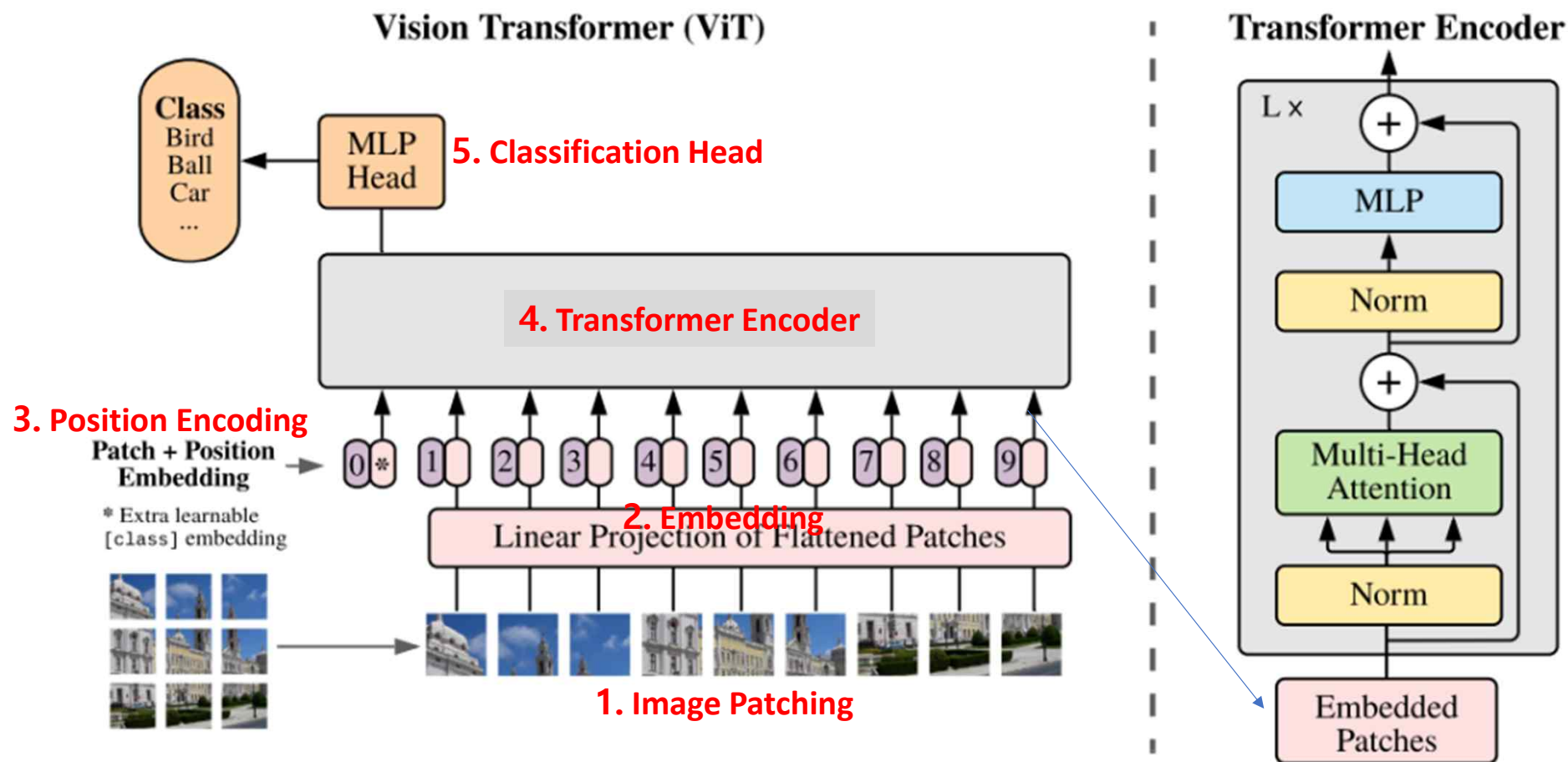
✂ create_agent는 그대로 재사용

- @tool: 에이전트 코드 안
→ 프로젝트 전용 간단한 도구
- MCP: 별도 Server 프로세스
→ 표준 프로토콜, 공용·외부 제공
- transport: stdio(로컬) / streamable-http(원격, 노트북용)

Vision Transformer

Model Overview

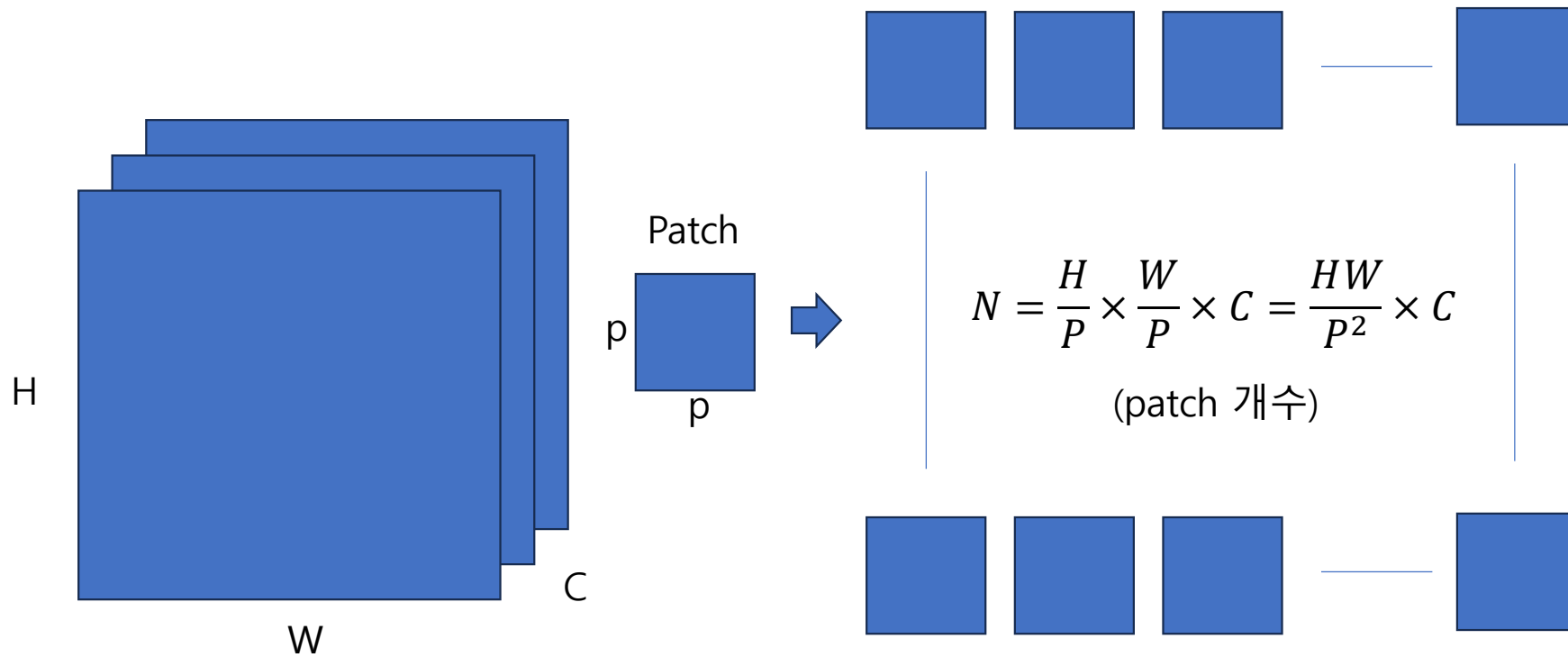
Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M



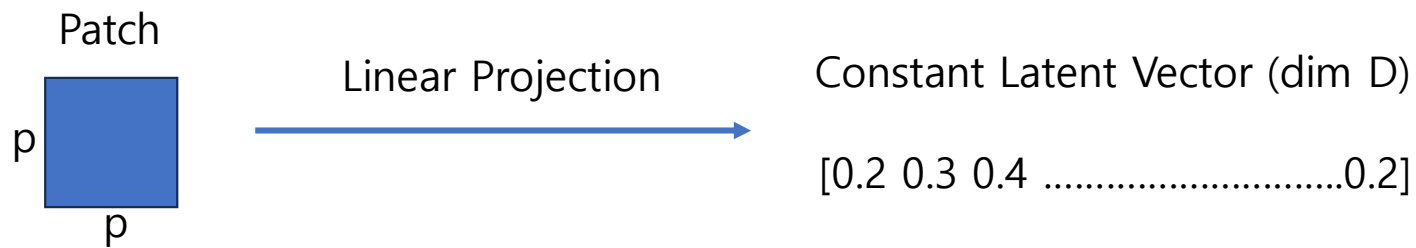
ViT의 작동 방식

- 1. Image Patching:** 주어진 이미지를 여러 개의 고정 크기의 패치(patch)로 나눕니다.
예를 들어, 224×224 크기의 이미지를 16×16 크기의 패치로 나누면 총 196개의 패치가 생깁니다.
- 2. Embedding:** 각 패치를 1D 벡터로 평탄화(flatten)한 후, 선형 임베딩(linear embedding) 레이어를 통과시켜 고정된 크기의 임베딩 벡터로 변환
- 3. Position Encoding:** 각 패치의 위치 정보를 주입하기 위해 위치 인코딩(position encoding)을 추가. 이는 NLP에서의 Transformer와 동일한 개념.
- 4. Transformer Encoder:** 앞서 생성한 임베딩 벡터들은 Transformer 인코더에 입력됩니다. Transformer 인코더는 여러 개의 attention 및 feed-forward 네트워크 레이어로 구성.
- 5. Classification Head:** Transformer의 최종 출력은 분류 작업을 위한 MLP 레이어(또는 다른 헤더)에 입력

입력 Patch 구성



Linear Projection of Patch



CLIP

(Contrastive Language-Image Pre-training)

Learning Transferable Visual Models From Natural Language
Supervision

사용 dataset

- MS-COCO, Visual Genome: 고품질/소규모 데이터 (100,000개)
- YFCC100M (100M) → filtering 후 고품질 15M개 사용
- 인터넷에 공개되어 있는 400M개의 (image, text) pair로 새로운 dataset 구성
- 텍스트 query는 위키피디아에서 100번 이상 등장하는 단어 50만개로 구축
- 데이터 균형을 위해 query당 이미지-텍스트 pair 개수를 최대 2만개로 제한
- 위 3가지 데이터로 contrastive learning (대조학습) 수행

* constrasive learning 은 서로 다른 두 데이터 포인트가 얼마나 다른지 (또는 유사한지)를 비교하거나 측정하여 유사한 데이터 포인트들이 가까이 있도록 임베딩 공간을 구성

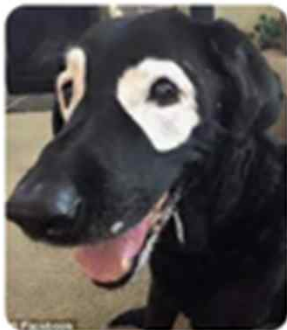
Data example

Google

a black dog with white spots


Dreamstime.com
Black Dog with White Spots...


Quora
What are some dog breeds that ar...

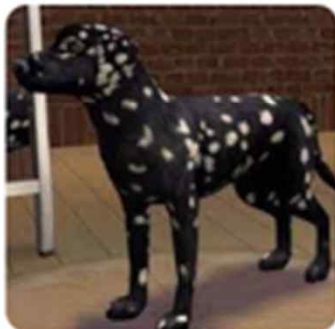

Daily Mail
Rowdy the Labrador's viti...


Can Stock Photo
Puppy black and white with spots. C...

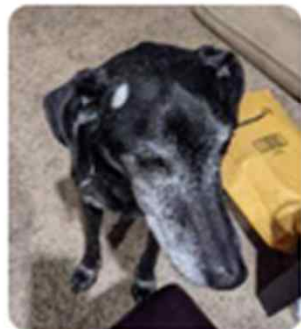

Adobe Stock
Black dog with white spots on the legs, chest ...


PetHelpful
90+ Unique Names for Dogs With Spot...

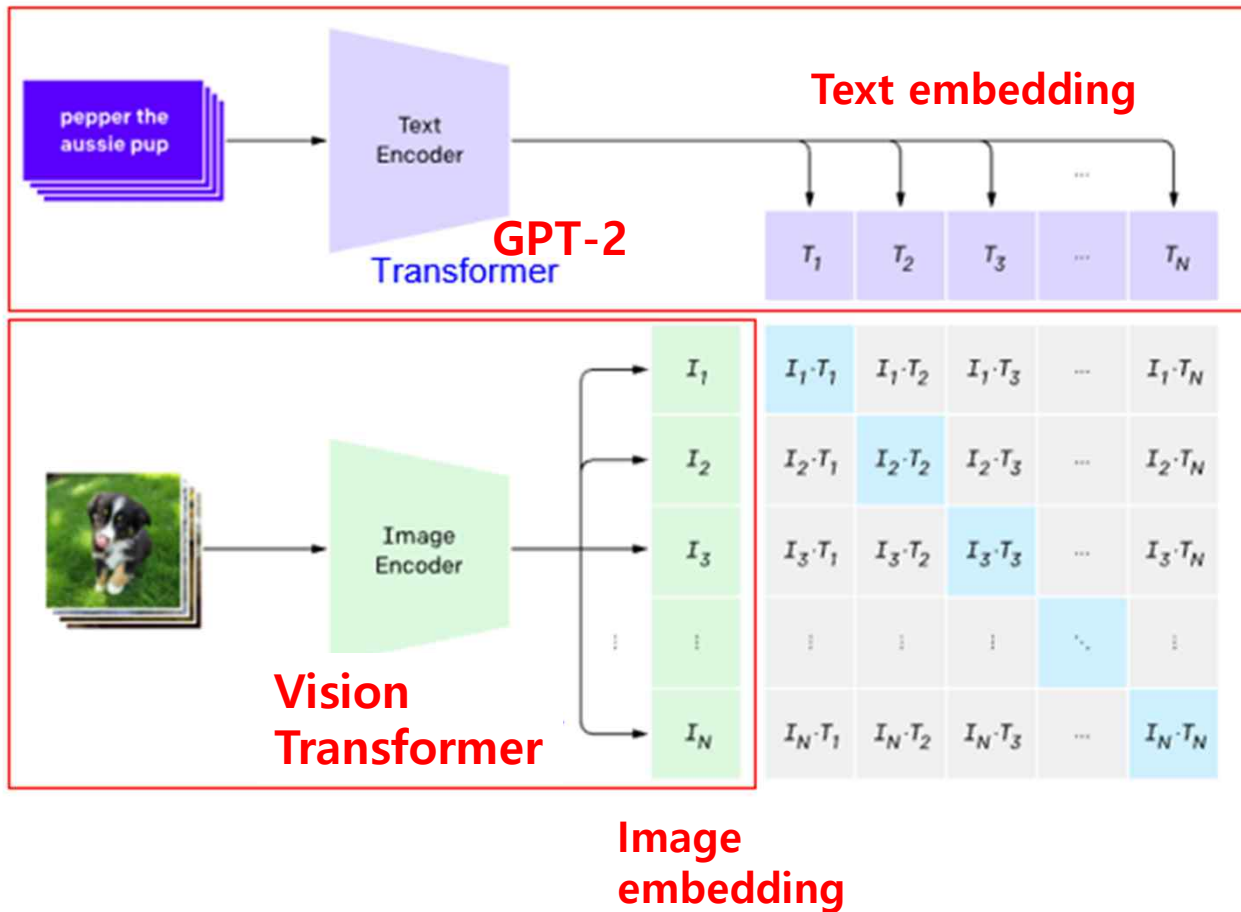

Alamy
Black dog with white spots on the legs...


Reddit
Black Dalmatian With White S...


Shutterstock
Black Dog White Spots Looks Yell...


Reddit
My all-black dog develope...

Contrastive Pre-Training



- N개의 이미지, 텍스트를 각각 encoder를 통과시켜 embedding vector 계산 (N: batch 사이즈)
- NxN matrix의 대각선 부분이 positive pair, 그 외 부분은 모두 negative pair (N개의 positive pair, $N^2 - N$ 개의 negative pair)
- Positive pair의 cosine 유사도는 maximize 하고, negative pair의 cosine 유사도는 minimize 하도록 train

MNIST

correct label: 7

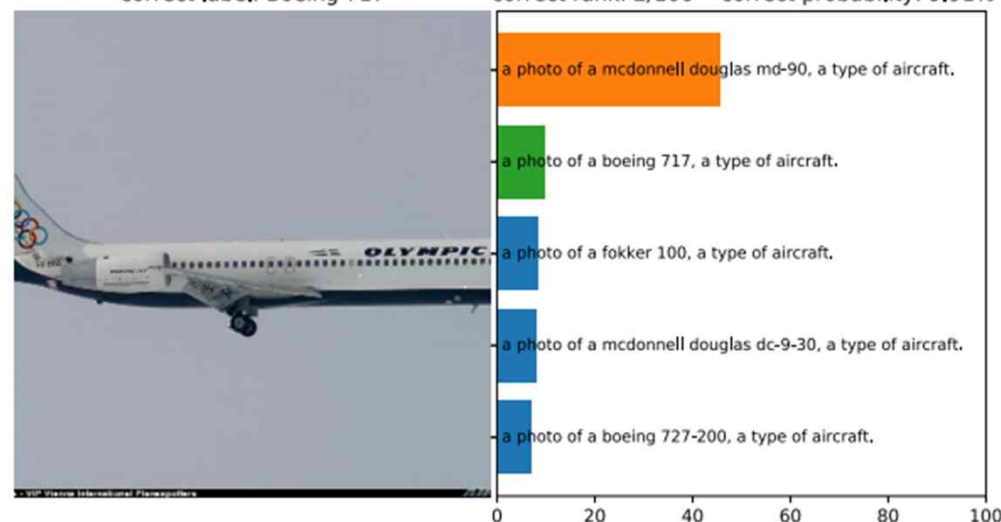
correct rank: 1/10 correct probability: 85.32%



FGVC Aircraft

correct label: Boeing 717

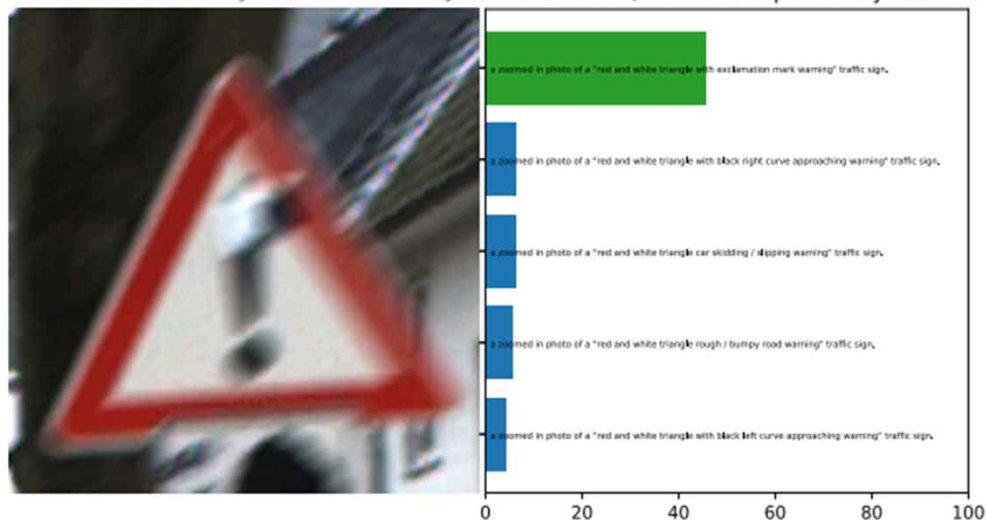
correct rank: 2/100 correct probability: 9.91%



German Traffic Sign Recognition Benchmark (GTSRB)

correct label: red and white triangle with exclamation mark warning

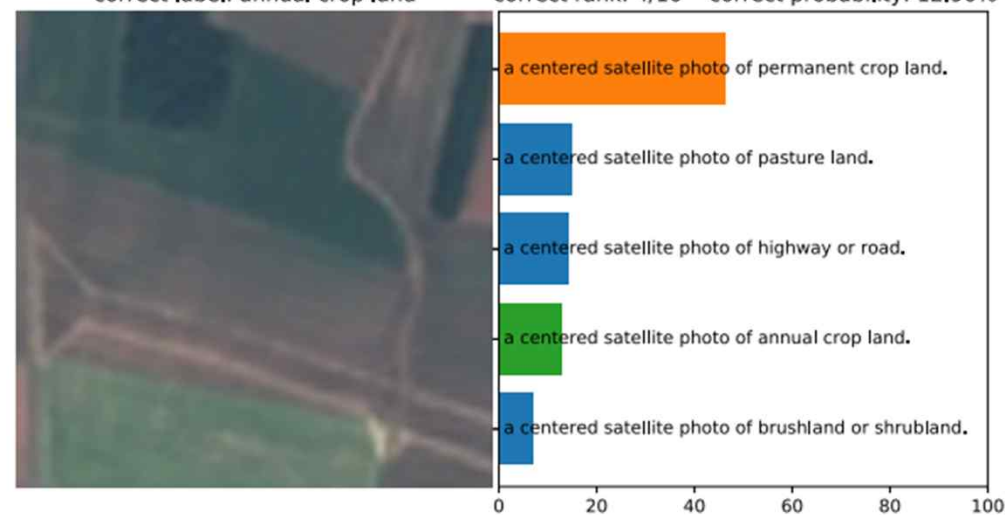
correct rank: 1/43 correct probability: 45.75%



EuroSAT

correct label: annual crop land

correct rank: 4/10 correct probability: 12.90%



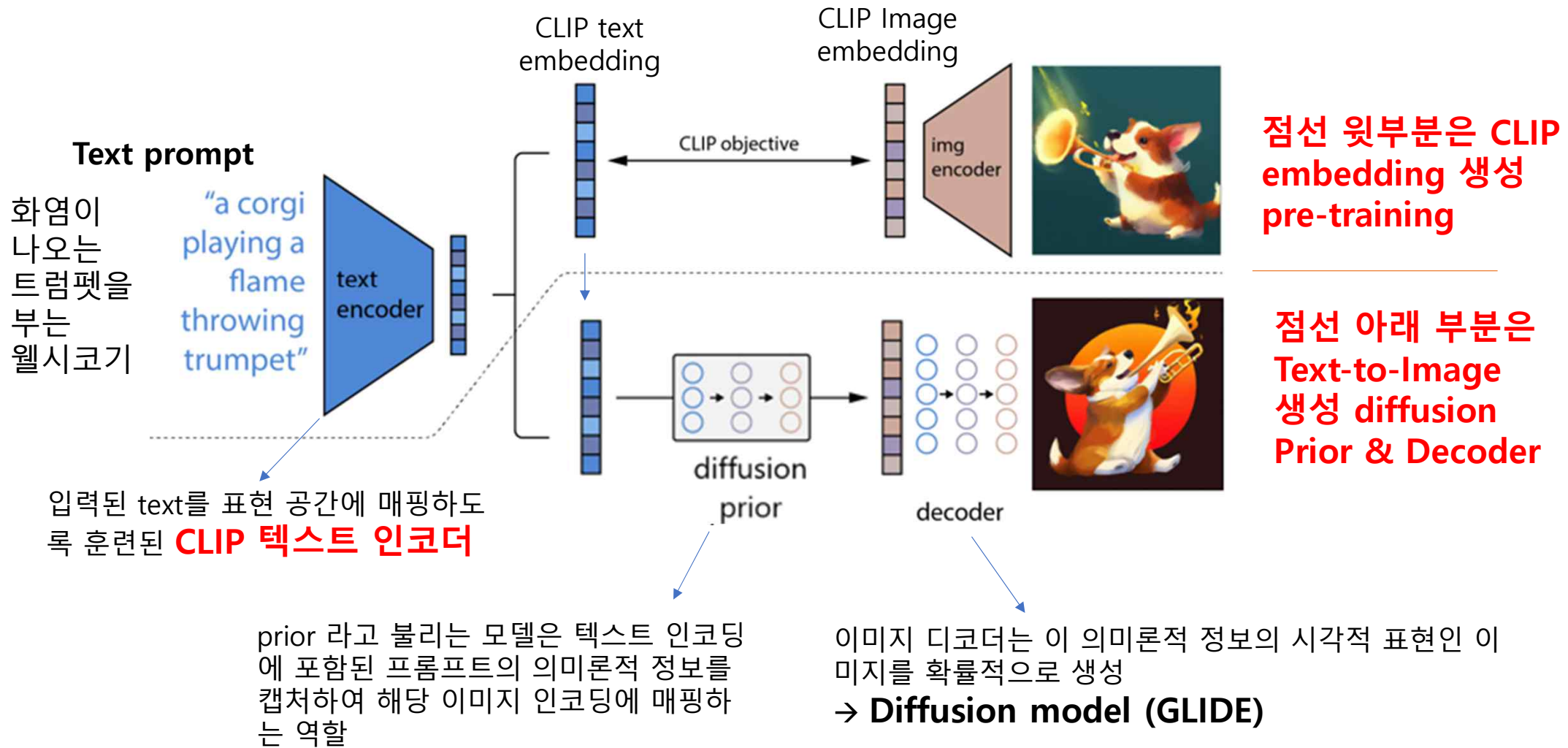
실습: 910_Interacting_with_CLIP

- CLIP model을 download 받아 실행
- 임의의 이미지와 텍스트 간의 유사성 계산
- Zero shot 이미지 분류 수행 방법 이해



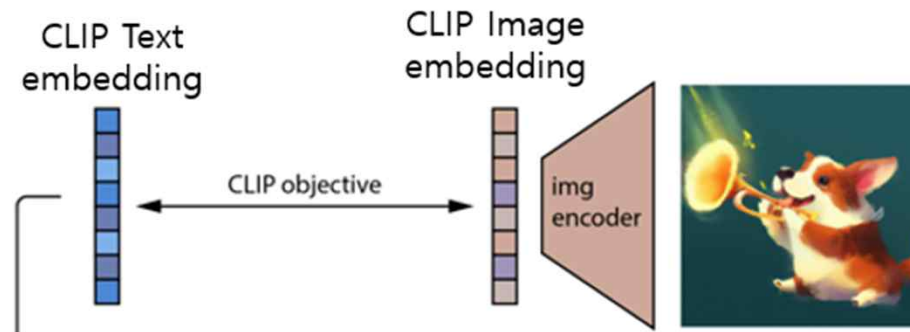
a person looking at a camera on a tripod	0.30	0.21	0.21	0.14	0.19	0.19	0.16	0.20
a rocket standing on a launchpad	0.20	0.30	0.18	0.14	0.19	0.17	0.16	0.17
a facial photo of a tabby cat	0.16	0.12	0.31	0.17	0.12	0.12	0.12	0.15
a cup of coffee on a saucer	0.17	0.12	0.18	0.29	0.15	0.14	0.12	0.15
a portrait of an astronaut with the American flag	0.17	0.22	0.17	0.15	0.28	0.13	0.15	0.16
a page of text about segmentation	0.20	0.16	0.20	0.20	0.15	0.35	0.16	0.20
a red motorcycle standing in a garage	0.12	0.16	0.15	0.13	0.15	0.14	0.32	0.16
a black-and-white silhouette of a horse	0.21	0.15	0.15	0.15	0.11	0.17	0.17	0.35

DALL-E2 Structure



DALL-E2 학습 방법. (1단계) – CLIP 모델 학습

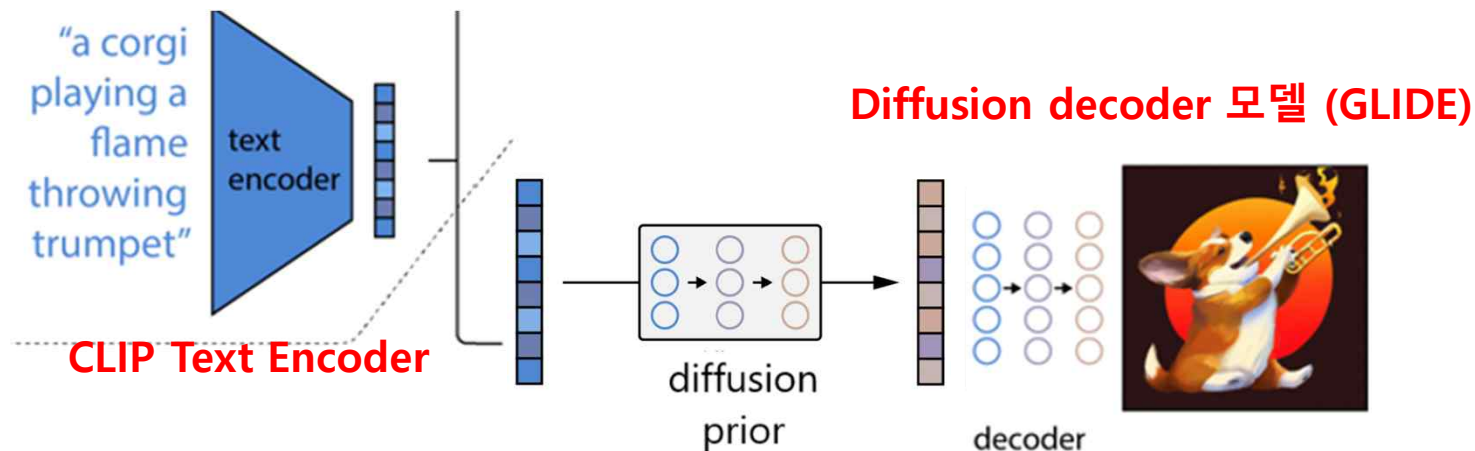
- DALL-E2는 OpenAI의 CLIP 모델을 이용하여 텍스트 의미론과 시각적 표현 사이의 연결을 학습. 즉, CLIP은 이미지가 주어진 캡션을 예측하는 대신 주어진 캡션이 이미지와 얼마나 관련이 있는지 학습



a corgi playing a flame throwing trumpet
(화염이 나오는 트럼펫을 부는 웰시코기)

DALL-E2 학습 방법. (2단계) – Diffusion model 학습

- 학습 후 CLIP 모델은 고정(frozen)되고 DALL-E 2는 다음 작업으로 이동
- CLIP이 학습한 이미지 인코딩을 뒤집어 이미지를 생성하는 방법을 학습
→ Diffusion Model(GLIDE)을 decoder로 이용하여 이미지 생성.



프롬프트 text를 CLIP의 text embedding을 사용하여 가장 가능성 높은
CLIP image embedding으로 mapping 하는 모델

감사합니다.